

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное
учреждение высшего образования
«Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий и искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

Отчет по курсовой работе
по дисциплине «Теория графов»
Реализация алгоритмов теории графов
для лабораторных работ №1–5

Студент,
группа 5130201/40001

_____ Ови Мд Шамин Ясир

Преподаватель,

_____ Востров Алексей Владимирович

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Математическое описание	5
1.1 Граф	5
1.2 Нормальное распределение	5
1.3 Экцентриситет, центр и диаметральные вершины	5
1.4 Метод Шимбелла	6
1.5 Обход графа поиском в глубину	6
1.6 Алгоритм Флойда-Уоршалла	7
1.7 Алгоритм Форда-Фалкерсона	7
1.8 Остов	7
1.9 Матричная теорема Кирхгофа	8
1.10 Алгоритм Борувки	8
1.11 Кодирование Прюфера	8
1.12 Минимальное рёберное покрытие	9
1.13 Эйлеров граф	9
1.14 Фундаментальная система разрезов	9
2 Особенности реализации	11
2.1 Структура проекта	11
2.2 Класс Graph	12
2.3 Генерация распределения, графа и весовой матрицы	12
2.4 Лабораторная работа №1	13
2.5 Лабораторная работа №2	14
2.6 Лабораторная работа №3	14
2.7 Лабораторная работа №4	15
2.8 Лабораторная работа №5	15
2.9 Функция main и меню программы	16
2.10 Подробное описание функций	16
3 Результаты работы программы	27
3.1 Задания 1–11: ориентированный граф на 5 вершинах	27
3.2 Задания 12–17: неориентированный граф на 7 вершинах	39
3.3 Обработка ошибочных ситуаций	47
Заключение	48
Список использованной литературы	50
Приложение А. Общие модули и объединяющая программа	51
Приложение Б. Исходный код лабораторной работы №1	67
Приложение В. Исходный код лабораторной работы №2	74
Приложение Г. Исходный код лабораторной работы №3	79
Приложение Д. Исходный код лабораторной работы №4	87
Приложение Е. Исходный код лабораторной работы №5	99

Введение

Отчёт состоит из описания 5 лабораторных работ. Каждая работа включает в себя реализацию различных алгоритмов на случайно созданных связных ациклических графах. Генерация графов осуществляется с использованием *нормального распределения* (по справочнику Вадзинского [7]).

ЛАБОРАТОРНАЯ РАБОТА № 1

1. Сформировать случайным образом связный ациклический ориентированный граф в соответствии с *нормальным распределением* (параметры распределения задаются как константы, подобранные экспериментально) с вводимым пользователем количеством вершин. Распределение взято из справочника Вадзинского [7]. Генерация происходит по степеням вершин.
2. Посчитать эксцентриситеты вершин, выделить центр графа и диаметральные вершины.
3. Реализовать метод Шимбелла для сгенерированной с помощью заданного распределения весовой матрицы (пользователь вводит количество рёбер). В ответе представить минимальный и/или максимальный путь в виде матрицы. Предусмотреть генерацию только положительных, только отрицательных и смешанных весов на выбор пользователя.
4. Определить возможность построения маршрута от одной заданной точки до другой (вершины вводит пользователь) и указывать количество таковых маршрутов.

Вариант: нормальное распределение (Normal distribution).

ЛАБОРАТОРНАЯ РАБОТА № 2

1. Для заданных графов (случайно сгенерированных в предыдущей работе) выполнить *обход вершин графа поиском в глубину*.
2. Сгенерировать весовую матрицу (положительную или с отрицательными весами — выбирает пользователь) и найти кратчайший путь для двух выбранных пользователем вершин с помощью *алгоритма Флойда-Уоршалла*. В результате выводится матрица расстояний, а также сам путь в виде последовательности вершин.
3. Сравнить скорости работы реализованных алгоритмов (по количеству итераций).

Вариант: (с) обход вершин графа поиском в глубину; (j) алгоритм Флойда-Уоршалла.

ЛАБОРАТОРНАЯ РАБОТА № 3

1. На основе сгенерированного ориентированного графа получить случайным образом матрицы пропускных способностей и стоимости.
2. Для полученного графа найти максимальный поток по *алгоритму Форда-Фалкерсона*.
3. Вычислить заданный поток минимальной стоимости (в качестве величины потока брать значение, равное $\lfloor 2/3 \cdot \text{max} \rfloor$, где max — максимальный поток). Использовать ранее реализованный алгоритм Флойда-Уоршалла.

ЛАБОРАТОРНАЯ РАБОТА № 4

1. Для заданных неориентированных графов (случайно сгенерированных в первой работе) найти число остовных деревьев, используя матричную теорему Кирхгофа.
2. Построить минимальный по весу остов для сгенерированного (неориентированного) взвешенного графа, используя *алгоритм Борувки*. Полученный остов закодировать с помощью кода Прюфера и декодировать его (с сохранением весов).
3. Реализовать на исходном графе и полученном остове (по выбору пользователя) *нахождение минимального рёберного покрытия*.

Вариант: (b) алгоритм Борувки; (g) нахождение минимального рёберного покрытия.

ЛАБОРАТОРНАЯ РАБОТА № 5

1. Для заданных неориентированных графов (случайно сгенерированных в первой работе) проверить, является ли граф эйлеровым. Если нет — модифицировать граф (логировать, что изменено). Построить эйлеров цикл.
2. Построить кратчайший остов (алгоритмом Борувки из четвёртой работы). На основе

данного остова получить *фундаментальную систему разрезов* (вывести на экран); реализовать получение всех остальных разрезов на основе фундаментальных с помощью операции симметрической разности (выбранные разрезы вводит пользователь).

Вариант: 36, чётный — фундаментальная система разрезов.

Лабораторные работы выполнены на языке C++ в среде разработки CLion.

1 Математическое описание

1.1 Граф

Простой граф $G(V, E)$ есть совокупность двух множеств — непустого множества V и множества E неупорядоченных пар различных элементов множества V . Множество V называется множеством вершин, множество E называется множеством рёбер.

$$G(V, E) = \langle V, E \rangle, \quad V \neq \emptyset, \quad E \subseteq V \times V, \quad \{v, v\} \notin E, \quad v \in V,$$

то есть множество E состоит из 2-элементных подмножеств множества V .

Сопутствующие термины:

- Вершина графа G есть элемент множества вершин $v \in V(G)$;
- Ребро графа G есть элемент множества рёбер $e \in E(G)$, или $e = \{v_1, v_2\}$, где $v_1 \in V(G)$, $v_2 \in V(G)$;
- Элементами графа G называются его вершины $v \in V(G)$ и рёбра $e \in E(G)$ графа;
- Вершина v инцидентна ребру e , если $v \in e$; тогда ещё говорят, что e есть ребро при v ;
- Соседние (смежные) вершины есть такие вершины v_1 и v_2 , что $\{v_1, v_2\} \in E(G)$;
- Степень вершины $d(v)$ есть количество инцидентных ей рёбер.

1.2 Нормальное распределение

Нормальное (гауссово) распределение — непрерывное распределение вероятностей. Плотность вероятности:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right),$$

где μ — математическое ожидание, σ^2 — дисперсия.

Для генерации случайных степеней вершин используется приближённый метод из справочника Вадзинского [7]. Случайная величина X , имеющая нормальное распределение $\mathcal{N}(\mu, \sigma^2)$, аппроксимируется следующим образом:

$$X \approx \sqrt{\frac{12}{n}} \left(\sum_{i=1}^n r_i - \frac{n}{2} \right),$$

где $r_i \sim U(0, 1)$ — независимые равномерно распределённые на $[0, 1]$ случайные числа, n — количество слагаемых (параметр точности аппроксимации). Распределение взято из справочника по вероятностным распределениям [7].

1.3 Экцентриситет, центр и диаметральные вершины

Экцентриситетом вершины v в графе G называется максимальное расстояние от v до остальных вершин:

$$\varepsilon(v) = \max_{u \in V} d(v, u),$$

где $d(v, u)$ — длина кратчайшего пути между вершинами v и u .

Радиусом графа называется минимальный экцентриситет среди всех вершин:

$$\text{rad}(G) = \min_{v \in V} \varepsilon(v).$$

Диаметром графа называется максимальный эксцентриситет:

$$\text{diam}(G) = \max_{v \in V} \varepsilon(v).$$

Центром графа называется множество вершин с минимальным эксцентриситетом, равным радиусу:

$$Z(G) = \{v \in V \mid \varepsilon(v) = \text{rad}(G)\}.$$

Диаметральными называются вершины с эксцентриситетом, равным диаметру графа. Эксцентриситеты вычисляются с помощью поиска в ширину (BFS) из каждой вершины за время $O(V(V + E))$.

1.4 Метод Шимбелла

Метод Шимбелла позволяет находить минимальные (максимальные) пути между вершинами, состоящие из заданного количества рёбер.

- Операция умножения двух величин a и b при возведении матрицы в степень соответствует их алгебраической сумме, то есть:

$$a * b = b * a \rightarrow a + b = b + a, \quad a * 0 = 0 * a = 0 \rightarrow a + 0 = 0.$$

- Операция сложения двух величин a и b заменяется выбором из этих величин максимума или минимума, то есть:

$$a + b = b + a = \min(\max)\{a, b\}.$$

Нули при этом игнорируются.

$$w_{ij} = \min(\max(w_{i1} + w_{1j}), \dots, (w_{in} + w_{nj})).$$

1.5 Обход графа поиском в глубину

Обход в глубину — один из основных методов обхода графа, часто используемый для проверки связности, поиска цикла и компонент сильной связности и для топологической сортировки.

Общая идея алгоритма состоит в следующем: для каждой не пройденной вершины необходимо найти все не пройденные смежные вершины и повторить поиск для них.

- Выбираем любую вершину из ещё не пройденных, обозначим её как u ;
- Запускаем процедуру $\text{dfs}(u)$:
 - Помечаем вершину u как пройденную;
 - Для каждой не пройденной смежной с u вершины (назовём её v) запускаем $\text{dfs}(v)$;
- Повторяем шаги 1 и 2, пока все вершины не окажутся пройденными.

Время работы алгоритма оценивается как $O(V + E)$.

1.6 Алгоритм Флойда-Уоршалла

Алгоритм Флойда-Уоршалла предназначен для нахождения кратчайших путей между всеми парами вершин во взвешенном графе, в том числе с отрицательными весами рёбер (при отсутствии отрицательных циклов).

Обозначим через $d^{(k)}[i][j]$ длину кратчайшего пути от вершины i до вершины j , проходящего только через промежуточные вершины из множества $\{1, 2, \dots, k\}$. Рекуррентное соотношение:

$$d^{(k)}[i][j] = \min(d^{(k-1)}[i][j], d^{(k-1)}[i][k] + d^{(k-1)}[k][j]).$$

Начальные условия:

$$d^{(0)}[i][j] = \begin{cases} w(i, j), & \text{если ребро } (i, j) \text{ существует,} \\ \infty, & \text{иначе.} \end{cases}$$

В результате $d^{(n)}[i][j]$ содержит длину кратчайшего пути от i до j . В ответе выводится матрица расстояний, а также сам путь в виде последовательности вершин. Трудоемкость алгоритма — $O(V^3)$.

1.7 Алгоритм Форда-Фалкерсона

Алгоритм Форда-Фалкерсона решает задачу нахождения максимального потока в сети. Идея алгоритма заключается в следующем:

- Изначально величине потока присваивается значение 0 для всех u и v в транспортной сети.
- Затем величина потока итеративно увеличивается посредством поиска увеличивающего пути от источника s к стоку t .
- Процесс повторяется, пока можно найти увеличивающий путь.

В реализованном варианте Эдмондса-Карпа, где увеличивающие пути ищутся поиском в ширину, трудоемкость алгоритма составляет $O(VE^2)$.

Алгоритм поиска заданного потока минимальной стоимости может использовать любой из алгоритмов поиска кратчайшего пути из истока в сток. Для заданного суммарного потока n последовательно n раз ищется кратчайший по стоимости путь потока 1 и далее на этом пути пропускная способность всех рёбер уменьшается на 1. Далее n таких стоимостей суммируются. Целевая функция:

$$\min \left(\sum_{(u,v) \in E} c(u, v) \cdot f(u, v) \right),$$

где E — множество всех рёбер в графе, $c(u, v)$ — стоимость потока по ребру (u, v) , $f(u, v)$ — поток по ребру (u, v) . Асимптотика алгоритма — $O(n \cdot (V + E))$.

1.8 Остов

Пусть $G = (V, E)$ — граф. Остовный подграф графа $G = (V, E)$ — подграф, содержащий все вершины. Остовный подграф, являющийся деревом, называется остовом. Несвязный граф не имеет остова. Связный граф может иметь много остовов.

Минимальное остовное дерево графа G — остовное дерево $T = (V, E_T)$ графа G , сумма весов рёбер которого минимальна.

1.9 Матричная теорема Кирхгофа

Матрица Кирхгофа. Матрица B графа G размера $n \times n$ определяется следующим образом:

$$B[i, j] = \begin{cases} -1, & \text{если вершины с номерами } i \text{ и } j \text{ смежны;} \\ 0, & \text{если } i \neq j \text{ и вершины не смежны;} \\ \deg(v_i), & \text{если } i = j. \end{cases}$$

Свойства матрицы Кирхгофа:

1. Суммы элементов в каждой строке и каждом столбце матрицы равны 0.
2. Алгебраические дополнения всех элементов матрицы равны между собой.
3. Определитель матрицы Кирхгофа равен нулю.

Матричная теорема Кирхгофа. Число остовных деревьев в связном графе G порядка $n \geq 2$ равно алгебраическому дополнению любого элемента матрицы Кирхгофа B графа G .

1.10 Алгоритм Борувки

Алгоритм Борувки — алгоритм построения минимального остовного дерева взвешенного связного неориентированного графа, предложенный Отакаром Борувкой в 1926 году.

Алгоритм работает следующим образом:

1. Изначально каждая вершина образует отдельную компоненту (лес из одиночных вершин).
2. Для каждой компоненты находится ребро минимального веса, соединяющее её с другой компонентой.
3. Все найденные рёбра добавляются в остов, соответствующие компоненты объединяются.
4. Шаги 2–3 повторяются, пока не останется одна компонента.

На каждой итерации число компонент уменьшается как минимум вдвое, поэтому выполняется не более $O(\log V)$ итераций. Общая трудоёмкость — $O(E \log V)$.

1.11 Кодирование Прюфера

Алгоритм кодирования Прюфера состоит из следующих шагов. Пока количество вершин больше двух:

- Выбираем лист v с минимальным номером.
- В код Прюфера добавляем номер вершины, смежной с v .
- Вершина v и инцидентное ей ребро удаляются из дерева.

Полученная последовательность называется кодом Прюфера для заданного дерева.

Алгоритм декодирования: имеется массив кода Прюфера размера $n - 2$ и массив всех вершин графа $[1 \dots n]$. Далее $n - 2$ раза повторяется следующая процедура: берётся первый элемент массива кода Прюфера, и в массиве вершин производится поиск наименьшей вершины, не содержащейся в коде. Найденная вершина и текущий элемент кода составляют ребро дерева. Данные вершины удаляются из соответствующих массивов. В конце в массиве с вершинами остаётся две вершины — они составляют последнее ребро дерева. Асимптотика алгоритмов — $O(n)$.

1.12 Минимальное рёберное покрытие

Рёберным покрытием графа $G = (V, E)$ называется такое подмножество рёбер $S \subseteq E$, что каждая вершина графа инцидентна хотя бы одному ребру из S . **Минимальным рёберным покрытием** называется рёберное покрытие наименьшей мощности.

Связь с максимальным паросочетанием выражается теоремой Галлаи: для графа без изолированных вершин

$$\alpha'(G) + \beta'(G) = |V(G)|,$$

где $\alpha'(G)$ — размер максимального паросочетания, $\beta'(G)$ — размер минимального рёберного покрытия.

Алгоритм нахождения минимального рёберного покрытия:

1. Найти максимальное паросочетание M в графе.
2. Для каждой вершины, не покрытой паросочетанием M , добавить в покрытие произвольное инцидентное ей ребро.

Трудоёмкость определяется сложностью нахождения максимального паросочетания.

1.13 Эйлеров граф

Эйлеров граф — это граф, в котором существует эйлеров цикл, то есть простой цикл, проходящий по всем рёбрам графа.

Граф является эйлеровым тогда и только тогда, когда он связан и степени всех его вершин чётны.

Эйлеровым путём называется простой путь, содержащий все рёбра. В данной работе производится поиск эйлерова цикла. Сначала осуществляется проверка, является ли граф эйлеровым. Если нет, то граф достраивается так, чтобы степень каждой вершины была чётной.

Для поиска эйлерова цикла используется алгоритм, основанный на обходе в глубину (DFS):

- Выбрать произвольную вершину графа и поместить её в стек.
- Пока стек не пуст, повторять следующие действия:
 - извлечь вершину из стека;
 - если у этой вершины есть непосещённые соседние вершины, выбрать одну из них и поместить её в стек;
 - если у вершины нет непосещённых соседних вершин, добавить её в список эйлерова цикла.
- Вернуться к списку эйлерова цикла в обратном порядке.

Асимптотика алгоритма поиска эйлерова цикла оценивается как $O(V + E)$.

1.14 Фундаментальная система разрезов

Разрезом (разрезающим множеством) в графе $G = (V, E)$ называется такое множество рёбер $C \subseteq E$, удаление которых увеличивает число компонент связности графа.

Фундаментальным разрезом относительно остовного дерева T называется разрез, соответствующий одному ребру дерева $t \in E_T$: при удалении t дерево T распадается на две компоненты T_1 и T_2 ; фундаментальный разрез C_t — множество всех рёбер исходного графа G , у которых одна конечная вершина лежит в T_1 , а другая в T_2 .

Фундаментальная система разрезов строится для выбранного остовного дерева T и содержит ровно $|V| - 1$ разрезов (по числу рёбер дерева).

Все остальные разрезы графа можно получить с помощью операции симметрической разности фундаментальных разрезов:

$$C = C_{t_1} \Delta C_{t_2} \Delta \cdots \Delta C_{t_k},$$

где Δ — операция симметрической разности множеств.

2 Особенности реализации

2.1 Структура проекта

Программа реализована на языке C++17 и имеет модульную структуру. Общие структуры данных и функции генерации вынесены в каталог **shared**, алгоритмы разделены по лабораторным работам, а каталог **lab_combined** содержит объединяющую консольную программу с меню. Полный исходный код приведён в приложениях А–Е.

```
1 graph-theory-labs/
2 |-- CMakeLists.txt
3 |-- README.md
4 |-- shared/
5 |   |-- constants.h
6 |   |-- graph.h
7 |   |-- graph.cpp
8 |   |-- distribution.h
9 |   |-- distribution.cpp
10 |   |-- dag_generator.h
11 |   |-- dag_generator.cpp
12 |   |-- weight_matrix.h
13 |   '-- weight_matrix.cpp
14 |-- lab1/
15 |   |-- eccentricity.h
16 |   |-- eccentricity.cpp
17 |   |-- shimbell.h
18 |   |-- shimbell.cpp
19 |   |-- path_counter.h
20 |   '-- path_counter.cpp
21 |-- lab2/
22 |   |-- dfs.h
23 |   |-- dfs.cpp
24 |   |-- floyd_warshall.h
25 |   '-- floyd_warshall.cpp
26 |-- lab3/
27 |   |-- ford_fulkerson.h
28 |   |-- ford_fulkerson.cpp
29 |   |-- min_cost_flow.h
30 |   '-- min_cost_flow.cpp
31 |-- lab4/
32 |   |-- kirchhoff.h
33 |   |-- kirchhoff.cpp
34 |   |-- boruvka.h
35 |   |-- boruvka.cpp
36 |   |-- edge_cover.h
37 |   |-- edge_cover.cpp
38 |   |-- prufer.h
39 |   '-- prufer.cpp
40 |-- lab5/
41 |   |-- eulerian.h
42 |   |-- eulerian.cpp
43 |   |-- fundamental_cutsets.h
44 |   '-- fundamental_cutsets.cpp
45 '-- lab_combined/
46 |   |-- CMakeLists.txt
47 |   '-- main.cpp
```

Листинг 1. Структура проекта

Каталог **shared** содержит класс графа, генератор нормального распределения, гене-

ратор графа и генератор весовой матрицы. Каталоги `lab1–lab5` содержат алгоритмы соответствующих лабораторных работ. Файл `lab_combined/main.cpp` связывает все модули в единое меню пользователя.

2.2 Класс Graph

Класс `Graph` является базовой структурой для хранения ориентированного или неориентированного графа. Его объявление приведено в листинге 3, а реализация методов – в листинге 4. В классе хранятся количество вершин `n`, признак ориентированности `directed` и матрица смежности `adj`.

Функция `Graph(int n, bool directed)` создаёт пустой граф нужного типа.

Вход: количество вершин `n`, логический признак ориентированности `directed`.

Выход: объект графа с матрицей смежности размера $n \times n$, заполненной нулями.

Функция `addEdge` добавляет ребро или дугу между двумя вершинами. Если граф неориентированный, то матрица смежности заполняется симметрично.

Вход: номера вершин `u` и `v`.

Выход: изменённая матрица смежности графа.

Функция `hasEdge` проверяет существование ребра или дуги.

Вход: номера вершин `u` и `v`.

Выход: значение `true`, если ребро существует, иначе `false`.

Функция `neighbors` возвращает все вершины, смежные с заданной. Для ориентированного графа возвращаются исходящие соседи, для неориентированного – все соседние вершины.

Вход: номер вершины `u`.

Выход: вектор номеров соседних вершин.

Функции `printAdjMatrix` и `printFull` используются для вывода графа в консоль. Первая выводит только матрицу смежности, а вторая также печатает список смежности, список рёбер и, при наличии, весовую матрицу.

Вход: объект графа `g`, для `printFull`, необязательная весовая матрица.

Выход: текстовое представление графа в консоли.

2.3 Генерация распределения, графа и весовой матрицы

Функции генерации распределения описаны в листингах 5 и 6. Для генерации используется приближённая формула нормального распределения через сумму равномерно распределённых случайных величин.

Функция `normalSample` генерирует одно случайное число, приближённо распределённое по нормальному закону.

Вход: количество равномерных случайных величин `n`, используемых в аппроксимации.

Выход: вещественное число, соответствующее приближённому нормальному распределению.

Функция `sampleDegree` переводит значение нормального распределения в допустимую степень вершины.

Вход: максимальная допустимая степень `maxDeg`.

Выход: целое число в диапазоне от 1 до `maxDeg`.

Функция `generateDegreeSequence` строит последовательность степеней для заданного количества вершин. Сумма степеней корректируется до значения $2(n - 1)$, что соответствует связному ациклическому неориентированному графу.

Вход: количество вершин `numVertices`.

Выход: вектор степеней вершин.

Генерация графа реализована в файлах `dag_generator.h` и `dag_generator.cpp`, приведённых в листингах 7 и 8. Основная функция `generateDAG` создаёт ориентированный ациклический граф или неориентированный граф, полученный на основе ориентированной модели.

Вход: количество вершин `n` и признак ориентированности `directed`.

Выход: объект `Graph` со сгенерированной матрицей смежности.

Для ориентированного графа сначала строится случайный топологический порядок вершин, затем добавляется путь, обеспечивающий связность, после чего добавляются дополнительные дуги только вперёд по топологическому порядку. Благодаря этому граф остаётся ациклическим. Для неориентированного графа направления дуг игнорируются, и граф становится симметричным.

Генерация весовой матрицы реализована в листингах 9 и 10. Функция `generateWeightMatrix` заполняет веса только для существующих рёбер графа.

Вход: граф `g` и режим генерации весов `mode`.

Выход: матрица весов, где для отсутствующих рёбер используется значение `INF`.

Режим `POSITIVE` создаёт только положительные веса, `NEGATIVE` – только отрицательные, а `MIXED` – смешанные веса. Для неориентированного графа веса записываются симметрично.

2.4 Лабораторная работа №1

В первой лабораторной работе реализованы вычисление метрических характеристик графа, метод Шимбелла и поиск маршрутов. Код расположен в файлах `eccentricity.cpp`, `shimbell.cpp` и `path_counter.cpp`; полные тексты приведены в листингах 13, 15 и 17.

Функция `bfs` выполняет поиск в ширину из заданной вершины и используется для вычисления расстояний до остальных вершин.

Вход: граф `g`, начальная вершина `src`.

Выход: вектор расстояний от вершины `src`; значение `-1` означает, что вершина недостижима.

Функция `computeMetrics` вычисляет эксцентриситеты, радиус, диаметр, центр и диаметральные вершины графа. Для каждой вершины запускается `bfs`, после чего из полученных расстояний выбирается максимальное достижимое расстояние.

Вход: граф `g`.

Выход: структура `GraphMetrics`, содержащая матрицу расстояний, эксцентриситеты, радиус, диаметр, центр и диаметральные вершины.

Функция `shimbell` реализует метод Шимбелла для путей, состоящих ровно из k рёбер. Для минимальных путей используется тропическое умножение с выбором минимума, а для максимальных – с выбором максимума.

Вход: весовая матрица `W`, количество рёбер `k`.

Выход: пара матриц: матрица минимальных путей и матрица максимальных путей.

Функция `findAllPaths` ищет все простые маршруты между двумя вершинами. Реализация использует рекурсивный поиск в глубину с массивом посещённых вершин, чтобы не допускать повторения вершин в одном маршруте.

Вход: граф `g`, начальная вершина `src`, конечная вершина `dst`.

Выход: вектор маршрутов, где каждый маршрут представлен последовательностью вершин.

Функции `pathExists` и `countPaths` являются вспомогательными. Первая проверяет наличие хотя бы одного маршрута, вторая возвращает количество найденных маршрутов. Функция `printPaths` выводит маршруты в консоль.

Вход: граф или уже найденный список маршрутов.

Выход: логическое значение, количество маршрутов или текстовый вывод в консоль.

2.5 Лабораторная работа №2

Во второй лабораторной работе реализованы обход графа поиском в глубину и алгоритм Флойда-Уоршалла. Код приведён в листингах 19 и 21.

Функция `dfs` выполняет обход графа в глубину из заданной вершины. В реализации используется стек, поэтому алгоритм работает итеративно. Дополнительно подсчитывается количество итераций.

Вход: граф `g`, стартовая вершина `start`.

Выход: порядок обхода, список недостижимых вершин и количество итераций, выведенные в консоль.

Функция `floydWarshall` находит кратчайшие пути между всеми парами вершин. Матрица `T` хранит расстояния, а матрица `H` хранит информацию для восстановления пути.

Вход: граф `g`, весовая матрица `W`.

Выход: структура `FloydResult`, содержащая матрицу расстояний, матрицу восстановления пути, количество итераций и признак отрицательного цикла.

Функция `printFloydResult` выводит результат работы алгоритма и восстанавливает путь между двумя выбранными пользователем вершинами. Если путь не существует, программа выводит соответствующее сообщение.

Вход: результат алгоритма Флойда-Уоршалла, исходная весовая матрица, количество вершин, начальная и конечная вершины.

Выход: матрицы `C`, `T`, `H`, кратчайший путь и его длина в консоли.

2.6 Лабораторная работа №3

В третьей лабораторной работе реализованы генерация матриц пропускных способностей и стоимостей, алгоритм Форда-Фалкерсона и поиск потока минимальной стоимости. Код расположен в листингах 23 и 25.

Функция `generateCapacityMatrix` создаёт матрицу пропускных способностей для существующих дуг ориентированного графа.

Вход: ориентированный граф `g`.

Выход: матрица пропускных способностей, где ноль означает отсутствие дуги.

Функция `generateCostMatrix` создаёт матрицу стоимости прохождения единицы потока по каждой дуге.

Вход: ориентированный граф `g`.

Выход: матрица стоимостей, соответствующая структуре графа.

Функция `fordFulkerson` реализует алгоритм Форда-Фалкерсона в варианте Эдмондса-Карпа, то есть увеличивающий путь ищется с помощью поиска в ширину. На каждом шаге находится пропускная способность пути, после чего остаточная сеть обновляется.

Вход: количество вершин, матрица пропускных способностей, исток `src`, сток `dst`.

Выход: структура `FFResult`, содержащая матрицу потока и значение максимального потока.

Функция `fordFulkersonSilent` выполняет тот же алгоритм, но без вывода промежуточных путей. Она используется внутри алгоритма потока минимальной стоимости.

Вход: количество вершин, матрица пропускных способностей, исток и сток.

Выход: значение максимального потока и матрица потока без печати в консоль.

Функция `minCostFlow` вычисляет поток минимальной стоимости для величины $\lfloor 2/3 \cdot \varphi_{\max} \rfloor$. Сначала находится максимальный поток, затем в остаточной сети последовательно выбираются кратчайшие по стоимости пути, найденные алгоритмом Флойда-Уоршалла.

Вход: количество вершин, матрица пропускных способностей, матрица стоимостей, исток и сток.

Выход: структура `MCFResult`, содержащая максимальный поток, требуемый поток,

достигнутый поток, итоговую стоимость и матрицу потока.

Функция `printMCResult` выводит итоговый поток минимальной стоимости и проверочный расчёт стоимости по дугам.

Вход: результат потока минимальной стоимости, матрица пропускных способностей, матрица стоимостей.

Выход: таблица дуг с положительным потоком и итоговая стоимость.

2.7 Лабораторная работа №4

В четвёртой лабораторной работе реализованы матричная теорема Кирхгофа, алгоритм Борувки, минимальное рёберное покрытие и код Прюфера. Полный код приведён в листингах [27](#), [29](#), [31](#) и [33](#).

Функция `countSpanningTrees` считает количество остовных деревьев графа. Для этого строится матрица Кирхгофа, удаляется первая строка и первый столбец, после чего вычисляется определитель полученной матрицы.

Вход: неориентированный граф `g`.

Выход: количество остовных деревьев графа.

Функция `boruvka` строит минимальное остовное дерево. На каждой итерации для каждой компоненты выбирается минимальное исходящее ребро, после чего компоненты объединяются.

Вход: неориентированный граф `g`, весовая матрица `W`.

Выход: структура `BoruvkaResult`, содержащая рёбра минимального остова и его суммарный вес.

Функция `minEdgeCover` находит минимальное рёберное покрытие. Сначала строится максимальное паросочетание, затем для непокрытых вершин добавляются инцидентные рёбра.

Вход: граф `g`, весовая матрица, признак работы с остовом или исходным графом, список рёбер остова.

Выход: структура `EdgeCoverResult`, содержащая рёбра покрытия и размер максимального паросочетания.

Функция `pruferEncode` кодирует минимальный остов кодом Прюфера. На каждом шаге удаляется лист с минимальным номером, а в код записывается его сосед. Вес удалённого ребра сохраняется отдельно.

Вход: список рёбер остова, количество вершин.

Выход: код Прюфера и список весов рёбер в порядке удаления.

Функция `pruferDecode` восстанавливает дерево по коду Прюфера и сохранённым весам. На каждом шаге выбирается минимальный лист, отсутствующий в текущем коде.

Вход: код Прюфера, список весов, количество вершин.

Выход: список рёбер восстановленного дерева с весами.

2.8 Лабораторная работа №5

В пятой лабораторной работе реализованы проверка графа на эйлеровость, построение эйлерова цикла и фундаментальная система разрезов. Код приведён в листингах [35](#) и [37](#).

Функция `buildEulerianCycle` проверяет связность графа и чётность степеней всех вершин. Если граф не является эйлеровым, программа пытается изменить его, добавляя или удаляя рёбра так, чтобы степени стали чётными и связность сохранилась.

Вход: неориентированный граф `g`.

Выход: структура `EulerianResult`, содержащая исходные степени, список нечётных вершин, список изменений и построенный эйлеров цикл.

Функция `hierholzer` строит эйлеров цикл после того, как граф стал эйлеровым. Алгоритм последовательно удаляет пройденные рёбра и формирует цикл в обратном порядке.

Вход: матрица смежности эйлерова графа.

Выход: последовательность вершин эйлерова цикла.

Функция `buildFundamentalCutsets` строит фундаментальную систему разрезов относительно минимального остова. Для каждого ребра остова оно временно удаляется, после чего находятся две компоненты дерева и все рёбра исходного графа, соединяющие эти компоненты.

Вход: исходный граф `g`, список рёбер минимального остова.

Выход: вектор фундаментальных разрезов.

Функция `printCutsetSymmetricDifference` вычисляет симметрическую разность выбранных фундаментальных разрезов. Ребро, встречающееся два раза, удаляется из результата, а ребро, встречающееся один раз, остаётся.

Вход: список фундаментальных разрезов и номера выбранных разрезов.

Выход: множество рёбер, полученное операцией симметрической разности.

2.9 Функция `main` и меню программы

Объединение всех лабораторных работ выполнено в файле `lab_combined/main.cpp`, полный код которого приведён в листинге 11. В программе используются глобальные переменные для хранения текущего графа, весовой матрицы, матриц пропускных способностей и стоимостей, минимального остова и результата кодирования Прюфера.

Функция `readInt` обеспечивает безопасный ввод целого числа. Если пользователь вводит некорректные данные, поток ввода очищается, и запрос повторяется.

Вход: текст приглашения `prompt`.

Выход: корректно считанное целое число.

Функции `requireGraph`, `requireWeight`, `requireFlowMatrices` и `requireMST` проверяют, были ли подготовлены необходимые данные перед запуском алгоритма.

Вход: текущее состояние программы.

Выход: сообщение об ошибке, если нужные данные отсутствуют.

Функции меню, например `menuGenerateGraph`, `menuShimbell`, `menuFloydWarshall`, `menuFordFulkerson`, `menuBoruvka` и другие, считывают параметры пользователя, проверяют корректность состояния программы и вызывают соответствующие алгоритмы.

Вход: выбор пользователя и дополнительные значения, введённые в консоль.

Выход: запуск выбранного алгоритма и вывод результата в консоль.

Функция `main` организует главный цикл программы. Пока пользователь не выберет пункт выхода, на экран выводится меню, считывается номер команды и вызывается соответствующая функция обработки.

Вход: команды пользователя из консольного меню.

Выход: последовательное выполнение лабораторных алгоритмов до завершения программы.

Таким образом, объединяющая программа позволяет выполнять лабораторные работы в одном интерфейсе: сначала генерируется граф, затем строятся необходимые матрицы, после чего пользователь может запускать алгоритмы из разных разделов работы.

2.10 Подробное описание функций

Ниже приведено более подробное описание основных функций программы. Для каждой функции указано её полное имя так, как оно используется в исходном коде, затем описаны назначение, входные данные и результат работы. Сам исходный код не дублируется в данном разделе, потому что он полностью приведён в приложениях А–Е; здесь

код используется как основа для объяснения реализации. Названия функций в данном подразделе являются гиперссылками на соответствующие листинги с исходным кодом.

Общие структуры данных и генерация

Функция `Graph::Graph()`

Конструктор по умолчанию создаёт пустой объект графа. Он нужен для глобальных переменных и ситуаций, когда граф будет сгенерирован позже через меню программы. Внутри конструктора количество вершин устанавливается равным нулю, а граф считается неориентированным до момента явного создания нового объекта.

Вход: входные параметры отсутствуют.

Выход: пустой объект `Graph`, для которого метод `isEmpty()` возвращает значение `true`.

Функция `Graph::Graph(int n, bool directed)`

Основной конструктор класса `Graph`. Он создаёт матрицу смежности размера $n \times n$ и заполняет её нулями. Параметр `directed` определяет, будет ли граф ориентированным. Благодаря хранению графа в виде матрицы смежности проверка наличия ребра выполняется за постоянное время.

Вход: количество вершин `n`; логический параметр `directed`, равный `true` для ориентированного графа и `false` для неориентированного.

Выход: объект графа с подготовленной матрицей смежности и сохранённым типом графа.

Функция `Graph::addEdge(int u, int v)`

Метод добавляет ребро между вершинами `u` и `v`. Если граф ориентированный, то в матрице смежности устанавливается только элемент `adj[u][v]`. Если граф неориентированный, дополнительно устанавливается симметричный элемент `adj[v][u]`, поэтому одно ребро хранится сразу в двух ячейках матрицы.

Вход: номера вершин `u` и `v` во внутренней нумерации, начиная с нуля.

Выход: изменённая матрица смежности текущего объекта `Graph`.

Функция `Graph::hasEdge(int u, int v) const`

Метод проверяет, существует ли ребро или дуга из вершины `u` в вершину `v`. Реализация обращается к одной ячейке матрицы смежности и сравнивает её с единицей.

Вход: номера вершин `u` и `v`.

Выход: значение `true`, если `adj[u][v] == 1`; иначе значение `false`.

Функция `Graph::neighbors(int u) const`

Метод формирует список всех вершин, достижимых из вершины `u` за один шаг. Для ориентированного графа это исходящие соседи, а для неориентированного графа это все смежные вершины, потому что матрица смежности симметрична.

Вход: номер вершины `u`.

Выход: вектор `std::vector<int>` с номерами соседних вершин.

Функция `Graph::printAdjMatrix() const`

Метод выводит матрицу смежности в консоль. Вершины при выводе переводятся из внутренней нумерации $0, 1, \dots, n-1$ в пользовательскую нумерацию $1, 2, \dots, n$. Для отсутствующего ребра выводится `inf`, а для существующего ребра – значение 1.

Вход: текущий объект `Graph`.

Выход: таблица матрицы смежности в консоли.

Функция `Graph::printFull(const std::vector<std::vector<double>*> weightMatrix) const`

Метод выводит расширенное представление графа: список смежности, список рёбер, матрицу смежности и, если она была передана, весовую матрицу. Указатель на весовую матрицу сделан необязательным, поэтому одна функция используется и до генерации весов, и после неё.

Вход: текущий граф; необязательный указатель `weightMatrix` на матрицу весов.

Выход: полное текстовое описание графа в консоли.

Функция `Graph::isEmpty() const`

Метод используется для быстрой проверки, был ли уже создан граф. Он возвращает истину, если число вершин равно нулю.

Вход: текущий объект `Graph`.

Выход: логическое значение, показывающее, пустой граф или нет.

Функция `double normalSample(int n)`

Функция генерирует одно значение, приближённо распределённое по нормальному закону. Для этого используется сумма `n` равномерно распределённых случайных величин и нормировка по формуле, указанной в комментарии исходного файла. Такой подход удобен, потому что он прост в реализации и не требует отдельного сложного генератора нормального распределения.

Вход: число `n`, задающее количество равномерных случайных величин для аппроксимации.

Выход: вещественное число типа `double`, имитирующее выборку из нормального распределения.

Функция `int sampleDegree(int maxDeg)`

Функция переводит случайное значение нормального распределения в целую степень вершины. Полученное число ограничивается диапазоном от 1 до `maxDeg`, чтобы каждая вершина получила допустимую положительную степень.

Вход: максимальная допустимая степень `maxDeg`.

Выход: целое число, представляющее степень вершины.

Функция `std::vector<int> generateDegreeSequence(int numVertices)`

Функция создаёт последовательность степеней для заданного количества вершин. Сначала для каждой вершины генерируется степень, затем сумма степеней корректируется так, чтобы она соответствовала требуемой модели генерации графа. Эта последовательность далее используется при построении ориентированного ациклического графа.

Вход: количество вершин `numVertices`.

Выход: вектор целых чисел, где каждый элемент соответствует степени одной вершины.

Функция `Graph generateDAG(int n, bool directed)`

Функция является основной точкой генерации графа. Если параметр `directed` равен `true`, строится ориентированный ациклический граф. Для этого создаётся случайный топологический порядок, добавляется путь для связности, а дополнительные дуги проводятся только вперёд по этому порядку. Если параметр `directed` равен `false`, сначала строится ориентированная модель, а затем направления рёбер игнорируются и получается неориентированный граф.

Вход: количество вершин `n`; признак ориентированности `directed`.

Выход: сгенерированный объект `Graph`.

Функция `Matrix generateWeightMatrix(const Graph& g, WeightMode mode)`

Функция создаёт матрицу весов для уже сгенерированного графа. Вес назначается только там, где в графе существует ребро. Для отсутствующих рёбер используется специальное значение бесконечности, что важно для алгоритма Шимбелла и алгоритма Флойда-Уоршалла. Режим `POSITIVE` создаёт положительные веса, `NEGATIVE` – отрицательные, а `MIXED` – смешанные.

Вход: граф `g`; режим генерации весов `mode`.

Выход: матрица `Matrix` размера $n \times n$ с весами существующих рёбер.

Функции лабораторной работы №1

Функция `std::vector<int> bfs(const Graph& g, int src)`

Функция выполняет поиск в ширину из вершины `src`. Она используется как вспомогательная часть вычисления метрических характеристик графа. Поиск идёт по списку соседей, полученному из матрицы смежности. Если вершина достижима, для неё записывается кратчайшее расстояние в количестве рёбер; если вершина недостижима, значение остаётся равным `-1`.

Вход: граф `g`; начальная вершина `src`.

Выход: вектор расстояний от вершины `src` до всех остальных вершин.

Функция `GraphMetrics computeMetrics(const Graph& g)`

Функция вычисляет основные метрические характеристики графа. Для каждой вершины вызывается `bfs`, после чего формируется матрица расстояний. Эксцентриситет вершины определяется как максимальное расстояние от неё до достижимых вершин. Затем среди эксцентриситетов выбираются радиус, диаметр, центральные вершины и диаметральные вершины.

Вход: граф `g`.

Выход: структура `GraphMetrics`, содержащая матрицу расстояний, эксцентриситеты, радиус, диаметр, центр и диаметральные вершины.

Функция `void printMetrics(const GraphMetrics& m)`

Функция отвечает только за вывод результатов, полученных в `computeMetrics`. Она печатает матрицу расстояний, список эксцентриситетов, радиус, диаметр, центр и диаметральные вершины в удобном для проверки виде.

Вход: структура `GraphMetrics`.

Выход: форматированный вывод метрических характеристик в консоли.

Функция `std::pair<Matrix, Matrix> shimbell(const Matrix& W, int k)`

Функция реализует метод Шимбелла для нахождения путей, состоящих ровно из `k` рёбер. Внутри используются операции тропического умножения матриц: для минимальных путей выбирается минимум суммы весов, а для максимальных путей – максимум. При `k = 0` возвращается аналог единичной матрицы, где путь из вершины в саму себя имеет длину ноль.

Вход: весовая матрица `W`; количество рёбер в пути `k`.

Выход: пара матриц: первая содержит минимальные веса путей, вторая – максимальные веса путей для ровно `k` шагов.

Функция `void printMatrix(const Matrix& M)`

Функция выводит матрицу весов на экран. Большие положительные значения отображаются как `INF`, большие отрицательные значения – как `-INF`. Это делает вывод понятным, так как бесконечность означает отсутствие допустимого пути.

Вход: матрица `M`.

Выход: таблица значений матрицы в консоли.

Функция `std::vector<std::vector<int>> findAllPaths(const Graph& g, int src, int dst)`

Функция находит все простые маршруты между вершинами `src` и `dst`. Для этого применяется рекурсивный поиск в глубину. Массив посещённых вершин не позволяет одной и той же вершине повторяться в одном маршруте, поэтому результат содержит именно простые пути.

Вход: граф `g`; начальная вершина `src`; конечная вершина `dst`.

Выход: список маршрутов, где каждый маршрут представлен вектором номеров вершин.

Функция `bool pathExists(const Graph& g, int src, int dst)`

Функция является удобной обёрткой над поиском маршрутов. Она вызывает поиск путей и проверяет, найден ли хотя бы один путь между выбранными вершинами.

Вход: граф `g`; начальная вершина `src`; конечная вершина `dst`.

Выход: значение `true`, если путь существует; иначе `false`.

Функция `int countPaths(const Graph& g, int src, int dst)`

Функция подсчитывает количество простых маршрутов между двумя вершинами. Она использует тот же результат, что и `findAllPaths`, но возвращает только число маршрутов.

Вход: граф `g`; начальная вершина `src`; конечная вершина `dst`.

Выход: целое число, равное количеству найденных маршрутов.

Функция `void printPaths(const std::vector<std::vector<int>& paths, int src, int dst)`

Функция выводит найденные маршруты в пользовательской нумерации вершин. Если список пуст, она сообщает, что маршрутов между выбранными вершинами нет.

Вход: список маршрутов `paths`; начальная вершина `src`; конечная вершина `dst`.

Выход: текстовый список маршрутов в консоли.

Функции лабораторной работы №2

Функция `void dfs(const Graph& g, int start)`

Функция выполняет обход графа поиском в глубину, начиная с вершины `start`. В реализации используется стек, поэтому алгоритм не зависит от глубины рекурсии. Во время работы фиксируется порядок посещения вершин и количество итераций. После обхода программа также показывает вершины, которые остались недостижимыми из выбранной начальной вершины.

Вход: граф `g`; стартовая вершина `start`.

Выход: порядок обхода, список недостижимых вершин и количество итераций в консоли.

Функция `FloydResult floydWarshall(const Graph& g, const std::vector<std::vector<double>& W)`

Функция реализует алгоритм Флойда-Уоршалла для поиска кратчайших путей между всеми парами вершин. Матрица `T` хранит текущие расстояния, а матрица `H` используется для восстановления маршрута. После выполнения основного тройного цикла проверяются диагональные элементы матрицы расстояний, чтобы обнаружить отрицательные циклы.

Вход: граф `g`; весовая матрица `W`, в которой диагональ для этого алгоритма равна нулю.

Выход: структура `FloydResult` с матрицей расстояний, матрицей восстановления пути, числом итераций и признаком отрицательного цикла.

Функция `void printFloydResult(const FloydResult& res, const std::vector<std::vector<double>& W, int n, int src, int dst)`

Функция выводит результат алгоритма Флойда-Уоршалла. Она печатает исходную весовую матрицу, итоговую матрицу кратчайших расстояний, матрицу восстановления пути и путь между выбранными пользователем вершинами. Если путь отсутствует или найден отрицательный цикл, функция выводит соответствующее предупреждение.

Вход: результат `res`; весовая матрица `W`; количество вершин `n`; начальная вершина `src`; конечная вершина `dst`.

Выход: подробный вывод таблиц и восстановленного кратчайшего пути в консоли.

Функции лабораторной работы №3

Функция `std::vector<std::vector<int>> generateCapacityMatrix(const Graph& g)`

Функция создаёт матрицу пропускных способностей для ориентированного графа. Для каждой существующей дуги генерируется целое значение пропускной способности, а для отсутствующих дуг остаётся ноль. Эта матрица используется в алгоритмах максимального потока и потока минимальной стоимости.

Вход: ориентированный граф `g`.

Выход: матрица пропускных способностей, где положительное число означает наличие дуги и её ёмкость.

Функция `std::vector<std::vector<int> generateCostMatrix(const Graph& g)`

Функция создаёт матрицу стоимостей для тех же дуг, которые присутствуют в графе. Стоимость показывает цену прохождения одной единицы потока по конкретной дуге.

Вход: ориентированный граф `g`.

Выход: матрица стоимостей, где ноль для отсутствующей дуги означает, что поток по ней не допускается.

Функция `FFResult fordFulkerson(int n, const std::vector<std::vector<int>& cap, int src, int dst)`

Функция реализует алгоритм Форда-Фалкерсона в варианте Эдмондса-Карпа. Это означает, что каждый увеличивающий путь в остаточной сети ищется с помощью поиска в ширину. После нахождения пути определяется минимальная остаточная пропускная способность на этом пути, затем поток увеличивается, а обратные рёбра в остаточной сети обновляются.

Вход: количество вершин `n`; матрица пропускных способностей `cap`; исток `src`; сток `dst`.

Выход: структура `FFResult`, содержащая итоговую матрицу потока, значение максимального потока, исток и сток.

Функция `FFResult fordFulkersonSilent(int n, const std::vector<std::vector<int>& cap, int src, int dst)`

Функция выполняет тот же расчёт максимального потока, что и `fordFulkerson`, но не печатает промежуточные увеличивающие пути. Она используется внутри функции минимального стоимостного потока, где нужен только численный результат и матрица потока.

Вход: количество вершин `n`; матрица пропускных способностей `cap`; исток `src`; сток `dst`.

Выход: структура `FFResult` без промежуточного вывода в консоль.

Функция `void printCapacityMatrix(const std::vector<std::vector<int>& cap, int n)`

Функция печатает матрицу пропускных способностей. Она нужна для того, чтобы пользователь мог видеть исходные данные перед запуском алгоритма максимального потока.

Вход: матрица `cap`; количество вершин `n`.

Выход: таблица пропускных способностей в консоли.

Функция `void printCostMatrix(const std::vector<std::vector<int>& cost, int n)`

Функция печатает матрицу стоимостей дуг. Она используется после генерации матриц для лабораторной работы №3.

Вход: матрица стоимостей `cost`; количество вершин `n`.

Выход: таблица стоимостей в консоли.

Функция `void printFFResult(const FFResult& res, const std::vector<std::vector<int>& cap, int n)`

Функция выводит результат алгоритма Форда-Фалкерсона. Она показывает максимальный поток, дуги с положительным потоком и соотношение потока к пропускной способности.

Вход: результат `res`; матрица пропускных способностей `cap`; количество вершин `n`.

Выход: форматированный отчёт о максимальном потоке в консоли.

Функция `MCFResult minCostFlow(int n, const std::vector<std::vector<int>& cap, const std::vector<std::vector<int>& cost, int src, int dst)`

Функция вычисляет поток минимальной стоимости. Сначала с помощью

`fordFulkersonSilent` находится максимальный поток, затем целевой поток принимается равным $\lfloor 2/3 \cdot \varphi_{max} \rfloor$. После этого поток набирается последовательным добавлением кратчайших по стоимости путей в остаточной сети. Для каждого выбранного пути учитываются остаточные ёмкости и обратные дуги.

Вход: количество вершин `n`; матрица пропускных способностей `cap`; матрица стоимостей `cost`; исток `src`; сток `dst`.

Выход: структура `MCFResult` с максимальным потоком, целевым потоком, достигнутым потоком, итоговой стоимостью и матрицей потока.

Функция `void printMCFResult(const MCFResult& res, const std::vector<std::vector<int>>& cap, const std::vector<std::vector<int>>& cost, int n)`

Функция выводит результат минимального стоимостного потока. Она печатает значение максимального потока, требуемый поток, фактически найденный поток, список задействованных дуг и проверочный расчёт общей стоимости.

Вход: результат `res`; матрица пропускных способностей `cap`; матрица стоимостей `cost`; количество вершин `n`.

Выход: подробный вывод потока минимальной стоимости в консоли.

Функции лабораторной работы №4

Функция `long long countSpanningTrees(const Graph& g)`

Функция считает количество остовных деревьев с помощью матричной теоремы Кирхгофа. Сначала строится матрица Лапласа: на диагонали записываются степени вершин, а на местах существующих рёбер вне диагонали ставится -1. Затем удаляются одна строка и один столбец, после чего вычисляется определитель полученной матрицы.

Вход: неориентированный граф `g`.

Выход: количество остовных деревьев типа `long long`.

Функция `void printKirchhoffResult(const Graph& g, long long count)`

Функция выводит результат применения теоремы Кирхгофа. Она показывает количество остовных деревьев для текущего графа.

Вход: граф `g`; найденное количество остовных деревьев `count`.

Выход: текстовый результат в консоли.

Функция `BoruvkaResult boruvka(const Graph& g, const std::vector<std::vector<double>>& W)`

Функция реализует алгоритм Борувки для построения минимального остовного дерева. На каждой итерации для каждой компоненты связности выбирается самое дешёвое исходящее ребро. Эти рёбра добавляются в остов, а соответствующие компоненты объединяются. Процесс продолжается, пока не останется одна компонента или пока дальнейшее объединение невозможно.

Вход: неориентированный граф `g`; весовая матрица `W`.

Выход: структура `BoruvkaResult`, содержащая список рёбер минимального остова и его суммарный вес.

Функция `void printBoruvkaResult(const BoruvkaResult& res)`

Функция печатает рёбра минимального остовного дерева и общую стоимость остова. Она используется после запуска алгоритма Борувки и перед алгоритмами, которым нужен построенный остов.

Вход: структура `BoruvkaResult`.

Выход: список рёбер остова и его вес в консоли.

Функция `EdgeCoverResult minEdgeCover(const Graph& g, const std::vector<std::vector<double>>& W, bool useMST, const std::vector<MSTEdge>& mstEdges)`

Функция строит минимальное рёберное покрытие. Если выбран режим `useMST`, покрытие строится по рёбрам минимального остова; иначе используется весь исходный граф.

Сначала ищется максимальное паросочетание, затем для каждой непокрытой вершины добавляется любое подходящее инцидентное ребро. Для общего неориентированного графа используется алгоритм Эдмондса с сжатием цветков.

Вход: граф `g`; весовая матрица `W`; флаг `useMST`; список рёбер остова `mstEdges`.

Выход: структура `EdgeCoverResult`, содержащая рёбра покрытия и размер найденного паросочетания.

Функция `void printEdgeCoverResult(const EdgeCoverResult& res)`

Функция выводит результат построения рёберного покрытия. Для каждого ребра показываются его концы и вес, а также печатается размер паросочетания, использованного при построении покрытия.

Вход: структура `EdgeCoverResult`.

Выход: список рёбер минимального покрытия в консоли.

Функция `PruferResult pruferEncode(const std::vector<MSTEdge>& mstEdges, int n)`

Функция кодирует минимальное остовное дерево кодом Прюфера. На каждом шаге выбирается лист с минимальным номером, в код записывается его единственный сосед, затем лист удаляется из текущего дерева. Вес удалённого ребра сохраняется отдельно, чтобы при декодировании восстановить не только структуру дерева, но и веса рёбер.

Вход: список рёбер остова `mstEdges`; количество вершин `n`.

Выход: структура `PruferResult`, содержащая код Прюфера и список весов удаляемых рёбер.

Функция `std::vector<MSTEdge> pruferDecode(const std::vector<int>& code, const std::vector<double>& weights, int n)`

Функция восстанавливает дерево по коду Прюфера. На каждом шаге выбирается минимальный лист, которого нет в текущем коде, и соединяется с первым элементом кода. Вес ребра берётся из сохранённого массива `weights`. После обработки всех элементов кода соединяются две оставшиеся вершины.

Вход: код Прюфера `code`; список весов `weights`; количество вершин `n`.

Выход: список рёбер восстановленного дерева.

Функция `void printPruferResult(const PruferResult& res, int n)`

Функция выводит код Прюфера и сохранённые веса рёбер. Она показывает, какая последовательность получилась после кодирования остова.

Вход: результат кодирования `res`; количество вершин `n`.

Выход: код Прюфера и веса в консоли.

Функция `void printDecodedTree(const std::vector<MSTEdge>& edges)`

Функция выводит дерево, восстановленное из кода Прюфера. Это позволяет сравнить исходный остов и результат декодирования.

Вход: список восстановленных рёбер `edges`.

Выход: список рёбер декодированного дерева в консоли.

Функции лабораторной работы №5

Функция `EulerianResult buildEulerianCycle(const Graph& g)`

Функция проверяет, является ли неориентированный граф эйлеровым, и строит эйлеров цикл. Сначала вычисляются степени всех вершин и проверяется связность. Если граф уже связный и все степени чётные, сразу запускается построение цикла. Если условия не выполнены, программа пытается локально изменить копию графа: добавить или удалить рёбра так, чтобы сделать степени чётными и не нарушить связность. Исходный объект `Graph` при этом не изменяется.

Вход: неориентированный граф `g`.

Выход: структура `EulerianResult`, содержащая исходные степени, нечётные вершины, выполненные изменения и найденный эйлеров цикл.

Функция `void printEulerianResult(const EulerianResult& res)`

Функция выводит результат проверки графа на эйлеровость. Она печатает исходные степени вершин, список нечётных вершин, информацию о добавленных и удалённых рёбрах, а также сам эйлеров цикл, если его удалось построить.

Вход: структура `EulerianResult`.

Выход: подробный отчёт об эйлеровом цикле в консоли.

Функция `std::vector<FundamentalCutset> buildFundamentalCutsets(const Graph& g, const std::vector<MSTEdge>& mstEdges)`

Функция строит фундаментальную систему разрезов относительно минимального остова. Для каждого ребра остова оно временно удаляется из дерева. После удаления дерево распадается на две компоненты, и функция ищет все рёбра исходного графа, которые соединяют эти две компоненты. Полученный набор рёбер образует фундаментальный разрез.

Вход: исходный граф `g`; список рёбер минимального остова `mstEdges`.

Выход: вектор структур `FundamentalCutset`, каждая из которых содержит ребро остова и соответствующий разрез.

Функция `void printFundamentalCutsets(const std::vector<FundamentalCutset>& cutsets)`

Функция выводит все построенные фундаментальные разрезы. Для каждого разреза показывается номер, соответствующее ребро остова и список рёбер, входящих в разрез.

Вход: список фундаментальных разрезов `cutsets`.

Выход: таблица фундаментальных разрезов в консоли.

Функция `void printCutsetSymmetricDifference(const std::vector<FundamentalCutset>& cutsets, const std::vector<int>& selectedIndices)`

Функция вычисляет симметрическую разность выбранных фундаментальных разрезов. Если ребро встречается в выбранных разрезах нечётное число раз, оно остаётся в результате; если чётное число раз, оно исключается. Это соответствует сложению множеств рёбер по модулю два.

Вход: список разрезов `cutsets`; номера выбранных разрезов `selectedIndices`.

Выход: множество рёбер, полученное в результате симметрической разности, выведенное в консоль.

Функции объединяющего меню

Функция `static inline int D(int v)`

Функция переводит внутреннюю нумерацию вершин в пользовательскую. В программе вершины хранятся с нуля, но пользователю они показываются с единицы.

Вход: номер вершины `v` во внутренней нумерации.

Выход: номер вершины `v + 1` для вывода в консоль.

Функция `static void clearInput()`

Функция очищает поток ввода после неправильного или завершённого чтения. Она сбрасывает флаг ошибки и удаляет оставшиеся символы до конца строки.

Вход: стандартный поток ввода `std::cin`.

Выход: очищенное состояние потока ввода.

Функция `static int readInt(const std::string& prompt)`

Функция безопасно считывает целое число. Она выводит приглашение, пытается прочитать значение и повторяет запрос, если пользователь ввёл некорректные данные. Это используется во всех пунктах меню, где требуется ввод чисел.

Вход: строка-приглашение `prompt`.

Выход: корректно считанное целое число.

Функции `requireGraph()`, `requireWeight()`, `requireFlowMatrices()`, `requireMST()`

Эти функции выводят сообщения об ошибках, если пользователь пытается запустить ал-

горитм без необходимых предварительных данных. Например, весовую матрицу нельзя использовать до генерации графа, а код Прюфера нельзя построить до нахождения минимального остова.

Вход: текущее состояние глобальных переменных программы.

Выход: диагностическое сообщение в консоли.

Функция `static bool requireLab4Undirected()`

Функция проверяет, можно ли запускать алгоритмы лабораторной работы №4. Для этих алгоритмов требуется уже сгенерированный неориентированный граф.

Вход: состояние текущего графа.

Выход: `true`, если граф существует и является неориентированным; иначе `false` и сообщение об ошибке.

Функция `static bool requireLab5Undirected()`

Функция выполняет аналогичную проверку для лабораторной работы №5. Она не даёт запускать эйлеров цикл и фундаментальные разрезы на ориентированном графе.

Вход: состояние текущего графа.

Выход: `true` при выполнении условий запуска; иначе `false`.

Функция `void menuGenerateGraph()`

Функция реализует пункт меню генерации графа. Она считывает количество вершин и тип графа, вызывает `generateDAG`, сохраняет результат в глобальную переменную и сбрасывает ранее построенные матрицы и результаты, потому что они уже не соответствуют новому графу.

Вход: количество вершин и тип графа, введённые пользователем.

Выход: новый граф в глобальной переменной `gGraph` и вывод матрицы смежности.

Функция `void menuGenerateWeightMatrix()`

Функция реализует пункт меню генерации весовой матрицы. Она проверяет наличие графа, считывает выбранный режим весов, вызывает `generateWeightMatrix` и выводит полученную матрицу.

Вход: текущий граф и выбранный режим весов.

Выход: глобальная матрица `gWeightMatrix` и её вывод в консоли.

Функция `static void printShimbellMatrix(const Matrix& M)`

Функция используется в меню метода Шимбелла для аккуратного вывода матриц минимальных и максимальных путей. Она отдельно обрабатывает значения бесконечности.

Вход: матрица `M`.

Выход: форматированный вывод матрицы.

Функции `menuEccentricity()`, `menuShimbell()`, `menuPaths()`

Эти функции связывают пункты меню лабораторной работы №1 с алгоритмами. Они проверяют наличие нужных данных, считывают параметры пользователя и вызывают соответственно `computeMetrics`, `shimbell` и `findAllPaths`.

Вход: текущий граф, весовая матрица при необходимости и параметры, введённые пользователем.

Выход: результаты алгоритмов лабораторной работы №1 в консоли.

Функции `menuDFS()` и `menuFloydWarshall()`

Эти функции отвечают за запуск алгоритмов лабораторной работы №2. Первая считывает стартовую вершину и запускает `dfs`. Вторая подготавливает весовую матрицу для Флойда-Уоршалла, устанавливая нули на диагонали, затем считывает начальную и конечную вершины и вызывает `floydWarshall`.

Вход: текущий граф, весовая матрица и выбранные вершины.

Выход: обход в глубину или кратчайший путь между выбранными вершинами.

Функции `menuGenerateFlowMatrices()`, `menuFordFulkerson()`, `menuMinCostFlow()`

Эти функции реализуют пункты меню лабораторной работы №3. Первая создаёт матрицы пропускных способностей и стоимостей, вторая запускает максимальный поток, третья запускает поток минимальной стоимости. Перед запуском выполняется проверка, что граф

ориентированный и необходимые матрицы уже созданы.

Вход: ориентированный граф, выбранные исток и сток, матрицы пропускных способностей и стоимостей.

Выход: максимальный поток или поток минимальной стоимости, выведенный в консоль.

Функции `menuKirchhoff()`, `menuBoruvka()`, `menuEdgeCover()`, `menuPrufer()`

Эти функции запускают алгоритмы лабораторной работы №4. Они работают только с неориентированным графом. `menuKirchhoff` считает количество остовных деревьев, `menuBoruvka` строит минимальный остов, `menuEdgeCover` строит рёберное покрытие, а `menuPrufer` кодирует и декодирует остов кодом Прюфера.

Вход: неориентированный граф, весовая матрица, а также построенный остов для задач, которые от него зависят.

Выход: результаты алгоритмов лабораторной работы №4.

Функции `menuEulerianCycle()` и `menuFundamentalCutsets()`

Эти функции реализуют пункты меню лабораторной работы №5. Первая строит эйлеров цикл или показывает, почему это невозможно. Вторая строит минимальный остов, затем по нему формирует фундаментальные разрезы и позволяет выбрать разрезы для вычисления симметрической разности.

Вход: неориентированный граф, весовая матрица и выбранные пользователем номера разрезов.

Выход: эйлеров цикл либо система фундаментальных разрезов и их симметрическая разность.

Функция `void printMenu()`

Функция печатает главное меню программы. В меню перечислены все пункты: генерация графа, построение матриц, запуск алгоритмов лабораторных работ №1–№5 и выход.

Вход: входных параметров нет.

Выход: текст главного меню в консоли.

Функция `int main()`

Главная функция программы организует общий цикл работы приложения. Пока пользователь не выбрал пункт 0, программа печатает меню, считывает номер команды через `readInt` и с помощью оператора `switch` вызывает соответствующую функцию меню. Такой подход позволяет объединить пять лабораторных работ в одном исполняемом файле.

Вход: команды пользователя из консоли.

Выход: выполнение выбранных алгоритмов и завершение программы после выбора пункта выхода.

3 Результаты работы программы

3.1 Задания 1–11: ориентированный граф на 5 вершинах

После запуска программы выводится главное меню, в котором доступны пункты всех пяти лабораторных работ. Для заданий 1–11 был сгенерирован ориентированный граф на 5 вершинах, так как первые три лабораторные работы используют ориентированный ациклический граф и алгоритмы для маршрутов, кратчайших путей и потоков. Главное меню программы показано на рис. 1.

```
C:\Users\shami\OneDrive\Documents\theoryGraph\graph-theory-labs1\cmake-build-debug\lab_combined\lab_combined.exe

=====
GRAPH THEORY LABS
=====
1. Generate graph
2. Show graph (adjacency list / matrix)
3. Generate weight matrix
====Lab 1====
4. Eccentricities, center, diametral vertices
5. Shimbell's method (min/max paths, k steps)
6. Find all routes between two vertices
====Lab 2====
7. DFS traversal
8. Shortest path: Floyd-Warshall
====Lab 3====
9. Generate capacity and cost matrices
10. Max flow: Ford-Fulkerson
11. Min-cost flow
====Lab 4====
12. Kirchhoff: count spanning trees
13. Boruvka: minimum spanning tree
14. Minimum edge cover
15. Prufer encode / decode
====Lab 5====
16. Eulerian check / modify / Euler cycle
17. Fundamental cutsets from MST
0. Exit
-----
Enter choice:
```

Рис. 1. Главное меню объединённой программы

При выборе пункта «1» пользователь вводит количество вершин и тип графа. Для первой части результатов выбран ориентированный граф; программа строит матрицу смежности и сообщает, что для дальнейших алгоритмов необходимо сгенерировать весовую матрицу — рис. 2.

Enter choice:1

Number of vertices:5

Graph type:

1. Directed

2. Undirected

Choose [1/2]:1

ADJACENCY MATRIX [DIRECTED]

	1	2	3	4	5
1	0	inf	inf	inf	inf
2	1	0	1	1	1
3	1	inf	0	1	1
4	1	inf	inf	0	1
5	1	inf	inf	inf	0

Graph with 5 vertices generated.

Generate weight matrix with option 3.

Рис. 2. Генерация ориентированного графа на 5 вершинах

При выборе пункта «2» выводится полное представление графа: список смежности, список дуг и матрица смежности. Это позволяет проверить структуру сгенерированного ориентированного графа перед запуском алгоритмов — рис. 3.

```

-----
ADJACENCY LIST [DIRECTED]
-----

Vertex 1 --> (isolated) (degree=0)
Vertex 2 --> 1, 3, 4, 5 (degree=4)
Vertex 3 --> 1, 4, 5 (degree=3)
Vertex 4 --> 1, 5 (degree=2)
Vertex 5 --> 1 (degree=1)

EDGE LIST:
    2 ----> 1
    2 ----> 3
    2 ----> 4
    2 ----> 5
    3 ----> 1
    3 ----> 4
    3 ----> 5
    4 ----> 1
    4 ----> 5
    5 ----> 1

-----
ADJACENCY MATRIX [DIRECTED]
-----

      1    2    3    4    5
-----
1 |  0  inf  inf  inf  inf
2 |  1   0   1   1   1
3 |  1  inf   0   1   1
4 |  1  inf  inf   0   1
5 |  1  inf  inf  inf   0
-----

```

Рис. 3. Вывод списка смежности, списка дуг и матрицы смежности

При выборе пункта «3» пользователь выбирает режим генерации весов: положительные, отрицательные или смешанные. После этого программа выводит весовую матрицу, где значение INF означает отсутствие дуги — рис. 4.

```
Enter choice:3

Weight mode:
  1. Positive only
  2. Negative only
  3. Mixed
Choose [1/2/3]:3

Weight matrix generated.
```

	1	2	3	4	5
1	INF	INF	INF	INF	INF
2	1	INF	-1	4	-8
3	1	INF	INF	6	-1
4	3	INF	INF	INF	4
5	2	INF	INF	INF	INF

Рис. 4. Генерация весовой матрицы для ориентированного графа

При выборе пункта «4» вычисляются расстояния между вершинами, эксцентриситеты, радиус, диаметр, центр графа и диаметральные вершины. Результат работы алгоритма представлен в виде матрицы расстояний и итоговой таблицы — рис. 5.

```
Enter choice:4

      1    2    3    4    5
-----
1 |    0  inf  inf  inf  inf
2 |    1    0    1    1    1
3 |    1  inf    0    1    1
4 |    1  inf  inf    0    1
5 |    1  inf  inf  inf    0

Vertex | Eccentricity | Role
-----+-----+-----
1 |          0 | -
2 |          1 | DIAMETRICAL + CENTER
3 |          1 | DIAMETRICAL + CENTER
4 |          1 | DIAMETRICAL + CENTER
5 |          1 | DIAMETRICAL + CENTER

Radius   = 1
Diameter = 1

CENTER vertices      = { 2 3 4 5 }
DIAMETRICAL vertices = { 2 3 4 5 }
```

Рис. 5. Эксцентриситеты, центр и диаметральные вершины графа

При выборе пункта «5» пользователь задаёт количество шагов k , после чего программа применяет метод Шимбелла. В результате выводятся матрицы минимальных и максимальных путей, состоящих ровно из заданного количества дуг — рис. 6.

```
Enter k (number of steps):2

-----
SHIMBELL MIN-path (k=2)
-----

      1      2      3      4      5
-----
1 |  INF  INF  INF  INF  INF
2 |  -6   INF  INF   5  -2
3 |   1   INF  INF  INF  10
4 |   6   INF  INF  INF  INF
5 |  INF  INF  INF  INF  INF

-----
SHIMBELL MAX-path (k=2)
-----

      1      2      3      4      5
-----
1 |  -INF -INF -INF -INF -INF
2 |   7  -INF -INF   5   8
3 |   9  -INF -INF -INF  10
4 |   6  -INF -INF -INF -INF
5 |  -INF -INF -INF -INF -INF
```

Рис. 6. Метод Шимбелла для ориентированного взвешенного графа

При выборе пункта «6» пользователь вводит начальную и конечную вершины. Программа определяет, существует ли маршрут между ними, подсчитывает количество простых маршрутов и выводит найденные маршруты — рис. 7.


```
ADJACENCY MATRIX [DIRECTED]
-----

      1    2    3    4    5
-----
1 |    0  inf  inf  inf  inf|
2 |    1    0    1    1    1
3 |    1  inf    0    1    1
4 |    1  inf  inf    0    1
5 |    1  inf  inf  inf    0

-----

WEIGHT MATRIX (INF = no edge, 0 = self)
-----

      1    2    3    4    5
-----
1 |    INF    INF    INF    INF    INF
2 |    1    INF    -1    4    -8
3 |    1    INF    INF    6    -1
4 |    3    INF    INF    INF    4
5 |    2    INF    INF    INF    INF

-----

Vertices are numbered 1 to 5.
Source vertex:2

Destination vertex:4

Path from 2 to 4: EXISTS (2 routes)

Routes:
Route 1: { 2 -> 3 -> 4 }
Route 2: { 2 -> 4 }
```

Рис. 7. Поиск всех маршрутов между двумя вершинами

При выборе пункта «7» выполняется обход графа поиском в глубину из заданной вершины. В результате выводится порядок обхода, количество посещённых вершин и число итераций алгоритма — рис. 8.

```
Enter choice:7
```

```
Enter start vertex:2
```

```
-----  
DFS TRAVERSAL (start = vertex 2)  
-----
```

```
Traversal order:
```

```
2 -> 1 -> 3 -> 4 -> 5
```

```
Vertices visited   : 5 / 5
```

```
Total iterations   : 23
```

Рис. 8. Обход графа поиском в глубину

При выборе пункта «8» пользователь вводит начальную и конечную вершины для поиска кратчайшего пути. Алгоритм Флойда-Уоршалла выводит исходную весовую матрицу, матрицу кратчайших расстояний, матрицу восстановления пути, сам путь и количество итераций — рис. 9.

```

FLOYD-WARSHALL (src=2 dst=4)
-----

C = WEIGHT MATRIX (INF = no edge, 0 = self):

      1      2      3      4      5
-----
1 |      0      INF      INF      INF      INF
2 |      1       0      -1       4      -8
3 |      1      INF       0       6      -1
4 |      3      INF      INF       0       4
5 |      2      INF      INF      INF       0

T = DISTANCE MATRIX (all-pairs shortest paths):

      1      2      3      4      5
-----
1 |      0      INF      INF      INF      INF
2 |     -6       0      -1       4      -8
3 |      1      INF       0       6      -1
4 |      3      INF      INF       0       4
5 |      2      INF      INF      INF       0

H = PATH MATRIX (H[i][j] = first next hop, 0 = no path):

      1      2      3      4      5
-----
1 |      1       0       0       0       0
2 |      5       2       3       4       5
3 |      1       0       3       4       5
4 |      1       0       0       4       5
5 |      1       0       0       0       5

Shortest path from 2 to 4:
Path      : 2 -> 4
Distance: 4

Total iterations: 125 (n=5 where n^3=125)

```

Рис. 9. Поиск кратчайшего пути алгоритмом Флойда-Уоршалла

При выборе пункта «9» на основе ориентированного графа генерируются матрицы пропускных способностей и стоимостей. Дополнительно программа выводит входящие и исходящие степени вершин, чтобы пользователю было проще выбрать исток и сток — рис. 10.

Enter choice:9

CAPACITY MATRIX $\Omega[i][j]$:

	1	2	3	4	5
1	0	0	0	0	0
2	3	0	2	10	9
3	20	0	0	11	6
4	15	0	0	0	16
5	14	0	0	0	0

COST MATRIX $D[i][j]$:

	1	2	3	4	5
1	0	0	0	0	0
2	9	0	4	10	6
3	1	0	0	1	3
4	6	0	0	0	9
5	10	0	0	0	0

Vertex degrees (to help choose source / sink):

Vertex | Out-degree | In-degree

1	0		4
2	4		0
3	3		1
4	2		2
5	1		3

Good source: high out-degree, low in-degree.

Good sink: high in-degree, low out-degree.

Рис. 10. Генерация матриц пропускных способностей и стоимостей

При выборе пункта «10» программа находит максимальный поток алгоритмом Форда-Фалкерсона. На экран выводятся увеличивающие пути, значение максимального потока и распределение потока по дугам — рис. 11.

```

Enter choice:10

Source vertex:2

Sink vertex:4

Path 1: 2 -> 4 (delta = 10)
Path 2: 2 -> 3 -> 4 (delta = 2)

-----
FORD-FULKERSON RESULT
Source: 2   Sink: 4
Maximum flow  phi_max = 12
-----

CAPACITY MATRIX  Omega[i][j]:

      1    2    3    4    5
-----
1 |    0    0    0    0    0
2 |    3    0    2   10    9
3 |   20    0    0   11    6
4 |   15    0    0    0   16
5 |   14    0    0    0    0

FLOW on each arc (only arcs with flow > 0):
(2 -> 3)  flow=2  cap=2
(2 -> 4)  flow=10 cap=10
(3 -> 4)  flow=2  cap=11

```

Рис. 11. Нахождение максимального потока алгоритмом Форда-Фалкерсона

При выборе пункта «11» вычисляется поток минимальной стоимости. Величина требуемого потока берётся равной $\lfloor 2/3 \cdot \varphi_{\max} \rfloor$. Программа выводит найденные кратчайшие по стоимости пути, достигнутый поток и итоговую стоимость — рис. 12.

Enter choice:11

Source vertex:2

Sink vertex:4

$\phi_{\max} = 12$ $\theta = \text{floor}(2/3 * 12) = 8$

Step 1 [Floyd-Warshall]: 2 -> 3 -> 4 $\Delta=2$ $\text{path_cost}=5$ $\text{total_flow}=2$

Step 2 [Floyd-Warshall]: 2 -> 4 $\Delta=6$ $\text{path_cost}=10$ $\text{total_flow}=8$

MIN-COST FLOW RESULT

$\phi_{\max} = 12$

$\theta (2/3 * \max) = 8$

Flow achieved = 8

Total cost = 70

FLOW DETAILS (arcs with flow > 0):

Arc	flow	cap	cost/unit	cost
-----	------	-----	-----------	------

(2->3)	2	2	4	8
(2->4)	6	10	10	60
(3->4)	2	11	1	2

Total cost = 70

Рис. 12. Вычисление потока минимальной стоимости

3.2 Задания 12–17: неориентированный граф на 7 вершинах

Для заданий 12–17 был сгенерирован неориентированный граф на 7 вершинах, так как алгоритмы четвёртой и пятой лабораторных работ работают с неориентированным графом. После генерации выводится матрица смежности — рис. 13.

```
Enter choice:1

Number of vertices:7

Graph type:
  1. Directed
  2. Undirected
Choose [1/2]:2

-----
ADJACENCY MATRIX [UNDIRECTED]
-----

      1   2   3   4   5   6   7
-----
1 |   0   1  inf inf   1   1   1
2 |   1   0   1  inf   1   1   1
3 |  inf   1   0   1   1  inf  inf
4 |  inf  inf   1   0  inf   1   1
5 |   1   1   1  inf   0   1   1
6 |   1   1  inf   1   1   0   1
7 |   1   1  inf   1   1   1   0

Note: Undirected => matrix is symmetric (mat[i][j] == mat[j][i])

-----
Graph with 7 vertices generated (undirected).
Generate weight matrix with option 3.
```

Рис. 13. Генерация неориентированного графа на 7 вершинах

При выборе пункта «12» программа строит матрицу Кирхгофа, удаляет первую строку и первый столбец для получения алгебраического дополнения и вычисляет определитель. Полученное значение является числом остовных деревьев графа — рис. 14.

KIRCHHOFF'S MATRIX-TREE THEOREM

Kirchhoff (Laplacian) matrix B:

$B[i][i] = \text{degree}(i)$, $B[i][j] = -1$ if edge $\{i,j\}$, else 0

	1	2	3	4	5	6	7
1	4	-1	0	0	-1	-1	-1
2	-1	5	-1	0	-1	-1	-1
3	0	-1	3	-1	-1	0	0
4	0	0	-1	3	0	-1	-1
5	-1	-1	-1	0	5	-1	-1
6	-1	-1	0	-1	-1	5	-1
7	-1	-1	0	-1	-1	-1	5

Cofactor A₁₁ – delete row 1 and col 1:

	2	3	4	5	6	7
2	5	-1	0	-1	-1	-1
3	-1	3	-1	-1	0	0
4	0	-1	3	0	-1	-1
5	-1	-1	0	5	-1	-1
6	-1	0	-1	-1	5	-1
7	-1	0	-1	-1	-1	5

$\det(A_{11}) = 1584$

Number of spanning trees = 1584

Рис. 14. Подсчёт числа остовных деревьев по теореме Кирхгофа

При выборе пункта «13» строится минимальное остовное дерево алгоритмом Борувки. Программа по шагам выводит компоненты связности, добавляемые безопасные рёбра и итоговый вес полученного остова — рис. 15.

Enter choice:13

--- Round 1 ---

Components: {1} {2} {3} {4} {5} {6} {7}

Add edge (1, 2) weight=1

Add edge (2, 3) weight=1

Add edge (3, 4) weight=1

Add edge (1, 5) weight=1

Add edge (2, 6) weight=1

Add edge (1, 7) weight=1

BORUVKA MST RESULT

MST edges:

Arc	weight
-----	--------

(1 -> 2)	1
----------	---

(2 -> 3)	1
----------	---

(3 -> 4)	1
----------	---

(1 -> 5)	1
----------	---

(2 -> 6)	1
----------	---

(1 -> 7)	1
----------	---

Total MST weight = 6

Рис. 15. Построение минимального остова алгоритмом Борувки

При выборе пункта «14» пользователь выбирает, искать минимальное рёберное покрытие на исходном графе или на построенном остове. В результате выводится мощность максимального паросочетания и рёбра минимального покрытия — рис. 16.

```
Enter choice:14

Work on:
  1. MST (requires Boruvka, option 13)
  2. Full graph
Choose [1/2]:1

-----
MINIMUM EDGE COVER
-----

Maximum matching  |M| = 3
Min edge cover    |C| = 4  (= n - |M| when no isolated vertices)

Cover edges:
  Arc          weight
  -----
  (1 -- 5)      1
  (2 -- 6)      1
  (3 -- 4)      1
  (7 -- 1)      1
```

Рис. 16. Нахождение минимального рёберного покрытия

При выборе пункта «15» построенный минимальный остов кодируется кодом Прюфера с сохранением весов, после чего выполняется обратное декодирование. На экран выводятся удаляемые листья, код Прюфера, веса рёбер и восстановленное дерево — рис. 17.

```

Remove leaf 4  neighbor=3  weight=1
Remove leaf 3  neighbor=2  weight=1
Remove leaf 5  neighbor=1  weight=1
Remove leaf 6  neighbor=2  weight=1
Remove leaf 2  neighbor=1  weight=1
Last edge: (1, 7)  weight=1

-----

PRUFER CODE  (n=7, code length=5)
-----

Prufer sequence: [ 3, 2, 1, 2, 1 ]

Edge weights (in removal order, length 6):
[ 1, 1, 1, 1, 1, 1 ]
(last weight is for the final edge not in Prufer sequence)

-----

Connect 4 -- 3  weight=1
Connect 3 -- 2  weight=1
Connect 5 -- 1  weight=1
Connect 6 -- 2  weight=1
Connect 2 -- 1  weight=1
Connect 1 -- 7  weight=1  (final edge)

-----

DECODED TREE EDGES
-----

(4, 3)  weight=1
(3, 2)  weight=1
(5, 1)  weight=1
(6, 2)  weight=1
(2, 1)  weight=1
(1, 7)  weight=1

Total weight = 6

```

Рис. 17. Кодирование и декодирование остова кодом Прюфера

При выборе пункта «16» программа проверяет, является ли граф эйлеровым. Если граф не удовлетворяет условию чётности степеней, выполняется модификация рёбер; после этого строится эйлеров цикл — рис. 18.

LAB 5: EULERIAN GRAPH CHECK AND EULER CYCLE

Connected: YES

Original graph is Eulerian: NO

Original degrees:

Vertex	Degree	Parity
1	4	even
2	5	odd
3	3	odd
4	3	odd
5	5	odd
6	5	odd
7	5	odd

Modification log:

- Odd vertices: 2 3 4 5 6 7
- Added missing edge (2 -- 4).
- Odd vertices: 3 5 6 7
- Added missing edge (3 -- 6).
- Odd vertices: 5 7
- Removed edge (5 -- 7); graph remains connected.

Modified degrees:

```
deg(1) = 4 even
deg(2) = 6 even
deg(3) = 4 even
deg(4) = 4 even
deg(5) = 4 even
deg(6) = 6 even
deg(7) = 4 even
```

Euler cycle:

```
1 -> 2 -> 3 -> 4 -> 2 -> 5 -> 1 -> 6 -> 2 -> 7 -> 4 -> 6 -> 3 -> 5 -> 6 -> 7 -> 1
```

Рис. 18. Проверка графа на эйлеровость, модификация и построение эйлерова цикла

При выборе пункта «17» программа строит минимальный остов алгоритмом Борувки, затем по каждому ребру остова получает фундаментальный разрез. После вывода фундаментальной системы пользователь выбирает номера разрезов, а программа вычисляет их симметрическую разность — рис. 19 и рис. 20.

Enter choice:17

Building MST with Boruvka (Lab 4 algorithm)...

--- Round 1 ---

Components: {1} {2} {3} {4} {5} {6} {7}

Add edge (1, 2) weight=1

Add edge (2, 3) weight=1

Add edge (3, 4) weight=1

Add edge (1, 5) weight=1

Add edge (2, 6) weight=1

Add edge (1, 7) weight=1

BORUVKA MST RESULT

MST edges:

Arc	weight
-----	--------

(1 -> 2)	1
----------	---

(2 -> 3)	1
----------	---

(3 -> 4)	1
----------	---

(1 -> 5)	1
----------	---

(2 -> 6)	1
----------	---

(1 -> 7)	1
----------	---

Total MST weight = 6

Рис. 19. Построение фундаментальной системы разрезов

```

-----
LAB 5: FUNDAMENTAL CUTSET SYSTEM (EVEN VARIANT)
-----

Cutset 1 for tree edge (1 -- 2):
  { (1--2), (1--6), (2--5), (2--7), (3--5), (4--7), (5--6), (6--7) }
Cutset 2 for tree edge (2 -- 3):
  { (2--3), (3--5), (4--6), (4--7) }
Cutset 3 for tree edge (3 -- 4):
  { (3--4), (4--6), (4--7) }
Cutset 4 for tree edge (1 -- 5):
  { (1--5), (2--5), (3--5), (5--6), (5--7) }
Cutset 5 for tree edge (2 -- 6):
  { (1--6), (2--6), (4--6), (5--6), (6--7) }
Cutset 6 for tree edge (1 -- 7):
  { (1--7), (2--7), (4--7), (5--7), (6--7) }

-----

Number of cutsets for symmetric difference:2

Cutset number:1

Cutset number:1

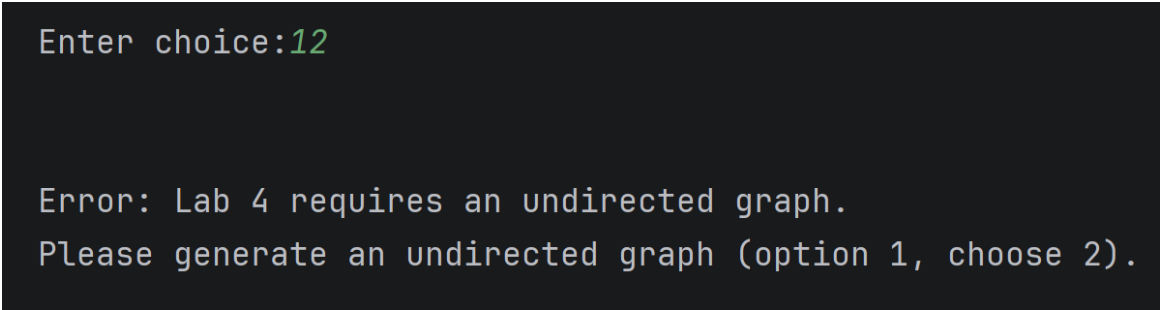
Symmetric difference of selected cutsets (1, 1):
  {  }

```

Рис. 20. Симметрическая разность выбранных фундаментальных разрезов

3.3 Обработка ошибочных ситуаций

В программе предусмотрены проверки корректности пользовательских действий. Если пользователь пытается вызвать алгоритм без предварительно сгенерированного графа или без весовой матрицы, программа выводит сообщение с указанием нужного пункта меню — рис. 21.

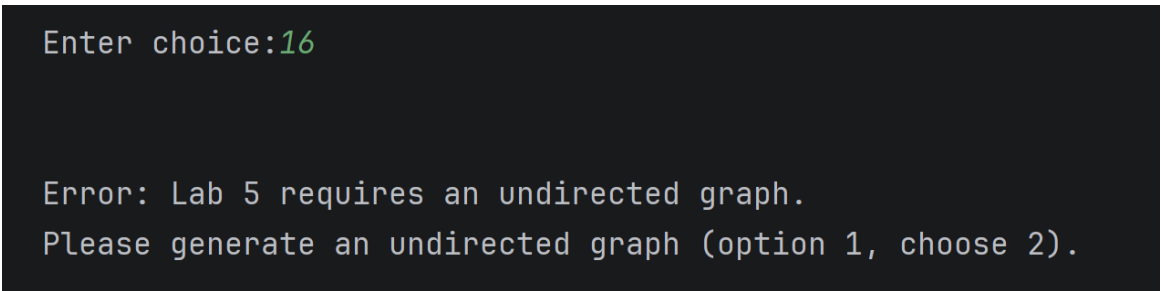


```
Enter choice:12

Error: Lab 4 requires an undirected graph.
Please generate an undirected graph (option 1, choose 2).
```

Рис. 21. Сообщение об ошибке при отсутствии необходимых данных

Для алгоритмов четвёртой и пятой лабораторных работ дополнительно проверяется тип графа. Если пользователь запускает алгоритм, требующий неориентированный граф, для ориентированного графа, программа сообщает о необходимости сгенерировать неориентированный граф — рис. 22.

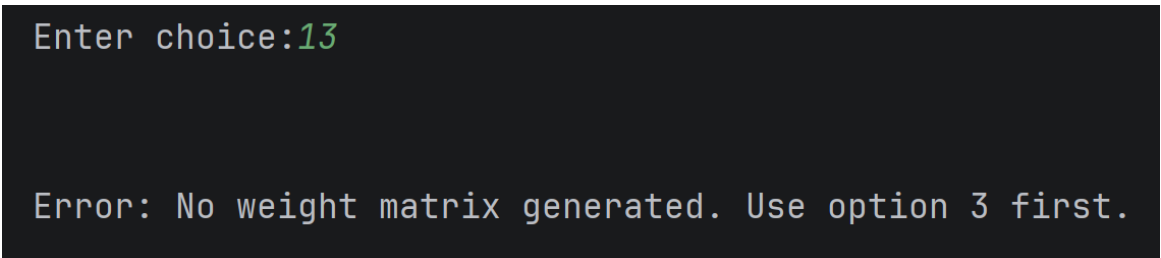


```
Enter choice:16

Error: Lab 5 requires an undirected graph.
Please generate an undirected graph (option 1, choose 2).
```

Рис. 22. Обработка неверного типа графа для алгоритма Эйлера

Также проверяется корректность номеров вершин и допустимость выбора истока и стока. При некорректном вводе программа не завершает работу аварийно, а выводит понятное сообщение и возвращает пользователя в меню — рис. 23.



```
Enter choice:13

Error: No weight matrix generated. Use option 3 first.
```

Рис. 23. Обработка некорректного пользовательского ввода

Заключение

По результатам выполнения лабораторных работ можно сделать следующие выводы:

В первой лабораторной работе был реализован алгоритм генерации случайного связного ациклического графа с использованием нормального распределения. Для полученного графа были вычислены эксцентриситеты вершин, радиус, диаметр, центр и диаметральные вершины. Также был реализован метод Шимбелла для поиска минимальных и максимальных путей заданной длины и алгоритм определения всех маршрутов между двумя wybranнми вершинами.

Во второй лабораторной работе был выполнен обход графа поиском в глубину. Для взвешенного графа был реализован алгоритм Флойда-Уоршалла, позволяющий находить кратчайшие пути между всеми парами вершин и восстанавливать путь между выбраннми пользователем вершинами. В программе также выводится количество итераций, что позволяет оценивать трудоёмкость выполненных алгоритмов.

В третьей лабораторной работе на основе ориентированного графа были сгенерированы матрицы пропускных способностей и стоимостей. Для построенной транспортной сети был найден максимальный поток алгоритмом Форда-Фалкерсона. После этого был вычислен поток минимальной стоимости для величины, равной $\lfloor 2/3 \cdot \varphi_{\max} \rfloor$, с использованием ранее реализованного алгоритма Флойда-Уоршалла для поиска кратчайших по стоимости путей.

В четвёртой лабораторной работе для неориентированного графа было найдено число остовных деревьев с помощью матричной теоремы Кирхгофа. Минимальное остовное дерево было построено алгоритмом Борувки. Полученный остов был закодирован кодом Прюфера и затем декодирован с сохранением весов рёбер. Также был реализован поиск минимального рёберного покрытия как для исходного графа, так и для построенного остова.

В пятой лабораторной работе была выполнена проверка графа на эйлеровость. Если граф не являлся эйлеровым, программа выполняла его модификацию с выводом внесённых изменений и затем строила эйлеров цикл. Для чётного варианта была реализована фундаментальная система разрезов на основе минимального остова, а также получение других разрезов с помощью операции симметрической разности выбранных фундаментальных разрезов.

Выполнение лабораторных работ позволило изучить и реализовать основные алгоритмы теории графов для ориентированных и неориентированных графов: обходы, кратчайшие пути, потоки, остовы, покрытия, кодирование деревьев, эйлеровы циклы и фундаментальные разрезы. Все алгоритмы объединены в одну консольную программу с меню, что позволяет последовательно генерировать граф, строить необходимые матрицы и применять к ним алгоритмы из разных лабораторных работ.

Достоинством программы является модульная структура: общие сущности графа, генерации распределения и весовой матрицы вынесены в отдельные файлы, а алгоритмы разделены по лабораторным работам. Использование контейнеров STL, в частности `vector`, упрощает работу с матрицами и списками рёбер. В программе предусмотрены проверки пользовательского ввода и ограничения на запуск алгоритмов, например требование ориентированного графа для потоков и неориентированного графа для остовов, эйлерова цикла и разрезов.

К недостаткам можно отнести отсутствие графического представления графа: результаты выводятся в консоль в виде матриц, списков рёбер и текстовых протоколов работы алгоритмов. Кроме того, некоторые алгоритмы имеют не самую оптимальную асимптотику для больших графов, например поиск потока минимальной стоимости через повторное применение алгоритма Флойда-Уоршалла. В дальнейшем программу можно расширить визуализацией графов и добавить более эффективные реализации для работы с графами большого размера.

Лабораторные работы были выполнены на языке C++ в среде разработки CLion.

Список использованной литературы

- [1] Новиков Ф. А. Дискретная математика для программистов / Ф. А. Новиков. — 3-е изд. — СПб.: Питер, 2009. — 364 с.
- [2] Шапоров С. Д. Дискретная математика / С. Д. Шапоров. — СПб.: БХВ-Петербург, 2010. — 400 с.
- [3] Романовский И. В. Дискретный анализ / И. В. Романовский. — 3-е изд. — СПб.: Невский диалект; БХВ-Петербург, 2003. — 320 с.
- [4] Хаггарт Р. Дискретная математика для программистов / Р. Хаггарт; пер. с англ. — М.: Техносфера, 2005. — 326 с.
- [5] Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы. Построение и анализ / пер. с англ. — 3-е изд. — М.: Вильямс, 2013. — 1328 с.
- [6] Асанов М. О., Баранский В. А., Расин В. В. Дискретная математика: графы, матроиды, алгоритмы / М. О. Асанов, В. А. Баранский, В. В. Расин. — 2-е изд. — СПб.: Лань, 2010. — 368 с.
- [7] Вадзинский Р. Н. Справочник по вероятностным распределениям / Р. Н. Вадзинский. — СПб.: Наука, 2001. — 295 с.
- [8] Эрикссон Дж. Алгоритмы / Дж. Эрикссон; пер. с англ. — М.: Вильямс, 2022. — 960 с.
- [9] Секция «Телематика» / текст: электронный. — URL: <https://tema.spbstu.ru/tgraph/> (Дата обращения: 20.05.2026)

Приложение А. Общие модули и объединяющая программа

В приложении приведён полный исходный код общих модулей, используемых всеми лабораторными работами, а также код объединяющей консольной программы с меню.

```
1 #pragma once
2
3 constexpr double SHIMBELL_INF = 1e15;
4 constexpr double SHIMBELL_NINF = -1e15;
```

Листинг 2. Файл constants.h

```
1 #pragma once
2 #include <vector>
3 #include <iostream>
4
5 class Graph {
6 public:
7     int n;
8     bool directed;
9     std::vector<std::vector<int>>> adj; // adj[i][j] == 1 if edge i->j exists
10
11     Graph() : n(0), directed(false) {}
12     Graph(int n, bool directed = false);
13
14     void addEdge(int u, int v);
15     bool hasEdge(int u, int v) const;
16
17     // Returns out-neighbors (or all neighbors for undirected)
18     std::vector<int> neighbors(int u) const;
19
20     // Just the adjacency matrix (used right after generation)
21     void printAdjMatrix() const;
22
23     // Full view: adjacency list + edge list + adjacency matrix
24     // weightMatrix may be nullptr - if provided it is printed as well
25     void printFull(const std::vector<std::vector<double>>>* weightMatrix = nullptr) const;
26
27     bool isEmpty() const { return n == 0; }
28 };
```

Листинг 3. Файл graph.h

```
1 #include "graph.h"
2 #include "constants.h" // SHIMBELL_INF, SHIMBELL_NINF
3 #include <iomanip>
4 #include <string>
5
6 static inline int D(int v) { return v + 1; }
7
8 Graph::Graph(int n, bool directed)
9     : n(n), directed(directed), adj(n, std::vector<int>(n, 0)) {}
10
11 void Graph::addEdge(int u, int v) {
12     adj[u][v] = 1;
13     if (!directed)
14         adj[v][u] = 1;
15 }
16
```

```

17 bool Graph::hasEdge(int u, int v) const {
18     return adj[u][v] == 1;
19 }
20
21 std::vector<int> Graph::neighbors(int u) const {
22     std::vector<int> result;
23     for (int v = 0; v < n; v++)
24         if (adj[u][v] == 1)
25             result.push_back(v);
26     return result;
27 }
28
29 // -- Just the adjacency matrix -----
30 void Graph::printAdjMatrix() const {
31     const std::string type = directed ? "DIRECTED" : "UNDIRECTED";
32     const std::string sep(60, '-');
33
34     std::cout << "\n " << sep << "\n";
35     std::cout << "  ADJACENCY MATRIX [" << type << "]\n";
36     std::cout << "  " << sep << "\n\n";
37
38     const int CW = 5;
39     std::cout << "      ";
40     for (int j = 0; j < n; j++)
41         std::cout << std::setw(CW) << D(j);
42     std::cout << "\n " << std::string(5 + n * CW, '-') << "\n";
43
44     for (int i = 0; i < n; i++) {
45         std::cout << "  " << std::setw(3) << D(i) << " |";
46         for (int j = 0; j < n; j++) {
47             if (i == j)
48                 std::cout << std::setw(CW) << 0;
49             else if (adj[i][j])
50                 std::cout << std::setw(CW) << 1;
51             else
52                 std::cout << std::setw(CW) << "inf";
53         }
54         std::cout << "\n";
55     }
56
57     if (!directed)
58         std::cout << "\n Note: Undirected => matrix is symmetric (mat[i][j] == mat[j][i])\n";
59     std::cout << "\n " << sep << "\n";
60 }
61
62 // -- Full view -----
63 void Graph::printFull(const std::vector<std::vector<double>>& W) const {
64     const std::string type = directed ? "DIRECTED" : "UNDIRECTED";
65     const std::string sep(60, '-');
66
67     // -- Adjacency List -----
68     std::cout << "\n " << sep << "\n";
69     std::cout << "  ADJACENCY LIST [" << type << "]\n";
70     std::cout << "  " << sep << "\n\n";
71
72     for (int u = 0; u < n; u++) {
73         auto nb = neighbors(u);
74         std::cout << " Vertex " << std::setw(3) << D(u) << " --> ";
75         if (nb.empty()) {

```

```

76         std::cout << "(isolated)";
77     } else {
78         for (int i = 0; i < (int)nb.size(); i++) {
79             if (i) std::cout << ", ";
80             std::cout << D(nb[i]);
81         }
82     }
83     std::cout << "    (degree=" << nb.size() << ")\n";
84 }
85
86 // -- Edge List -----
87 std::cout << "\n  EDGE LIST:\n";
88 const std::string arrow = directed ? "  ---->  " : "  ----  ";
89 for (int u = 0; u < n; u++)
90     for (int v = directed ? 0 : u + 1; v < n; v++)
91         if (adj[u][v])
92             std::cout << "      " << std::setw(3) << D(u)
93                 << arrow << std::setw(3) << D(v) << "\n";
94
95 // -- Adjacency Matrix -----
96 std::cout << "\n  " << sep << "\n";
97 std::cout << "  ADJACENCY MATRIX  [" << type << "]\n";
98 std::cout << "  " << sep << "\n\n";
99
100 const int ACW = 5;
101 std::cout << "      ";
102 for (int j = 0; j < n; j++)
103     std::cout << std::setw(ACW) << D(j);
104 std::cout << "\n  " << std::string(5 + n * ACW, '-') << "\n";
105
106 for (int i = 0; i < n; i++) {
107     std::cout << "  " << std::setw(3) << D(i) << " |";
108     for (int j = 0; j < n; j++) {
109         if (i == j)
110             std::cout << std::setw(ACW) << 0;
111         else if (adj[i][j])
112             std::cout << std::setw(ACW) << 1;
113         else
114             std::cout << std::setw(ACW) << "inf";
115     }
116     std::cout << "\n";
117 }
118 if (!directed)
119     std::cout << "\n  Note: Undirected => matrix is symmetric (mat[i][j] == mat[j][i])\n";
120
121 // -- Weight Matrix (if available) -----
122 if (W) {
123     std::cout << "\n  " << sep << "\n";
124     std::cout << "  WEIGHT MATRIX  (INF = no edge, 0 = self)\n";
125     std::cout << "  " << sep << "\n\n";
126
127     const int CW = 8;
128     std::cout << "      ";
129     for (int j = 0; j < n; j++)
130         std::cout << std::setw(CW) << D(j);
131     std::cout << "\n  " << std::string(5 + n * CW, '-') << "\n";
132
133     for (int i = 0; i < n; i++) {
134         std::cout << "  " << std::setw(3) << D(i) << " |";

```

```

135         for (int j = 0; j < n; j++) {
136             double v = (*W)[i][j];
137             if (v >= SHIMBELL_INF / 2)
138                 std::cout << std::setw(CW) << "INF";
139             else if (v <= SHIMBELL_NINF / 2)
140                 std::cout << std::setw(CW) << "-INF";
141             else
142                 std::cout << std::setw(CW) << static_cast<long long>(v);
143         }
144         std::cout << "\n";
145     }
146 }
147
148 std::cout << "\n " << sep << "\n";
149 }

```

Листинг 4. Файл graph.cpp

```

1 #pragma once
2 #include <vector>
3
4 // Generate one sample from N(0,1) using the Vazirani formula:
5 // x = sqrt(12/n) * (sum_{i=1}^n r_i - n/2)
6 // where r_i ~ Uniform(0,1) and n is the number of uniform samples.
7 double normalSample(int n = 12);
8
9 // Generate a positive integer degree from the distribution.
10 // Returns a value in [1, maxDeg].
11 int sampleDegree(int maxDeg = 6);
12
13 // NEW: "tree" replaced with "graph" - this function is used for both the
14 // undirected graph (where sum = 2*(n-1) matches a spanning tree) and
15 // the directed DAG (where degrees are scaled further by EXTRA_EDGE_SCALE).
16 // The constraint sum == 2*(numVertices-1) and all degrees >= 1 still holds.
17 std::vector<int> generateDegreeSequence(int numVertices);

```

Листинг 5. Файл distribution.h

```

1 #include "distribution.h"
2 #include <cmath>
3 #include <random>
4 #include <algorithm>
5
6
7 static std::mt19937& getRng() {
8     static std::mt19937 rng(std::random_device{}());
9     return rng;
10 }
11
12 double normalSample(int n) {
13     std::uniform_real_distribution<double> uniform(0.0, 1.0);
14     auto& rng = getRng();
15     double sum = 0.0;
16     for (int i = 0; i < n; i++)
17         sum += uniform(rng);
18     // x = sqrt(12/n) * (sum - n/2)
19     return std::sqrt(12.0 / n) * (sum - n / 2.0);
20 }
21
22 int sampleDegree(int maxDeg) {
23     // Map N(0,1) to a positive integer in [1, maxDeg]

```

```

24     double raw = std::abs(normalSample(12));
25     // raw is half-normal; scale to [1, maxDeg]
26     int d = 1 + static_cast<int>(raw * (maxDeg - 1) / 2.5);
27     return std::clamp(d, 1, maxDeg);
28 }
29
30 std::vector<int> generateDegreeSequence(int numVertices) {
31     if (numVertices <= 1) return std::vector<int>(numVertices, 0);
32
33     int target = 2 * (numVertices - 1);
34     std::vector<int> degrees(numVertices);
35     for (int i = 0; i < numVertices; i++)
36         degrees[i] = sampleDegree(numVertices - 1);
37
38     // Adjust sum to target by randomly incrementing/decrementing
39     auto& rng = getRng();
40     std::uniform_int_distribution<int> pick(0, numVertices - 1);
41
42     int sum = 0;
43     for (int d : degrees) sum += d;
44
45     int iterations = 0;
46     while (sum != target && iterations < 100000) {
47         int v = pick(rng);
48         if (sum > target && degrees[v] > 1) {
49             degrees[v]--;
50             sum--;
51         } else if (sum < target && degrees[v] < numVertices - 1) {
52             degrees[v]++;
53             sum++;
54         }
55         iterations++;
56     }
57
58     return degrees;
59 }

```

Листинг 6. Файл distribution.cpp

```

1 #pragma once
2 #include "graph.h"
3
4 // renamed from generateTree() to generateDAG() because the result is not
5 // a tree - a tree has exactly n-1 edges but our directed graph generates up to
6 // n*(n-1)/2 edges. The correct term is DAG (Directed Acyclic Graph).
7 // If directed == false, returns an undirected connected acyclic graph derived
8 // by symmetrizing the directed DAG (ignoring edge directions).
9 Graph generateDAG(int n, bool directed = false);

```

Листинг 7. Файл dag_generator.h

```

1 #include "dag_generator.h"
2 #include "distribution.h"
3 #include <random>
4 #include <vector>
5 #include <numeric>
6 #include <algorithm>
7
8 static std::mt19937& getRng() {
9     static std::mt19937 rng(std::random_device{}());
10    return rng;

```

```

11 }
12
13 // -----
14 // EXTRA_EDGE_SCALE: multiplier applied to the sampled out-degree for each
15 // vertex in the directed DAG. Controls graph density.
16 //
17 // 1.0 = sparse    4.0 = medium    8.0 = dense (current)
18 // -----
19 // separate scales
20 static constexpr double EXTRA_EDGE_SCALE_DIRECTED = 8.0; // dense DAG
21 static constexpr double EXTRA_EDGE_SCALE_UNDIRECTED = 1.5; // sparse enough to vary after
    symmetrize
22
23 static Graph buildDirectedDAG(int n, double scale);
24
25 // Build an undirected graph from the same directed DAG model by ignoring
26 // edge directions. This matches the lab wording: the undirected graph is
27 // obtained from the generated oriented graph. Because the directed graph may
28 // contain extra forward edges, the undirected version is connected and may
29 // contain cycles, so MST algorithms are non-trivial.
30 // -----
31 static Graph buildUndirectedGraph(int n) {
32     Graph dag = buildDirectedDAG(n, EXTRA_EDGE_SCALE_UNDIRECTED);
33     Graph undirected(n, false);
34
35     for (int u = 0; u < n; u++)
36         for (int v = 0; v < n; v++)
37             if (dag.hasEdge(u, v))
38                 undirected.addEdge(u, v);
39
40     return undirected;
41 }
42
43 // -----
44 // DIRECTED: connected acyclic directed graph (DAG)
45 // Algorithm - "generation by vertex degrees" as required by the lab:
46 //
47 // 1. Sample an OUT-DEGREE sequence from the Normal distribution via
48 //    generateDegreeSequence() - same function used for undirected.
49 //    Scale each degree by EXTRA_EDGE_SCALE for density; cap at available
50 //    forward targets so acyclicity is preserved.
51 //
52 // 2. Create a random topological ordering. Edges only go from lower to
53 //    higher position => acyclicity guaranteed by construction.
54 //
55 // 3. Spanning path topo[0]->topo[1]->...->topo[n-1] added first to
56 //    guarantee weak connectivity; counts against each vertex's degree budget.
57 //
58 // 4. Fill remaining out-degree budget per vertex with random forward edges.
59 // -----
60 static Graph buildDirectedDAG(int n, double scale) {
61     Graph g(n, true);
62     if (n <= 1) return g;
63     if (n == 2) { g.addEdge(0, 1); return g; }
64
65     auto& rng = getRng();
66
67     // Step 1: random topological ordering
68     std::vector<int> topo(n);
69     std::iota(topo.begin(), topo.end(), 0);

```



```

70     std::shuffle(topo.begin(), topo.end(), rng);
71
72     // Step 2: sample out-degree for each vertex from Normal distribution,
73     //         scaled by EXTRA_EDGE_SCALE, capped by available forward targets.
74     std::vector<int> outDeg = generateDegreeSequence(n);
75     for (int i = 0; i < n; i++) {
76         int maxOut = n - i - 1; // topo[i] can reach at most (n-i-1) vertices ahead
77         outDeg[i] = std::min((int)std::round(outDeg[i] * scale), maxOut);
78         outDeg[i] = std::max(outDeg[i], (i < n - 1) ? 1 : 0); // at least 1 (except last)
79     }
80
81     // Step 3: spanning path for connectivity; consume one slot per vertex
82     for (int i = 0; i < n - 1; i++) {
83         g.addEdge(topo[i], topo[i + 1]);
84         outDeg[i]--;
85     }
86
87     // Step 4: fill remaining degree budget with random forward edges
88     for (int i = 0; i < n - 2; i++) {
89         if (outDeg[i] <= 0) continue;
90
91         std::vector<int> candidates;
92         for (int j = i + 2; j < n; j++)
93             candidates.push_back(j);
94         std::shuffle(candidates.begin(), candidates.end(), rng);
95
96         int added = 0;
97         for (int j : candidates) {
98             if (added >= outDeg[i]) break;
99             int v = topo[j];
100             if (!g.hasEdge(topo[i], v)) {
101                 g.addEdge(topo[i], v);
102                 added++;
103             }
104         }
105     }
106
107     return g;
108 }
109
110 // -----
111 // Public entry point
112 //
113 // renamed to generateDAG() - the result is a DAG, not a tree.
114 // A tree has exactly n-1 edges; our directed graph has up to n*(n-1)/2 edges.
115 // -----
116 Graph generateDAG(int n, bool directed) {
117     if (directed) return buildDirectedDAG(n, EXTRA_EDGE_SCALE_DIRECTED);
118     else         return buildUndirectedGraph(n);
119 }

```

Листинг 8. Файл dag_generator.cpp

```

1 #pragma once
2 #include "graph.h"
3 #include "constants.h"
4 #include "distribution.h"
5 #include <vector>
6
7 using Matrix = std::vector<std::vector<double>>>;
8

```

```

9  enum WeightMode { POSITIVE, NEGATIVE, MIXED };
10
11 // Shared weight matrix generator used by all labs.
12 // Non-edges -> SHIMBELL_INF, diagonal -> SHIMBELL_INF (exact k-step semantics).
13 // Weights sampled from N(0,1)*5 rounded to integer, sign controlled by mode.
14 Matrix generateWeightMatrix(const Graph& g, WeightMode mode);

```

Листинг 9. Файл weight_matrix.h

```

1  #include "weight_matrix.h"
2  #include <cmath>
3
4  static double sampleWeight(WeightMode mode) {
5      double raw = normalSample(12) * 5.0;
6      int w = static_cast<int>(std::round(raw));
7      switch (mode) {
8          case POSITIVE: return (w <= 0) ? 1 : w;
9          case NEGATIVE: return (w >= 0) ? -1 : w;
10         case MIXED:
11             default: return (w == 0) ? 1 : w;
12     }
13 }
14
15 Matrix generateWeightMatrix(const Graph& g, WeightMode mode) {
16     int n = g.n;
17     Matrix W(n, std::vector<double>(n, SHIMBELL_INF));
18     for (int i = 0; i < n; i++) {
19         for (int j = g.directed ? 0 : i + 1; j < n; j++) {
20             if (i == j) continue;
21             if (g.hasEdge(i, j)) {
22                 double w = sampleWeight(mode);
23                 W[i][j] = w;
24                 if (!g.directed) W[j][i] = w; // undirected edge has one weight
25             }
26         }
27     }
28     return W;
29 }

```

Листинг 10. Файл weight_matrix.cpp

```

1  #include <iostream>
2  #include <limits>
3  #include <string>
4  #include <iomanip>
5  #include <cmath>
6  #include "graph.h"
7  #include "dag_generator.h"
8  #include "weight_matrix.h"
9  #include "eccentricity.h"
10 #include "shimbell.h"
11 #include "path_counter.h"
12 #include "dfs.h"
13 #include "floyd_warshall.h"
14 #include "ford_fulkerson.h"
15 #include "min_cost_flow.h"
16 #include "boruvka.h"
17 #include "edge_cover.h"
18 #include "kirchhoff.h"
19 #include "prufer.h"
20 #include "eulerian.h"

```

```

21 #include "fundamental_cutsets.h"
22
23
24 // -- Global state -----
25 static Graph gGraph;
26 static bool gGraphReady = false;
27
28 // Shared weight matrix (used by Lab 1 Shimbell, Lab 2 Floyd-Warshall, Lab 4 MST)
29 static Matrix gWeightMatrix;
30 static bool gWeightReady = false;
31
32 // static std::vector<std::vector<double>> gFWWeightMatrix; // removed: FW now uses
    gWeightMatrix directly
33 // static bool gFWWeightReady = false;
34
35 // Lab 3
36 static std::vector<std::vector<int>> gCapMatrix;
37 static std::vector<std::vector<int>> gCostMatrix;
38 static bool gFlowMatricesReady = false;
39
40 // Lab 4
41 static BoruvkaResult gMST;
42 static bool gMSTReady = false;
43 static PruferResult gPrufer;
44 static bool gPruferReady = false;
45
46 static inline int D(int v) { return v + 1; }
47
48 static void clearInput() {
49     std::cin.clear();
50     std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
51 }
52 static int readInt(const std::string& prompt) {
53     int v;
54     while (true) {
55         std::cout << prompt;
56         if (std::cin >> v) { clearInput(); return v; }
57         std::cout << " Invalid input. Please enter an integer.\n";
58         clearInput();
59     }
60 }
61 static void requireGraph() {
62     std::cout << " Error: No graph generated yet. Use option 1 first.\n";
63 }
64 static void requireWeight() {
65     std::cout << " Error: No weight matrix generated. Use option 3 first.\n";
66 }
67 static void requireFlowMatrices() {
68     std::cout << " Error: No capacity/cost matrices. Use option 9 first.\n";
69 }
70 static void requireMST() {
71     std::cout << " Error: No MST computed. Use option 13 (Boruvka) first.\n";
72 }
73 static bool requireLab4Undirected() {
74     if (!gGraphReady) {
75         requireGraph();
76         return false;
77     }
78     if (gGraph.directed) {
79         std::cout << " Error: Lab 4 requires an undirected graph.\n"

```

```

80         << " Please generate an undirected graph (option 1, choose 2).\n";
81         return false;
82     }
83     return true;
84 }
85 static bool requireLab5Undirected() {
86     if (!gGraphReady) {
87         requireGraph();
88         return false;
89     }
90     if (gGraph.directed) {
91         std::cout << " Error: Lab 5 requires an undirected graph.\n"
92         << " Please generate an undirected graph (option 1, choose 2).\n";
93         return false;
94     }
95     return true;
96 }
97
98 // -- 1. Generate graph -----
99 void menuGenerateGraph() {
100     int n = readInt(" Number of vertices: ");
101     if (n <= 0) { std::cout << " Must be > 0.\n"; return; }
102     std::cout << " Graph type:\n 1. Directed\n 2. Undirected\n";
103     bool directed = (readInt(" Choose [1/2]: ") == 1);
104     gGraph = generateDAG(n, directed);
105     gGraphReady = true;
106     gWeightReady = false;
107     gFlowMatricesReady = false;
108     gMSTReady = false;
109     gPruferReady = false;
110     gGraph.printAdjMatrix();
111     std::cout << " Graph with " << n << " vertices generated";
112     if (!directed) std::cout << " (undirected)";
113     std::cout << ".\n Generate weight matrix with option 3.\n";
114 }
115
116 // -- 3. Generate weight matrix (shared) -----
117 void menuGenerateWeightMatrix() {
118     if (!gGraphReady) { requireGraph(); return; }
119     std::cout << " Weight mode:\n 1. Positive only\n 2. Negative only\n 3. Mixed\n";
120     int wChoice = readInt(" Choose [1/2/3]: ");
121     WeightMode mode;
122     switch (wChoice) {
123         case 2: mode = NEGATIVE; break;
124         case 3: mode = MIXED; break;
125         default: mode = POSITIVE; break;
126     }
127     gWeightMatrix = generateWeightMatrix(gGraph, mode);
128     gWeightReady = true;
129     gMSTReady = false;
130     gPruferReady = false;
131     std::cout << " Weight matrix generated.\n";
132     // print it
133     const int W = 8, n = gGraph.n;
134     std::cout << "\n ";
135     for (int j = 0; j < n; j++) std::cout << std::setw(W) << D(j);
136     std::cout << "\n " << std::string(5 + n * W, '-') << "\n";
137     for (int i = 0; i < n; i++) {
138         std::cout << " " << std::setw(3) << D(i) << " |";

```

```

139     for (int j = 0; j < n; j++) {
140         double v = gWeightMatrix[i][j];
141         if (v >= SHIMBELL_INF / 2) std::cout << std::setw(W) << "INF";
142         else if (v <= SHIMBELL_NINF / 2) std::cout << std::setw(W) << "-INF";
143         else std::cout << std::setw(W) << static_cast<long long>(v);
144     }
145     std::cout << "\n";
146 }
147 }
148
149 // -- Shimbell pretty-print -----
150 static void printShimbellMatrix(const Matrix& M) {
151     int n = static_cast<int>(M.size());
152     const int W = 8;
153     std::cout << " ";
154     for (int j = 0; j < n; j++) std::cout << std::setw(W) << D(j);
155     std::cout << "\n " << std::string(5 + n * W, '-') << "\n";
156     for (int i = 0; i < n; i++) {
157         std::cout << " " << std::setw(3) << D(i) << " |";
158         for (int j = 0; j < n; j++) {
159             if (M[i][j] >= SHIMBELL_INF / 2) std::cout << std::setw(W) << "INF";
160             else if (M[i][j] <= SHIMBELL_NINF / 2) std::cout << std::setw(W) << "-INF";
161             else std::cout << std::setw(W) << static_cast<long long>(M[i][j]);
162         }
163         std::cout << "\n";
164     }
165 }
166
167 // -- 4. Eccentricities -----
168 void menuEccentricity() {
169     if (!gGraphReady) { requireGraph(); return; }
170     auto m = computeMetrics(gGraph);
171     printMetrics(m);
172 }
173
174 // -- 5. Shimbell -----
175 void menuShimbell() {
176     if (!gGraphReady) { requireGraph(); return; }
177     if (!gWeightReady) { requireWeight(); return; }
178     const std::string sep(60, '-');
179     std::cout << "\n " << sep << "\n WEIGHT MATRIX\n " << sep << "\n\n";
180     printShimbellMatrix(gWeightMatrix);
181     int k = readInt("\n Enter k (number of steps): ");
182     if (k < 0) { std::cout << " k must be >= 0.\n"; return; }
183     auto [minMat, maxMat] = shimbell(gWeightMatrix, k);
184     std::cout << "\n " << sep << "\n SHIMBELL MIN-path (k=" << k << ")\n " << sep << "\n\n";
185     printShimbellMatrix(minMat);
186     std::cout << "\n " << sep << "\n SHIMBELL MAX-path (k=" << k << ")\n " << sep << "\n\n";
187     printShimbellMatrix(maxMat);
188 }
189
190 // -- 6. Routes -----
191 void menuPaths() {
192     if (!gGraphReady) { requireGraph(); return; }
193     if (!gWeightReady) { requireWeight(); return; }
194     gGraph.printFull(&gWeightMatrix);
195     std::cout << " Vertices are numbered 1 to " << gGraph.n << ".\n";
196     int src = readInt(" Source vertex: ") - 1;

```

```

197     int dst = readInt(" Destination vertex: ") - 1;
198     if (src < 0 || src >= gGraph.n || dst < 0 || dst >= gGraph.n) {
199         std::cout << " Vertex out of range.\n"; return;
200     }
201     auto paths = findAllPaths(gGraph, src, dst);
202     printPaths(paths, src, dst);
203 }
204
205 // -- 7. DFS -----
206 void menuDFS() {
207     if (!gGraphReady) { requireGraph(); return; }
208     int start = readInt(" Enter start vertex: ") - 1;
209     if (start < 0 || start >= gGraph.n) { std::cout << " Vertex out of range.\n"; return; }
210     dfs(gGraph, start);
211 }
212
213 // -- 8. Floyd-Warshall -----
214 void menuFloydWarshall() {
215     if (!gGraphReady) { requireGraph(); return; }
216     if (!gWeightReady) { requireWeight(); return; }
217
218     // Build FW matrix from shared weight matrix: Shimbell has INF on diagonal,
219     // Floyd-Warshall needs 0 there.
220     const int n = gGraph.n;
221     std::vector<std::vector<double>> fw(gWeightMatrix);
222     for (int i = 0; i < n; i++) fw[i][i] = 0.0;
223
224     int src = readInt(" Source vertex: ") - 1;
225     int dst = readInt(" Destination vertex: ") - 1;
226     if (src < 0 || src >= n || dst < 0 || dst >= n) {
227         std::cout << " Vertex out of range.\n"; return;
228     }
229     FloydResult res = floydWarshall(gGraph, fw);
230     printFloydResult(res, fw, n, src, dst);
231 }
232
233 // -- 9. Generate capacity and cost matrices (Lab 3) -----
234 void menuGenerateFlowMatrices() {
235     if (!gGraphReady) { requireGraph(); return; }
236     if (!gGraph.directed) {
237         std::cout << " Error: Lab 3 requires a directed graph.\n"
238             << " Please generate a directed graph (option 1, choose 1).\n";
239         return;
240     }
241     gCapMatrix = generateCapacityMatrix(gGraph);
242     gCostMatrix = generateCostMatrix(gGraph);
243     gFlowMatricesReady = true;
244     const int n = gGraph.n;
245     printCapacityMatrix(gCapMatrix, n);
246     printCostMatrix(gCostMatrix, n);
247     // Show out-degree / in-degree to help pick source and sink
248     std::cout << "\n Vertex degrees (to help choose source / sink):\n";
249     std::cout << " Vertex | Out-degree | In-degree\n";
250     std::cout << " " << std::string(34, '-') << "\n";
251     for (int i = 0; i < n; i++) {
252         int out = 0, in = 0;
253         for (int j = 0; j < n; j++) {
254             if (gCapMatrix[i][j] > 0) out++;
255             if (gCapMatrix[j][i] > 0) in++;
256         }

```

```

257         std::cout << "          " << std::setw(3) << i+1
258             << " |          " << std::setw(3) << out
259             << " |          " << std::setw(3) << in << "\n";
260     }
261     std::cout << "    Good source: high out-degree, low in-degree.\n";
262     std::cout << "    Good sink:   high in-degree, low out-degree.\n";
263 }
264
265 // -- 10. Ford-Fulkerson -----
266 void menuFordFulkerson() {
267     if (!gGraphReady) { requireGraph(); return; }
268     if (!gFlowMatricesReady) { requireFlowMatrices(); return; }
269     const int n = gGraph.n;
270     int src = readInt("    Source vertex: ") - 1;
271     int dst = readInt("    Sink vertex:   ") - 1;
272     if (src < 0 || src >= n || dst < 0 || dst >= n) {
273         std::cout << "    Vertex out of range.\n"; return;
274     }
275     if (src == dst) {
276         std::cout << "    Source and sink must be different vertices.\n"; return;
277     }
278     FFResult res = fordFulkerson(n, gCapMatrix, src, dst);
279     if (res.maxFlow == 0)
280         std::cout << "    No path exists from vertex " << src+1 << " to vertex " << dst+1
281             << ".\n    Max flow = 0. Choose a different source/sink pair.\n"
282             << "    Hint: pick a vertex with only outgoing edges as source,\n"
283             << "                and a vertex with only incoming edges as sink.\n";
284     else
285         printFFResult(res, gCapMatrix, n);
286 }
287
288 // -- 11. Min-cost flow -----
289 void menuMinCostFlow() {
290     if (!gGraphReady) { requireGraph(); return; }
291     if (!gFlowMatricesReady) { requireFlowMatrices(); return; }
292     const int n = gGraph.n;
293     int src = readInt("    Source vertex: ") - 1;
294     int dst = readInt("    Sink vertex:   ") - 1;
295     if (src < 0 || src >= n || dst < 0 || dst >= n) {
296         std::cout << "    Vertex out of range.\n"; return;
297     }
298     if (src == dst) {
299         std::cout << "    Source and sink must be different vertices.\n"; return;
300     }
301     MCFResult res = minCostFlow(n, gCapMatrix, gCostMatrix, src, dst);
302     printMCFResult(res, gCapMatrix, gCostMatrix, n);
303 }
304
305 // -- 12. Kirchhoff (spanning tree count) -----
306 void menuKirchhoff() {
307     if (!requireLab4Undirected()) { return; }
308     long long count = countSpanningTrees(gGraph);
309     printKirchhoffResult(gGraph, count);
310 }
311
312 // -- 13. Boruvka (MST) -----
313 void menuBoruvka() {
314     if (!requireLab4Undirected()) { return; }
315     if (!gWeightReady) { requireWeight(); return; }
316     gMST = boruvka(gGraph, gWeightMatrix);

```

```

317     gMSTReady    = true;
318     gPruferReady = false;
319     printBoruvkaResult(gMST);
320 }
321
322 // -- 14. Edge cover -----
323 void menuEdgeCover() {
324     if (!requireLab4Undirected()) { return; }
325     if (!gWeightReady) { requireWeight(); return; }
326     std::cout << "  Work on:\n    1. MST (requires Boruvka, option 13)\n    2. Full graph\n"
327     ;
328     bool useMST = (readInt("  Choose [1/2]: ") == 1);
329     if (useMST && !gMSTReady) { requireMST(); return; }
330     EdgeCoverResult res = minEdgeCover(gGraph, gWeightMatrix, useMST,
331                                     useMST ? gMST.edges : std::vector<MSTEdge>{});
332     printEdgeCoverResult(res);
333 }
334
335 // -- 15. Prufer encode / decode -----
336 void menuPrufer() {
337     if (!requireLab4Undirected()) { return; }
338     if (!gMSTReady) { requireMST(); return; }
339     if ((int)gMST.edges.size() != gGraph.n - 1) {
340         std::cout << "  Error: Prüfer coding requires a spanning tree with n-1 edges.\n"
341         << "  Re-run Boruvka on a connected undirected weighted graph.\n";
342         return;
343     }
344     gPrufer = pruferEncode(gMST.edges, gGraph.n);
345     gPruferReady = true;
346     printPruferResult(gPrufer, gGraph.n);
347     auto decoded = pruferDecode(gPrufer.code, gPrufer.weights, gGraph.n);
348     printDecodedTree(decoded);
349 }
350
351 // -- 16. Eulerian graph / Euler cycle -----
352 void menuEulerianCycle() {
353     if (!requireLab5Undirected()) { return; }
354     EulerianResult res = buildEulerianCycle(gGraph);
355     printEulerianResult(res);
356 }
357
358 // -- 17. Fundamental cutsets (variant 36 is even) -----
359 void menuFundamentalCutsets() {
360     if (!requireLab5Undirected()) { return; }
361     if (!gWeightReady) { requireWeight(); return; }
362
363     std::cout << "  Building MST with Boruvka (Lab 4 algorithm)...\n";
364     gMST = boruvka(gGraph, gWeightMatrix);
365     gMSTReady = true;
366     gPruferReady = false;
367     printBoruvkaResult(gMST);
368
369     if ((int)gMST.edges.size() != gGraph.n - 1) {
370         std::cout << "  Error: fundamental cutsets require a spanning tree.\n";
371         return;
372     }
373
374     auto cutsets = buildFundamentalCutsets(gGraph, gMST.edges);
375     printFundamentalCutsets(cutsets);

```



```

376     if (cutsets.empty()) {
377         return;
378     }
379
380     int count = readInt("  Number of cutsets for symmetric difference: ");
381     if (count < 0) {
382         std::cout << "  Count must be >= 0.\n";
383         return;
384     }
385
386     std::vector<int> selected;
387     for (int i = 0; i < count; i++) {
388         int idx = readInt("  Cutset number: ");
389         if (idx < 1 || idx > (int)cutsets.size()) {
390             std::cout << "  Ignoring invalid cutset number " << idx << ".\n";
391             continue;
392         }
393         selected.push_back(idx);
394     }
395     printCutsetSymmetricDifference(cutsets, selected);
396 }
397
398 // -- Menu -----
399 void printMenu() {
400     std::cout << "\n===== \n"
401         << "  GRAPH THEORY LABS\n"
402         << "===== \n"
403         << "  1. Generate graph\n"
404         << "  2. Show graph (adjacency list / matrix)\n"
405         << "  3. Generate weight matrix\n"
406         << "  ====Lab 1==== \n"
407         << "  4. Eccentricities, center, diametral vertices\n"
408         << "  5. Shimbell's method (min/max paths, k steps)\n"
409         << "  6. Find all routes between two vertices\n"
410         << "  ====Lab 2==== \n"
411         << "  7. DFS traversal\n"
412         << "  8. Shortest path: Floyd-Warshall\n"
413         << "  ====Lab 3==== \n"
414         << "  9.  Generate capacity and cost matrices\n"
415         << "  10. Max flow: Ford-Fulkerson\n"
416         << "  11. Min-cost flow\n"
417         << "  ====Lab 4==== \n"
418         << "  12. Kirchhoff: count spanning trees\n"
419         << "  13. Boruvka: minimum spanning tree\n"
420         << "  14. Minimum edge cover\n"
421         << "  15. Prufer encode / decode\n"
422         << "  ====Lab 5==== \n"
423         << "  16. Eulerian check / modify / Euler cycle\n"
424         << "  17. Fundamental cutsets from MST\n"
425         << "  0.  Exit\n"
426         << "----- \n";
427 }
428
429 int main() {
430     int choice = -1;
431     while (choice != 0) {
432         printMenu();
433         choice = readInt("  Enter choice: ");
434         std::cout << "\n";
435         switch (choice) {

```

```

436     case 1: menuGenerateGraph(); break;
437     case 2:
438         if (gGraphReady)
439             gGraph.printFull(gWeightReady ? &gWeightMatrix : nullptr);
440         else requireGraph();
441         break;
442     case 3: menuGenerateWeightMatrix(); break;
443     case 4: menuEccentricity(); break;
444     case 5: menuShimbell(); break;
445     case 6: menuPaths(); break;
446     case 7: menuDFS(); break;
447     case 8: menuFloydWarshall(); break;
448     case 9: menuGenerateFlowMatrices(); break;
449     case 10: menuFordFulkerson(); break;
450     case 11: menuMinCostFlow(); break;
451     case 12: menuKirchhoff(); break;
452     case 13: menuBoruvka(); break;
453     case 14: menuEdgeCover(); break;
454     case 15: menuPrufer(); break;
455     case 16: menuEulerianCycle(); break;
456     case 17: menuFundamentalCutsets(); break;
457     case 0: std::cout << " Goodbye.\n"; break;
458     default: std::cout << " Unknown option.\n"; break;
459 }
460 }
461 return 0;
462 }

```

Листинг 11. Файл main.cpp объединяющей программы

Приложение Б. Исходный код лабораторной работы №1

```
1 #pragma once
2 #include "graph.h"
3 #include <vector>
4
5 // BFS from src following directed (or undirected) edges.
6 // Returns dist[v] = shortest-path distance, or -1 if unreachable.
7 std::vector<int> bfs(const Graph& g, int src);
8
9 struct GraphMetrics {
10     std::vector<int> eccentricities;           // ecc[v]; 0 if no other vertex is reachable
11     std::vector<std::vector<int>>> distMatrix; // distMatrix[u][v] = BFS distance, -1 if
12     // unreachable
13     int radius;                               // min eccentricity among vertices with ecc > 0
14     int diameter;                             // max eccentricity
15     std::vector<int> center;                   // vertices achieving the radius
16     std::vector<int> diametralVertices;       // vertices achieving the diameter
17 };
18
19 GraphMetrics computeMetrics(const Graph& g);
20 void printMetrics(const GraphMetrics& m);
```

Листинг 12. Файл eccentricity.h

```
1 #include "eccentricity.h"
2 #include <queue>
3 #include <algorithm>
4 #include <iostream>
5 #include <iomanip>
6
7 static inline int D(int v) { return v + 1; }
8
9 std::vector<int> bfs(const Graph& g, int src) {
10     std::vector<int> dist(g.n, -1);
11     std::queue<int> q;
12     dist[src] = 0;
13     q.push(src);
14     while (!q.empty()) {
15         int u = q.front(); q.pop();
16         for (int v : g.neighbors(u)) {
17             if (dist[v] == -1) {
18                 dist[v] = dist[u] + 1;
19                 q.push(v);
20             }
21         }
22     }
23     return dist;
24 }
25
26 GraphMetrics computeMetrics(const Graph& g) {
27     GraphMetrics m;
28     m.eccentricities.resize(g.n, 0);
29     m.distMatrix.resize(g.n);
30
31     for (int v = 0; v < g.n; v++) {
32         m.distMatrix[v] = bfs(g, v);
33         int maxDist = 0;
```

```

34     for (int u = 0; u < g.n; u++) {
35         if (u != v && m.distMatrix[v][u] != -1)
36             maxDist = std::max(maxDist, m.distMatrix[v][u]);
37     }
38     m.eccentricities[v] = maxDist;
39 }
40
41 m.diameter = *std::max_element(m.eccentricities.begin(), m.eccentricities.end());
42
43 m.radius = m.diameter;
44 for (int v = 0; v < g.n; v++)
45     if (m.eccentricities[v] > 0)
46         m.radius = std::min(m.radius, m.eccentricities[v]);
47 if (m.diameter == 0) m.radius = 0;
48
49 for (int v = 0; v < g.n; v++) {
50     if (m.eccentricities[v] == m.radius && m.radius > 0)
51         m.center.push_back(v);
52     if (m.eccentricities[v] == m.diameter && m.diameter > 0)
53         m.diametralVertices.push_back(v);
54 }
55
56 return m;
57 }
58
59 void printMetrics(const GraphMetrics& m) {
60     int n = static_cast<int>(m.eccentricities.size());
61
62     // -- Distance matrix -----
63     std::cout << "\n";
64     std::cout << "          ";
65     for (int j = 0; j < n; j++)
66         std::cout << std::setw(5) << D(j);
67     std::cout << "\n " << std::string(5 + n * 5, '-') << "\n";
68
69     for (int i = 0; i < n; i++) {
70         std::cout << " " << std::setw(3) << D(i) << " |";
71         for (int j = 0; j < n; j++) {
72             int d = m.distMatrix[i][j];
73             if (d == -1)
74                 std::cout << std::setw(5) << "inf";
75             else
76                 std::cout << std::setw(5) << d;
77         }
78         std::cout << "\n";
79     }
80
81     // -- Eccentricity table -----
82     std::cout << "\n";
83     std::cout << "  Vertex  | Eccentricity | Role\n";
84     std::cout << "  -----+-----+-----\n";
85     for (int v = 0; v < n; v++) {
86         int e = m.eccentricities[v];
87         std::string role = "";
88         if (m.diameter > 0 && e == m.diameter) role = "DIAMETRICAL";
89         if (m.radius > 0 && e == m.radius) {
90             role = (role.empty() ? "" : role + " + ");
91             role += "CENTER";
92         }
93         if (role.empty()) role = "-";

```

```

94         std::cout << " " << std::setw(6) << D(v)
95             << " | " << std::setw(8) << e
96             << " | " << role << "\n";
97     }
98
99     // -- Summary -----
100    std::cout << "\n Radius   = " << m.radius << "\n";
101    std::cout << " Diameter = " << m.diameter << "\n";
102
103    std::cout << "\n CENTER vertices   = { ";
104    for (int v : m.center) std::cout << D(v) << " ";
105    std::cout << "}\n";
106
107    std::cout << " DIAMETRICAL vertices = { ";
108    for (int v : m.diametralVertices) std::cout << D(v) << " ";
109    std::cout << "}\n";
110 }

```

Листинг 13. Файл eccentricity.cpp

```

1  #pragma once
2  #include "graph.h"
3  #include "constants.h"
4  #include "weight_matrix.h" // Matrix, WeightMode, generateWeightMatrix (shared)
5  #include <vector>
6
7  // Shimbell's algorithm.
8  //   k == 0 -> identity matrix (diagonal 0, off-diagonal +INF / -INF)
9  //   k >= 1 -> multiply W by itself k times in the (min/max, +) tropical semiring
10 //
11 // Returns {minMatrix, maxMatrix}.
12 std::pair<Matrix, Matrix> shimbell(const Matrix& W, int k);
13
14 // Pretty-print a matrix. Values >= INF_THRESHOLD are shown as "INF",
15 // values <= -INF_THRESHOLD are shown as "-INF".
16 void printMatrix(const Matrix& M);

```

Листинг 14. Файл shimbell.h

```

1  #include "shimbell.h"
2  // #include "distribution.h" // no longer needed here - moved to shared/weight_matrix.cpp
3  // #include <cmath> // no longer needed here
4  #include <algorithm>
5  #include <iostream>
6  #include <iomanip>
7
8
9  // -- Tropical matrix multiply -----
10
11 static Matrix tropicalMin(const Matrix& A, const Matrix& B) {
12     int n = static_cast<int>(A.size());
13     Matrix C(n, std::vector<double>(n, SHIMBELL_INF));
14     for (int i = 0; i < n; i++)
15         for (int k = 0; k < n; k++) {
16             if (A[i][k] >= SHIMBELL_INF) continue;
17             for (int j = 0; j < n; j++) {
18                 if (B[k][j] >= SHIMBELL_INF) continue;
19                 double v = A[i][k] + B[k][j];
20                 if (v < C[i][j]) C[i][j] = v;
21             }
22         }
23 }

```

```

23     return C;
24 }
25
26 static Matrix tropicalMax(const Matrix& A, const Matrix& B) {
27     int n = static_cast<int>(A.size());
28     Matrix C(n, std::vector<double>(n, SHIMBELL_NINF));
29     for (int i = 0; i < n; i++)
30         for (int k = 0; k < n; k++) {
31             if (A[i][k] <= SHIMBELL_NINF) continue;
32             for (int j = 0; j < n; j++) {
33                 if (B[k][j] <= SHIMBELL_NINF) continue;
34                 double v = A[i][k] + B[k][j];
35                 if (v > C[i][j]) C[i][j] = v;
36             }
37         }
38     return C;
39 }
40
41 // -- Identity matrices -----
42
43 static Matrix identityMin(int n) {
44     Matrix I(n, std::vector<double>(n, SHIMBELL_INF));
45     for (int i = 0; i < n; i++) I[i][i] = 0.0;
46     return I;
47 }
48
49 static Matrix identityMax(int n) {
50     Matrix I(n, std::vector<double>(n, SHIMBELL_NINF));
51     for (int i = 0; i < n; i++) I[i][i] = 0.0;
52     return I;
53 }
54
55 // Build the "max" version of the weight matrix:
56 // edges keep their weight, non-edges → SHIMBELL_NINF, diagonal → SHIMBELL_NINF.
57 // W[i][i] == SHIMBELL_INF, so the condition below correctly skips the diagonal.
58 static Matrix toMaxMatrix(const Matrix& W) {
59     int n = static_cast<int>(W.size());
60     Matrix M(n, std::vector<double>(n, SHIMBELL_NINF));
61     for (int i = 0; i < n; i++)
62         for (int j = 0; j < n; j++) {
63             if (W[i][j] < SHIMBELL_INF) // real edges only (diagonal is INF → skipped)
64                 M[i][j] = W[i][j];
65         }
66     return M;
67 }
68
69 // -- Public shimbell() -----
70
71 std::pair<Matrix, Matrix> shimbell(const Matrix& W, int k) {
72     int n = static_cast<int>(W.size());
73
74     if (k == 0)
75         return { identityMin(n), identityMax(n) };
76
77     Matrix Wmax = toMaxMatrix(W);
78
79     Matrix minResult = W;
80     Matrix maxResult = Wmax;
81
82     for (int step = 1; step < k; step++) {

```

```

83     minResult = tropicalMin(minResult, W);
84     maxResult = tropicalMax(maxResult, Wmax);
85 }
86
87     return { minResult, maxResult };
88 }
89
90 // -- Print -----
91
92 void printMatrix(const Matrix& M) {
93     int n = static_cast<int>(M.size());
94     const int W = 8;
95
96     std::cout << "      ";
97     for (int j = 0; j < n; j++)
98         std::cout << std::setw(W) << j;
99     std::cout << "\n      ";
100    for (int j = 0; j < n; j++)
101        std::cout << std::string(W, '-');
102    std::cout << "\n";
103
104    for (int i = 0; i < n; i++) {
105        std::cout << std::setw(2) << i << " |";
106        for (int j = 0; j < n; j++) {
107            if (M[i][j] >= SHIMBELL_INF / 2)
108                std::cout << std::setw(W) << "INF";
109            else if (M[i][j] <= SHIMBELL_NINF / 2)
110                std::cout << std::setw(W) << "-INF";
111            else {
112                std::cout << std::setw(W) << static_cast<long long>(M[i][j]);
113            }
114        }
115        std::cout << "\n";
116    }
117 }

```

Листинг 15. Файл shimbell.cpp

```

1 #pragma once
2 #include "graph.h"
3 #include <vector>
4
5 // Returns all simple paths from src to dst as lists of internal (0-based) indices.
6 // If src == dst returns one empty path.
7 std::vector<std::vector<int>> findAllPaths(const Graph& g, int src, int dst);
8
9 // Convenience wrappers
10 bool pathExists(const Graph& g, int src, int dst);
11 int countPaths (const Graph& g, int src, int dst);
12
13 // Pretty-print: "Route 1: { 1 -> 3 -> 5 }" (1-indexed display)
14 void printPaths(const std::vector<std::vector<int>>& paths, int src, int dst);

```

Листинг 16. Файл path_counter.h

```

1 #include "path_counter.h"
2 #include <iostream>
3 #include <iomanip>
4
5 static inline int D(int v) { return v + 1; }
6

```

```

7 // -- DFS that collects full paths -----
8 static void dfs(const Graph& g, int u, int dst,
9               std::vector<bool>& visited,
10              std::vector<int>& current,
11              std::vector<std::vector<int>>& result) {
12     if (u == dst) {
13         result.push_back(current);
14         return;
15     }
16     visited[u] = true;
17     for (int v : g.neighbors(u)) {
18         if (!visited[v]) {
19             current.push_back(v);
20             dfs(g, v, dst, visited, current, result);
21             current.pop_back();
22         }
23     }
24     visited[u] = false;
25 }
26
27 std::vector<std::vector<int>> findAllPaths(const Graph& g, int src, int dst) {
28     std::vector<std::vector<int>> result;
29     if (src < 0 || src >= g.n || dst < 0 || dst >= g.n)
30         return result;
31     if (src == dst) {
32         result.push_back({src});
33         return result;
34     }
35     std::vector<bool> visited(g.n, false);
36     std::vector<int> current = {src};
37     dfs(g, src, dst, visited, current, result);
38     return result;
39 }
40
41 bool pathExists(const Graph& g, int src, int dst) {
42     return !findAllPaths(g, src, dst).empty();
43 }
44
45 int countPaths(const Graph& g, int src, int dst) {
46     return static_cast<int>(findAllPaths(g, src, dst).size());
47 }
48
49 void printPaths(const std::vector<std::vector<int>>& paths, int src, int dst) {
50     std::cout << "\n Path from " << D(src) << " to " << D(dst) << ": ";
51     if (paths.empty()) {
52         std::cout << "DOES NOT EXIST\n";
53         return;
54     }
55     std::cout << "EXISTS (" << paths.size() << " route"
56               << (paths.size() == 1 ? "" : "s") << ")\n\n";
57     std::cout << " Routes:\n";
58     for (int i = 0; i < (int)paths.size(); i++) {
59         std::cout << " Route " << i + 1 << ": { ";
60         for (int j = 0; j < (int)paths[i].size(); j++) {
61             if (j) std::cout << " -> ";
62             std::cout << D(paths[i][j]);
63         }
64         std::cout << " }\n";
65     }
}

```


Листинг 17. Файл path_counter.cpp

Приложение В. Исходный код лабораторной работы №2

```
1 #pragma once
2 #include "graph.h"
3
4 // Performs DFS traversal from 'start'.
5 // Prints traversal order, discovery order, and total iteration count.
6 void dfs(const Graph& g, int start);
```

Листинг 18. Файл dfs.h

```
1 #include "dfs.h"
2 #include <iostream>
3 #include <stack>
4 #include <vector>
5 #include <iomanip>
6
7 // Iterative DFS using an explicit stack.
8 // Counts every "iteration" - each time we pop a vertex and inspect its neighbors.
9 void dfs(const Graph& g, int start) {
10     const std::string sep(60, '-');
11     std::cout << "\n " << sep << "\n";
12     std::cout << "  DFS TRAVERSAL (start = vertex " << start + 1 << ")\n";
13     std::cout << " " << sep << "\n\n";
14
15     std::vector<bool> visited(g.n, false);
16     std::vector<int> order;
17     std::stack<int> stk;
18     long long iterations = 0;
19
20     stk.push(start);
21
22     while (!stk.empty()) {
23         int u = stk.top();
24         stk.pop();
25         iterations++;
26
27         if (visited[u]) continue;
28         visited[u] = true;
29         order.push_back(u);
30         iterations++;
31
32         // Push neighbors in reverse order so smaller-index neighbors are
33         // visited first (matches recursive DFS order).
34         auto nb = g.neighbors(u);
35         for (int i = (int)nb.size() - 1; i >= 0; i--) {
36             iterations++;
37             if (!visited[nb[i]])
38                 stk.push(nb[i]);
39         }
40     }
41
42     // -- Traversal order -----
43     std::cout << "  Traversal order:\n ";
44     for (int i = 0; i < (int)order.size(); i++) {
45         if (i) std::cout << " -> ";
46         std::cout << order[i] + 1;
47     }
```

```

48     std::cout << "\n\n";
49
50     // -- Visited / unreachable -----
51     std::vector<int> unvisited;
52     for (int v = 0; v < g.n; v++)
53         if (!visited[v]) unvisited.push_back(v);
54
55     std::cout << "  Vertices visited    : " << order.size() << " / " << g.n << "\n";
56     if (!unvisited.empty()) {
57         std::cout << "  Unreachable      : ";
58         for (int i = 0; i < (int)unvisited.size(); i++) {
59             if (i) std::cout << ", ";
60             std::cout << unvisited[i] + 1;
61         }
62         std::cout << "\n";
63         std::cout << "  (Graph not fully reachable from vertex " << start + 1 << ")\n";
64     }
65
66     std::cout << "\n  Total iterations    : " << iterations << "\n";
67     std::cout << "\n  " << sep << "\n";
68 }

```

Листинг 19. Файл dfs.cpp

```

1  #pragma once
2  #include "graph.h"
3  #include "weight_matrix.h" // Matrix, WeightMode, generateWeightMatrix (shared)
4  #include <vector>
5
6
7  // Result bundle returned by floydWarshall()
8  struct FloydResult {
9      std::vector<std::vector<double>>> T; // distance matrix
10     std::vector<std::vector<int>>> H; // path matrix (1-based next hop, 0 = no path)
11     long long iterations; // always n^3 (unconditionally counted)
12     bool hasNegCycle;
13     std::vector<int> negCycleVerts; // 0-based indices of vertices with T[j][j]<0
14 };
15
16 // Runs Floyd-Warshall
17 // loop order i,j,k | condition j≠i & T[j][i]≠INF & i≠k & T[i][k]≠INF
18 // & (T[j][k]=INF or T[j][k] > T[j][i]+T[i][k])
19 // iterations counted unconditionally → always n^3
20 FloydResult floydWarshall(const Graph& g,
21                           const std::vector<std::vector<double>>>& W);
22
23 // Prints C, T, H matrices + path reconstruction + iteration count
24 void printFloydResult(const FloydResult& res,
25                      const std::vector<std::vector<double>>>& W,
26                      int n, int src, int dst);

```

Листинг 20. Файл floyd_warshall.h

```

1  #include "floyd_warshall.h"
2  #include "constants.h"
3  #include <iostream>
4  #include <iomanip>
5  // #include <random> // no longer needed here - moved to shared/weight_matrix.cpp
6  #include <vector>
7  #include <algorithm>
8

```

```

9
10 // -----
11 // Print helpers
12 // -----
13 static void printDMatrix(const std::vector<std::vector<double>>& M,
14                          int n, const std::string& title) {
15     const int CW = 8;
16     std::cout << "\n " << title << "\n\n";
17     std::cout << " ";
18     for (int j = 0; j < n; j++) std::cout << std::setw(CW) << j + 1;
19     std::cout << "\n " << std::string(7 + n * CW, '-') << "\n";
20     for (int i = 0; i < n; i++) {
21         std::cout << " " << std::setw(3) << i + 1 << " |";
22         for (int j = 0; j < n; j++) {
23             double v = M[i][j];
24             if (v >= SHIMBELL_INF / 2) std::cout << std::setw(CW) << "INF";
25             else if (v <= SHIMBELL_NINF / 2) std::cout << std::setw(CW) << "-INF";
26             else std::cout << std::setw(CW) << static_cast<long long>(v);
27         }
28         std::cout << "\n";
29     }
30 }
31
32 static void printHMatrix(const std::vector<std::vector<int>>& H,
33                          int n, const std::string& title) {
34     const int CW = 8;
35     std::cout << "\n " << title << "\n\n";
36     std::cout << " ";
37     for (int j = 0; j < n; j++) std::cout << std::setw(CW) << j + 1;
38     std::cout << "\n " << std::string(7 + n * CW, '-') << "\n";
39     for (int i = 0; i < n; i++) {
40         std::cout << " " << std::setw(3) << i + 1 << " |";
41         for (int j = 0; j < n; j++) std::cout << std::setw(CW) << H[i][j];
42         std::cout << "\n";
43     }
44 }
45
46 // -----
47 // Floyd-Warshall
48 //
49 // Loop order: i (intermediate vertex), j (source row), k (destination col)
50 // Condition: j≠i & T[j][i]≠INF & i≠k & T[i][k]≠INF
51 //             & (T[j][k]=INF OR T[j][k] > T[j][i]+T[i][k])
52 // Iterations: counted unconditionally inside innermost loop → always n3
53 // H matrix:   H[i][j] = j+1 (1-based) if direct edge, 0 if none
54 //             updated as H[j][k] := H[j][i] (first hop on new path)
55 // Neg-cycle:  after main loop, if T[j][j] < 0 → no solution
56 // -----
57 FloydResult floydWarshall(const Graph& g,
58                            const std::vector<std::vector<double>>& W) {
59     const int n = g.n;
60
61     // -- Initialize T and H -----
62     std::vector<std::vector<double>> T = W;
63     std::vector<std::vector<int>> H(n, std::vector<int>(n, 0));
64
65     for (int i = 0; i < n; i++)
66         for (int j = 0; j < n; j++)
67             H[i][j] = (W[i][j] < SHIMBELL_INF / 2) ? (j + 1) : 0;
68

```

```

69 // -- Triple loop - order: i, j, k -----
70 long long iterations = 0;
71 for (int i = 0; i < n; i++) {
72     for (int j = 0; j < n; j++) {
73         for (int k = 0; k < n; k++) {
74             iterations++;           // unconditional - always n^3 total
75
76             if (j == i) continue; //same ver
77             if (T[j][i] >= SHIMBELL_INF / 2) continue; //no path
78             if (i == k) continue;
79             if (T[i][k] >= SHIMBELL_INF / 2) continue;
80
81             double via = T[j][i] + T[i][k];
82             if (T[j][k] >= SHIMBELL_INF / 2 || T[j][k] > via) { //gocha
83                 H[j][k] = H[j][i]; // first hop on new shorter path
84                 T[j][k] = via;
85             }
86         }
87     }
88 }
89
90 // -- Negative-cycle detection -----
91 bool hasNegCycle = false;
92 std::vector<int> negCycleVerts;
93 for (int j = 0; j < n; j++) {
94     if (T[j][j] < -1e-9) {
95         hasNegCycle = true;
96         negCycleVerts.push_back(j);
97     }
98 }
99
100 return { T, H, iterations, hasNegCycle, negCycleVerts };
101 }
102
103 // -----
104 // Print full result
105 // -----
106 void printFloydResult(const FloydResult& res,
107                      const std::vector<std::vector<double>>& W,
108                      int n, int src, int dst) {
109     const std::string sep(60, '-');
110
111     std::cout << "\n " << sep << "\n";
112     std::cout << " FLOYD-WARSHALL (src=" << src + 1
113                 << " dst=" << dst + 1 << ") \n";
114     std::cout << " " << sep << "\n";
115
116     printDMatrix(W, n, "C = WEIGHT MATRIX (INF = no edge, 0 = self):");
117     printDMatrix(res.T, n, "T = DISTANCE MATRIX (all-pairs shortest paths):");
118     printHMatrix(res.H, n, "H = PATH MATRIX (H[i][j] = first next hop, 0 = no path):");
119
120     // Negative cycle
121     if (res.hasNegCycle) {
122         //std::cout << "\n *** NEGATIVE CYCLE DETECTED - no solution *** \n";
123         //std::cout << " Vertices on negative diagonal: ";
124         for (int i = 0; i < (int)res.negCycleVerts.size(); i++) {
125             if (i) std::cout << ", ";
126             std::cout << res.negCycleVerts[i] + 1;
127         }
128         std::cout << "\n";

```

```

129     std::cout << "\n Total iterations: " << res.iterations
130         << " (n=" << n << " where n^3=" << (long long)n*n*n << ")\n";
131     std::cout << "\n " << sep << "\n";
132     return;
133 }
134
135 // Path reconstruction
136 // w := src; yield w
137 // while w != dst do w := H[w][dst]; yield w
138 std::cout << "\n Shortest path from " << src + 1
139     << " to " << dst + 1 << ":\n";
140
141 if (src == dst) {
142     std::cout << " Path: " << src + 1 << " (same vertex, distance = 0)\n";
143 } else if (res.T[src][dst] >= SHIMBELL_INF / 2) {
144     std::cout << " No path exists.\n";
145 } else {
146     std::vector<int> path;
147     int w = src;
148     int guard = n + 2;
149     bool ok = true;
150     path.push_back(w);
151     while (w != dst) {
152         int nxt = res.H[w][dst] - 1; // H is 1-based to convert to 0-based
153         if (nxt < 0 || nxt >= n || --guard < 0) { ok = false; break; }
154         w = nxt;
155         path.push_back(w);
156     }
157     if (!ok) {
158         std::cout << " (reconstruction failed)\n";
159     } else {
160         std::cout << " Path : ";
161         for (int i = 0; i < (int)path.size(); i++) {
162             if (i) std::cout << " -> ";
163             std::cout << path[i] + 1;
164         }
165         std::cout << "\n";
166         std::cout << " Distance: "
167             << static_cast<long long>(res.T[src][dst]) << "\n";
168     }
169 }
170
171 std::cout << "\n Total iterations: " << res.iterations
172     << " (n=" << n << " where n^3=" << (long long)n*n*n << ")\n";
173 std::cout << "\n " << sep << "\n";
174 }

```

Листинг 21. Файл floyd_warshall.cpp

Приложение Г. Исходный код лабораторной работы №3

```
1 #pragma once
2 #include "graph.h"
3 #include <vector>
4 #include <climits>
5
6 // Capacity matrix: cap[i][j] in [1,20] for edge (i,j), else 0
7 std::vector<std::vector<int>> generateCapacityMatrix(const Graph& g);
8
9 // Cost matrix: D[i][j] in [1,10] for edge (i,j), else 0
10 std::vector<std::vector<int>> generateCostMatrix(const Graph& g);
11
12 struct FFResult {
13     std::vector<std::vector<int>> flow; // net flow on each arc
14     int maxFlow;
15     int src, dst;
16 };
17
18 // BFS-based Ford-Fulkerson (Edmonds-Karp), prints each augmenting path
19 FFResult fordFulkerson(int n,
20                         const std::vector<std::vector<int>>& cap,
21                         int src, int dst);
22
23 // Same but silent (no output), used internally by min-cost flow
24 FFResult fordFulkersonSilent(int n,
25                              const std::vector<std::vector<int>>& cap,
26                              int src, int dst);
27
28 void printCapacityMatrix(const std::vector<std::vector<int>>& cap, int n);
29 void printCostMatrix(const std::vector<std::vector<int>>& cost, int n);
30 void printFFResult(const FFResult& res,
31                   const std::vector<std::vector<int>>& cap, int n);
```

Листинг 22. Файл ford_fulkerson.h

```
1 #include "ford_fulkerson.h"
2 #include <iostream>
3 #include <iomanip>
4 #include <random>
5 #include <queue>
6 #include <algorithm>
7 #include <vector>
8
9 static std::mt19937 rng3(std::random_device{}());
10
11 std::vector<std::vector<int>> generateCapacityMatrix(const Graph& g) {
12     const int n = g.n;
13     std::vector<std::vector<int>> cap(n, std::vector<int>(n, 0));
14     std::uniform_int_distribution<int> dist(1, 20);
15     for (int u = 0; u < n; u++)
16         for (int v = 0; v < n; v++)
17             if (g.hasEdge(u, v))
18                 cap[u][v] = dist(rng3);
19     return cap;
20 }
21
22 std::vector<std::vector<int>> generateCostMatrix(const Graph& g) {
```

```

23     const int n = g.n;
24     std::vector<std::vector<int>>> cost(n, std::vector<int>(n, 0));
25     std::uniform_int_distribution<int> dist(1, 10);
26     for (int u = 0; u < n; u++)
27         for (int v = 0; v < n; v++)
28             if (g.hasEdge(u, v))
29                 cost[u][v] = dist(rng3);
30     return cost;
31 }
32
33 // BFS finds an augmenting path; prev[v] = predecessor of v on path
34 static bool bfs(const std::vector<std::vector<int>>& resCap, int n,
35               int src, int dst, std::vector<int>& prev) {
36     prev.assign(n, -1);
37     prev[src] = src;
38     std::queue<int> q;
39     q.push(src);
40     while (!q.empty()) {
41         int u = q.front(); q.pop();
42         for (int v = 0; v < n; v++) {
43             if (prev[v] == -1 && resCap[u][v] > 0) {
44                 prev[v] = u;
45                 if (v == dst) return true;
46                 q.push(v);
47             }
48         }
49     }
50     return false;
51 }
52
53 static FFResult runFF(int n, const std::vector<std::vector<int>>& cap,
54                     int src, int dst, bool verbose) {
55     std::vector<std::vector<int>>> resCap = cap;
56     std::vector<std::vector<int>>> flow(n, std::vector<int>(n, 0));
57     int maxFlow = 0;
58     int pathNum = 0;
59
60     std::vector<int> prev;
61     while (bfs(resCap, n, src, dst, prev)) {
62         // Bottleneck
63         int delta = INT_MAX;
64         for (int v = dst; v != src; v = prev[v])
65             delta = std::min(delta, resCap[prev[v]][v]);
66
67         // Augment flow along path
68         for (int v = dst; v != src; v = prev[v]) {
69             int u = prev[v];
70             resCap[u][v] -= delta;
71             resCap[v][u] += delta;
72             flow[u][v] += delta;
73             flow[v][u] -= delta;
74         }
75         maxFlow += delta;
76         pathNum++;
77
78         if (verbose) {
79             // Reconstruct path for display
80             std::vector<int> path;
81             for (int v = dst; v != src; v = prev[v]) path.push_back(v);
82             path.push_back(src);

```



```

83         std::reverse(path.begin(), path.end());
84         std::cout << "   Path " << pathNum << ": ";
85         for (int i = 0; i < (int)path.size(); i++) {
86             if (i) std::cout << " -> ";
87             std::cout << path[i] + 1;
88         }
89         std::cout << "   (delta = " << delta << ")\n";
90     }
91 }
92 return { flow, maxFlow, src, dst };
93 }
94
95 FFResult fordFulkerson(int n, const std::vector<std::vector<int>>& cap,
96                        int src, int dst) {
97     return runFF(n, cap, src, dst, true);
98 }
99
100 FFResult fordFulkersonSilent(int n, const std::vector<std::vector<int>>& cap,
101                              int src, int dst) {
102     return runFF(n, cap, src, dst, false);
103 }
104
105 static void printIntMatrix(const std::vector<std::vector<int>>& M, int n,
106                           const std::string& title) {
107     const int CW = 6;
108     std::cout << "\n " << title << "\n\n";
109     std::cout << " ";
110     for (int j = 0; j < n; j++) std::cout << std::setw(CW) << j + 1;
111     std::cout << "\n " << std::string(7 + n * CW, '-') << "\n";
112     for (int i = 0; i < n; i++) {
113         std::cout << " " << std::setw(3) << i + 1 << " |";
114         for (int j = 0; j < n; j++)
115             std::cout << std::setw(CW) << M[i][j];
116         std::cout << "\n";
117     }
118 }
119
120 void printCapacityMatrix(const std::vector<std::vector<int>>& cap, int n) {
121     printIntMatrix(cap, n, "CAPACITY MATRIX  Omega[i][j]:");
122 }
123
124 void printCostMatrix(const std::vector<std::vector<int>>& cost, int n) {
125     printIntMatrix(cost, n, "COST MATRIX  D[i][j]:");
126 }
127
128 void printFFResult(const FFResult& res,
129                   const std::vector<std::vector<int>>& cap, int n) {
130     const std::string sep(60, '-');
131     std::cout << "\n " << sep << "\n";
132     std::cout << "   FORD-FULKERSON RESULT\n";
133     std::cout << "   Source: " << res.src + 1 << "   Sink: " << res.dst + 1 << "\n";
134     std::cout << "   Maximum flow  phi_max = " << res.maxFlow << "\n";
135     std::cout << "   " << sep << "\n";
136     printCapacityMatrix(cap, n);
137
138     // Print only arcs that carry positive flow
139     std::cout << "\n   FLOW on each arc (only arcs with flow > 0):\n";
140     bool any = false;
141     for (int i = 0; i < n; i++)
142         for (int j = 0; j < n; j++)

```

```

143         if (res.flow[i][j] > 0) {
144             std::cout << "      (" << i + 1 << " -> " << j + 1 << ")  "
145                 << "flow=" << res.flow[i][j]
146                 << "   cap=" << cap[i][j] << "\n";
147             any = true;
148         }
149     if (!any) std::cout << "      (none)\n";
150     std::cout << "\n  " << sep << "\n";
151 }

```

Листинг 23. Файл ford_fulkerson.cpp

```

1  #pragma once
2  #include "ford_fulkerson.h"
3  #include <vector>
4
5  struct MCFResult {
6      std::vector<std::vector<int>>> flow; // net flow on each arc
7      int theta; // target flow = floor(2/3 * maxFlow)
8      int totalFlow; // flow achieved (== theta if feasible)
9      long long totalCost;
10     int maxFlow;
11 };
12
13 // Successive shortest paths (Bellman-Ford) to find min-cost flow of theta units
14 // theta = floor(2/3 * maxFlow)
15 MCFResult minCostFlow(int n,
16     const std::vector<std::vector<int>>>& cap,
17     const std::vector<std::vector<int>>>& cost,
18     int src, int dst);
19
20 void printMCFResult(const MCFResult& res,
21     const std::vector<std::vector<int>>>& cap,
22     const std::vector<std::vector<int>>>& cost, int n);

```

Листинг 24. Файл min_cost_flow.h

```

1  #include "min_cost_flow.h"
2  #include <iostream>
3  #include <iomanip>
4  #include <vector>
5  #include <algorithm>
6  #include <climits>
7
8  // -----
9  // Floyd-Warshall on the residual cost graph.
10 //
11 // Used at every iteration of the successive-shortest-paths algorithm
12 // (lab task: "use the previously implemented Floyd-Warshall").
13 //
14 // Builds T (distance) and H (next-hop, 1-based) matrices from:
15 //   resCap[u][v] > 0 → arc exists with cost resCost[u][v]
16 //   resCap[u][v] == 0 → arc absent
17 //
18 // Loop order: i (intermediate), j (source row), k (destination col).
19 // H[j][k] = first hop from j toward k (1-based vertex index).
20 // -----
21 struct FWCostResult {
22     std::vector<std::vector<int>>> T; // shortest cost distances
23     std::vector<std::vector<int>>> H; // next-hop matrix (1-based, 0 = no path)
24 };

```

```

25
26 static const int FW_INF = INT_MAX / 2;
27
28 static FWCostResult floydWarshallCost(
29     const std::vector<std::vector<int>>& resCap,
30     const std::vector<std::vector<int>>& resCost,
31     int n) {
32
33     std::vector<std::vector<int>> T(n, std::vector<int>(n, FW_INF));
34     std::vector<std::vector<int>> H(n, std::vector<int>(n, 0));
35
36     // Initialise: diagonal = 0, edges = cost where capacity > 0
37     for (int i = 0; i < n; i++) {
38         T[i][i] = 0;
39         for (int j = 0; j < n; j++) {
40             if (i != j && resCap[i][j] > 0) {
41                 T[i][j] = resCost[i][j];
42                 H[i][j] = j + 1;          // direct edge → first hop is j
43             }
44         }
45     }
46
47     // Triple loop - same order as Lab 2: i (intermediate), j (row), k (col)
48     for (int i = 0; i < n; i++) {
49         for (int j = 0; j < n; j++) {
50             if (j == i || T[j][i] == FW_INF) continue;
51             for (int k = 0; k < n; k++) {
52                 if (i == k || T[i][k] == FW_INF) continue;
53                 int via = T[j][i] + T[i][k];
54                 if (T[j][k] == FW_INF || T[j][k] > via) {
55                     T[j][k] = via;
56                     H[j][k] = H[j][i];    // first hop toward k is same as toward i
57                 }
58             }
59         }
60     }
61
62     return { T, H };
63 }
64
65 // Reconstruct path from src to dst using Floyd-Warshall next-hop matrix H.
66 // Returns empty vector if no path.
67 static std::vector<int> reconstructPath(
68     const std::vector<std::vector<int>>& H,
69     int src, int dst, int n) {
70
71     if (H[src][dst] == 0) return {};
72     std::vector<int> path;
73     int w = src;
74     path.push_back(w);
75     int guard = n + 2;
76     while (w != dst) {
77         int nxt = H[w][dst] - 1;          // convert 1-based → 0-based
78         if (nxt < 0 || nxt >= n || --guard < 0) return {};
79         w = nxt;
80         path.push_back(w);
81     }
82     return path;
83 }
84

```

```

85 // -----
86 // Min-cost flow - successive shortest paths via Floyd-Warshall
87 //
88 // Algorithm :
89 // 1. phi_max = Ford-Fulkerson(cap, src, dst).
90 //    theta = floor(2/3 * phi_max).
91 // 2. Build residual network (initially same as cap/cost).
92 // 3. While total_flow < theta:
93 //    a. Run Floyd-Warshall on residual cost graph.
94 //    b. Get shortest path src→dst from H matrix.
95 //    c. delta = min(residual caps on path, theta - total_flow).
96 //    d. Augment flow by delta along path.
97 //    e. Update residual network (lecture page 4 rules):
98 //        - Forward arc (u,v): resCap[u][v] -= delta
99 //        - Reverse arc (v,u): resCap[v][u] += delta,
100 //          resCost[v][u] = -d(u,v) (negated original cost)
101 // -----
102 MCFResult minCostFlow(int n,
103                       const std::vector<std::vector<int>>& cap,
104                       const std::vector<std::vector<int>>& cost,
105                       int src, int dst) {
106
107     // Step 1: max flow
108     FFResult ff = fordFulkersonSilent(n, cap, src, dst);
109     int maxFlow = ff.maxFlow;
110     int theta = (2 * maxFlow) / 3;
111
112     std::cout << " phi_max = " << maxFlow
113               << " theta = floor(2/3 * " << maxFlow << ") = " << theta << "\n";
114
115     if (maxFlow == 0) {
116         std::cout << " No path from source to sink - max flow is 0.\n";
117         return { std::vector<std::vector<int>>(n, std::vector<int>(n,0)),
118                 0, 0, OLL, 0 };
119     }
120     if (theta == 0) {
121         std::cout << " theta = 0, nothing to route.\n";
122         return { std::vector<std::vector<int>>(n, std::vector<int>(n,0)),
123                 0, 0, OLL, maxFlow };
124     }
125
126     // Step 2: residual network starts as the original capacity / cost matrices
127     std::vector<std::vector<int>> resCap = cap;
128     std::vector<std::vector<int>> resCost = cost;
129     std::vector<std::vector<int>> flow(n, std::vector<int>(n, 0));
130
131     int totalFlow = 0;
132     long long totalCost = 0;
133     int pathNum = 0;
134
135     // Step 3: successive shortest paths
136     while (totalFlow < theta) {
137
138         // 3a. Floyd-Warshall on current residual cost graph
139         FWCostResult fw = floydWarshallCost(resCap, resCost, n);
140
141         // 3b. shortest path src→dst
142         if (fw.T[src][dst] == FW_INF) break; // no path in residual graph
143         std::vector<int> path = reconstructPath(fw.H, src, dst, n);
144         if (path.empty()) break;

```

```

145
146 // 3c. bottleneck = min(residual cap on path, remaining demand)
147 int delta = theta - totalFlow;
148 for (int i = 0; i + 1 < (int)path.size(); i++)
149     delta = std::min(delta, resCap[path[i]][path[i+1]]);
150
151 // 3d-e. augment and update residual network
152 for (int i = 0; i + 1 < (int)path.size(); i++) {
153     int u = path[i], v = path[i+1];
154     int usedCost = resCost[u][v];
155     resCap[u][v] -= delta;           // forward arc: reduce capacity
156     resCap[v][u] += delta;           // reverse arc: add capacity
157     resCost[v][u] = -usedCost;       // reverse arc cancels traversed edge
158     flow[u][v] += delta;
159     flow[v][u] -= delta;
160 }
161 totalFlow += delta;
162 totalCost += (long long)delta * fw.T[src][dst];
163 pathNum++;
164
165 // Print this step
166 std::cout << " Step " << pathNum << " [Floyd-Warshall]: ";
167 for (int i = 0; i < (int)path.size(); i++) {
168     if (i) std::cout << " -> ";
169     std::cout << path[i] + 1;
170 }
171 std::cout << " delta=" << delta
172           << " path_cost=" << fw.T[src][dst]
173           << " total_flow=" << totalFlow << "\n";
174 }
175
176 return { flow, theta, totalFlow, totalCost, maxFlow };
177 }
178
179 // -----
180 // Print result
181 // -----
182 void printMCFResult(const MCFResult& res,
183                   const std::vector<std::vector<int>>& cap,
184                   const std::vector<std::vector<int>>& cost, int n) {
185     const std::string sep(60, '-');
186     std::cout << "\n " << sep << "\n";
187     std::cout << " MIN-COST FLOW RESULT\n";
188     std::cout << " phi_max      = " << res.maxFlow << "\n";
189     std::cout << " theta (2/3*max)= " << res.theta << "\n";
190     std::cout << " Flow achieved = " << res.totalFlow << "\n";
191     std::cout << " Total cost    = " << res.totalCost << "\n";
192     std::cout << " " << sep << "\n";
193
194     std::cout << "\n FLOW DETAILS (arcs with flow > 0):\n";
195     std::cout << " Arc      flow  cap  cost/unit  cost\n";
196     std::cout << " " << std::string(52, '-') << "\n";
197     long long checkCost = 0;
198     bool any = false;
199     for (int i = 0; i < n; i++)
200         for (int j = 0; j < n; j++)
201             if (res.flow[i][j] > 0) {
202                 long long c = (long long)cost[i][j] * res.flow[i][j];
203                 checkCost += c;
204                 std::cout << " (" << i+1 << "->" << j+1 << ") "

```

```

205         << std::setw(8) << res.flow[i][j]
206         << std::setw(6) << cap[i][j]
207         << std::setw(12) << cost[i][j]
208         << std::setw(8) << c << "\n";
209     any = true;
210 }
211 if (!any) std::cout << "    (none)\n";
212 std::cout << "    " << std::string(52, '-') << "\n";
213 std::cout << "    Total cost = " << checkCost << "\n";
214 std::cout << "\n    " << sep << "\n";
215 }

```

Листинг 25. Файл min_cost_flow.cpp

Приложение Д. Исходный код лабораторной работы №4

```
1 #pragma once
2 #include "graph.h"
3
4 // Kirchhoff's Matrix-Tree Theorem:
5 // Build Laplacian L where L[i][i]=degree(i), L[i][j]=-1 if edge(i,j)
6 // Remove any row k and column k; the determinant of the remaining
7 // (n-1)x(n-1) matrix equals the number of spanning trees.
8 // (Graph treated as undirected regardless of directed flag.)
9 long long countSpanningTrees(const Graph& g);
10
11 void printKirchhoffResult(const Graph& g, long long count);
```

Листинг 26. Файл kirchhoff.h

```
1 #include "kirchhoff.h"
2 #include <iostream>
3 #include <iomanip>
4 #include <vector>
5 #include <cmath>
6
7 // Build the Kirchhoff (Laplacian) matrix treating the graph as undirected.
8 // L[i][j] = -1 if edge {i,j} exists (in either direction), L[i][i] = degree(i).
9 static std::vector<std::vector<double>> buildLaplacian(const Graph& g) {
10     const int n = g.n;
11     std::vector<std::vector<double>> L(n, std::vector<double>(n, 0.0));
12     for (int i = 0; i < n; i++) {
13         for (int j = 0; j < n; j++) {
14             if (i == j) continue;
15             if (g.hasEdge(i, j) || g.hasEdge(j, i)) {
16                 L[i][j] = -1.0;
17                 L[i][i] += 1.0;
18             }
19         }
20     }
21     return L;
22 }
23
24 // Gaussian elimination determinant with partial pivoting.
25 static double gaussianDet(std::vector<std::vector<double>> M) {
26     int n = (int)M.size();
27     double det = 1.0;
28     for (int col = 0; col < n; col++) {
29         int pivot = -1;
30         double best = 0.0;
31         for (int row = col; row < n; row++) {
32             if (std::abs(M[row][col]) > best) {
33                 best = std::abs(M[row][col]);
34                 pivot = row;
35             }
36         }
37         if (pivot == -1 || best < 1e-9) return 0.0;
38         if (pivot != col) {
39             std::swap(M[pivot], M[col]);
40             det = -det;
41         }
42         det *= M[col][col];
```

```

43     double inv = 1.0 / M[col][col];
44     for (int row = col + 1; row < n; row++) {
45         double factor = M[row][col] * inv;
46         for (int k = col; k < n; k++)
47             M[row][k] -= factor * M[col][k];
48     }
49 }
50 return det;
51 }
52
53 long long countSpanningTrees(const Graph& g) {
54     int n = g.n;
55     if (n < 2) return 0;
56
57     auto L = buildLaplacian(g);
58
59     // Cofactor A_{00}: remove row 0 and col 0, compute det of (n-1)x(n-1) submatrix
60     std::vector<std::vector<double>> sub(n - 1, std::vector<double>(n - 1));
61     for (int i = 1; i < n; i++)
62         for (int j = 1; j < n; j++)
63             sub[i - 1][j - 1] = L[i][j];
64
65     double d = gaussianDet(sub);
66     return static_cast<long long>(std::round(d));
67 }
68
69 void printKirchhoffResult(const Graph& g, long long count) {
70     const int n = g.n;
71     const std::string sep(60, '-');
72
73     std::cout << "\n " << sep << "\n";
74     std::cout << "  KIRCHHOFF'S MATRIX-TREE THEOREM\n";
75     std::cout << " " << sep << "\n";
76
77     auto L = buildLaplacian(g);
78
79     const int CW = 5;
80     std::cout << "\n Kirchhoff (Laplacian) matrix B:\n";
81     std::cout << "  B[i][i] = degree(i), B[i][j] = -1 if edge {i,j}, else 0\n\n";
82     std::cout << " ";
83     for (int j = 0; j < n; j++) std::cout << std::setw(CW) << j + 1;
84     std::cout << "\n " << std::string(7 + n * CW, '-') << "\n";
85     for (int i = 0; i < n; i++) {
86         std::cout << " " << std::setw(3) << i + 1 << " |";
87         for (int j = 0; j < n; j++)
88             std::cout << std::setw(CW) << (int)L[i][j];
89         std::cout << "\n";
90     }
91
92     // Print cofactor submatrix (remove row 1, col 1)
93     std::cout << "\n Cofactor A_{11} - delete row 1 and col 1:\n\n";
94     std::cout << " ";
95     for (int j = 1; j < n; j++) std::cout << std::setw(CW) << j + 1;
96     std::cout << "\n " << std::string(7 + (n - 1) * CW, '-') << "\n";
97     for (int i = 1; i < n; i++) {
98         std::cout << " " << std::setw(3) << i + 1 << " |";
99         for (int j = 1; j < n; j++)
100             std::cout << std::setw(CW) << (int)L[i][j];
101         std::cout << "\n";
102     }

```



```

103
104     std::cout << "\n det(A_11) = " << count << "\n";
105     std::cout << "   Number of spanning trees = " << count << "\n";
106     std::cout << "\n " << sep << "\n";
107 }

```

Листинг 27. Файл kirchhoff.cpp

```

1  #pragma once
2  #include "graph.h"
3  #include <vector>
4
5  struct MSTEdge {
6      int u, v;
7      double weight;
8  };
9
10 struct BoruvkaResult {
11     std::vector<MSTEdge> edges; // MST edges (n-1 edges)
12     double totalWeight;
13 };
14
15 // Boruvka's algorithm (lecture tg9.pdf pp.62-66):
16 //   While more than 1 component: for each component find the cheapest
17 //   outgoing edge (safe edge); add all safe edges; merge components.
18 //   Tie-breaking: smallest edge index (u*n+v).
19 //   Graph treated as undirected. Weight matrix W[i][j] gives edge weights.
20 BoruvkaResult boruvka(const Graph& g,
21                       const std::vector<std::vector<double>>& W);
22
23 void printBoruvkaResult(const BoruvkaResult& res);

```

Листинг 28. Файл boruvka.h

```

1  #include "boruvka.h"
2  #include "constants.h"
3  #include <iostream>
4  #include <iomanip>
5  #include <vector>
6  #include <map>
7  #include <numeric>
8  #include <functional>
9
10 BoruvkaResult boruvka(const Graph& g,
11                      const std::vector<std::vector<double>>& W) {
12     const int n = g.n;
13
14     // Local Union-Find
15     std::vector<int> parent(n), rnk(n, 0);
16     std::iota(parent.begin(), parent.end(), 0);
17
18     std::function<int(int)> find = [&](int x) -> int {
19         if (parent[x] != x) parent[x] = find(parent[x]);
20         return parent[x];
21     };
22     auto unite = [&](int a, int b) -> bool {
23         a = find(a); b = find(b);
24         if (a == b) return false;
25         if (rnk[a] < rnk[b]) std::swap(a, b);
26         parent[b] = a;
27         if (rnk[a] == rnk[b]) rnk[a]++;

```

```

28     return true;
29 };
30
31 // Print current component partition
32 auto printComps = [&]() {
33     std::map<int, std::vector<int>> comps;
34     for (int i = 0; i < n; i++) comps[find(i)].push_back(i);
35     std::cout << "   Components:";
36     for (auto& [root, members] : comps) {
37         std::cout << " {";
38         for (int k = 0; k < (int)members.size(); k++) {
39             if (k) std::cout << ",";
40             std::cout << members[k] + 1;
41         }
42         std::cout << "}";
43     }
44     std::cout << "\n";
45 };
46
47 std::vector<MSTEdge> mst;
48 int round = 0;
49
50 while ((int)mst.size() < n - 1) {
51     round++;
52     std::cout << "   --- Round " << round << " ---\n";
53     printComps();
54
55     // For each component: find cheapest outgoing edge
56     std::vector<int> bestU(n, -1), bestV(n, -1);
57     std::vector<double> bestW(n, SHIMBELL_INF);
58
59     for (int u = 0; u < n; u++) {
60         for (int v = u + 1; v < n; v++) {
61             bool edge = g.hasEdge(u, v) || g.hasEdge(v, u);
62             if (!edge) continue;
63             // For undirected, W[u][v] == W[v][u] after the weight fix.
64             // Take whichever direction is valid.
65             double w = (W[u][v] < SHIMBELL_INF / 2) ? W[u][v]
66                     : (W[v][u] < SHIMBELL_INF / 2 ? W[v][u] : SHIMBELL_INF);
67             if (w >= SHIMBELL_INF / 2) continue;
68
69             int cu = find(u), cv = find(v);
70             if (cu == cv) continue;
71
72             // Update best for component of u
73             if (w < bestW[cu] ||
74                 (w == bestW[cu] && u * n + v < bestU[cu] * n + bestV[cu])) {
75                 bestW[cu] = w; bestU[cu] = u; bestV[cu] = v;
76             }
77             // Update best for component of v
78             if (w < bestW[cv] ||
79                 (w == bestW[cv] && u * n + v < bestU[cv] * n + bestV[cv])) {
80                 bestW[cv] = w; bestU[cv] = u; bestV[cv] = v;
81             }
82         }
83     }
84
85     // Add all safe edges
86     bool added = false;
87     for (int c = 0; c < n; c++) {

```

```

88         if (bestU[c] == -1) continue;
89         int u = bestU[c], v = bestV[c];
90         if (unite(u, v)) {
91             double w = bestW[c];
92             mst.push_back({ u, v, w });
93             std::cout << "    Add edge (" << u + 1 << ", " << v + 1
94                 << ") weight=" << w << "\n";
95             added = true;
96         }
97     }
98     if (!added) {
99         std::cout << " Warning: no safe edges found - graph may be disconnected.\n";
100         break;
101     }
102 }
103
104 double total = 0.0;
105 for (auto& e : mst) total += e.weight;
106 return { mst, total };
107 }
108
109 void printBoruvkaResult(const BoruvkaResult& res) {
110     const std::string sep(60, '-');
111     std::cout << "\n " << sep << "\n";
112     std::cout << " BORUVKA MST RESULT\n";
113     std::cout << " " << sep << "\n";
114     std::cout << "\n MST edges:\n";
115     std::cout << "    Arc        weight\n";
116     std::cout << "    " << std::string(22, '-') << "\n";
117     for (auto& e : res.edges)
118         std::cout << "    (" << e.u + 1 << " -> " << e.v + 1
119             << ")    " << e.weight << "\n";
120     std::cout << "\n Total MST weight = " << res.totalWeight << "\n";
121     std::cout << "\n " << sep << "\n";
122 }

```

Листинг 29. Файл boruvka.cpp

```

1  #pragma once
2  #include "graph.h"
3  #include "boruvka.h"
4  #include <vector>
5
6  struct EdgeCoverResult {
7      std::vector<MSTEdge> cover;    // edges in the minimum edge cover
8      int matchingSize;             // size of maximum matching used
9  };
10
11 // Minimum edge cover via:
12 // 1. Find maximum matching M (Edmonds blossom, general undirected graph)
13 // 2. For each unmatched vertex add any incident edge
14 // |min cover| = n - |M| for graphs without isolated vertices
15 //
16 // useMST: if true, work on mstEdges (a tree, which is always bipartite).
17 //         if false, work on the full undirected graph with weight matrix W.
18 EdgeCoverResult minEdgeCover(const Graph& g,
19                             const std::vector<std::vector<double>>& W,
20                             bool useMST,
21                             const std::vector<MSTEdge>& mstEdges);
22

```

```
23 void printEdgeCoverResult(const EdgeCoverResult& res);
```

Листинг 30. Файл edge_cover.h

```
1 #include "edge_cover.h"
2 #include "constants.h"
3 #include <iostream>
4 #include <iomanip>
5 #include <vector>
6 #include <queue>
7 #include <algorithm>
8
9 // Edmonds blossom maximum matching for a general undirected graph.
10 // Minimum edge cover size is  $n - |\text{maximum matching}|$  for graphs without isolated
11 // vertices, so this keeps the full-graph case correct even when it is not a tree.
12 class GeneralMatching {
13 public:
14     GeneralMatching(const std::vector<std::vector<int>>& adj, int n)
15         : adj(adj), n(n), match(n, -1), parent(n), base(n),
16           used(n), blossom(n) {}
17
18     int run(std::vector<int>& outMatch) {
19         int result = 0;
20         for (int root = 0; root < n; root++) {
21             if (match[root] != -1) continue;
22             int endpoint = findAugmentingPath(root);
23             if (endpoint == -1) continue;
24
25             while (endpoint != -1) {
26                 int pv = parent[endpoint];
27                 int next = match[pv];
28                 match[endpoint] = pv;
29                 match[pv] = endpoint;
30                 endpoint = next;
31             }
32             result++;
33         }
34         outMatch = match;
35         return result;
36     }
37
38 private:
39     const std::vector<std::vector<int>>& adj;
40     int n;
41     std::vector<int> match;
42     std::vector<int> parent;
43     std::vector<int> base;
44     std::vector<bool> used;
45     std::vector<bool> blossom;
46     std::queue<int> q;
47
48     int lca(int a, int b) {
49         std::vector<bool> seen(n, false);
50         while (true) {
51             a = base[a];
52             seen[a] = true;
53             if (match[a] == -1) break;
54             a = parent[match[a]];
55         }
56         while (true) {
57             b = base[b];
```

```

58         if (seen[b]) return b;
59         b = parent[match[b]];
60     }
61 }
62
63 void markPath(int v, int b, int child) {
64     while (base[v] != b) {
65         blossom[base[v]] = true;
66         blossom[base[match[v]]] = true;
67         parent[v] = child;
68         child = match[v];
69         v = parent[match[v]];
70     }
71 }
72
73 int findAugmentingPath(int root) {
74     std::fill(used.begin(), used.end(), false);
75     std::fill(parent.begin(), parent.end(), -1);
76     for (int i = 0; i < n; i++) base[i] = i;
77
78     q = std::queue<int>();
79     q.push(root);
80     used[root] = true;
81
82     while (!q.empty()) {
83         int v = q.front();
84         q.pop();
85
86         for (int u = 0; u < n; u++) {
87             if (!adj[v][u] || base[v] == base[u] || match[v] == u) continue;
88
89             if (u == root || (match[u] != -1 && parent[match[u]] != -1)) {
90                 int curBase = lca(v, u);
91                 std::fill(blossom.begin(), blossom.end(), false);
92                 markPath(v, curBase, u);
93                 markPath(u, curBase, v);
94
95                 for (int i = 0; i < n; i++) {
96                     if (!blossom[base[i]]) continue;
97                     base[i] = curBase;
98                     if (!used[i]) {
99                         used[i] = true;
100                         q.push(i);
101                     }
102                 }
103             } else if (parent[u] == -1) {
104                 parent[u] = v;
105                 if (match[u] == -1)
106                     return u;
107
108                 int next = match[u];
109                 used[next] = true;
110                 q.push(next);
111             }
112         }
113     }
114
115     return -1;
116 }
117 };

```

```

118
119 static int maxMatching(const std::vector<std::vector<int>>& adj, int n,
120                       std::vector<int>& match) {
121     return GeneralMatching(adj, n).run(match);
122 }
123
124 EdgeCoverResult minEdgeCover(const Graph& g,
125                             const std::vector<std::vector<double>>& W,
126                             bool useMST,
127                             const std::vector<MSTEdge>& mstEdges) {
128     const int n = g.n;
129
130     // Build undirected adjacency matrix for the chosen graph
131     std::vector<std::vector<int>>> adjM(n, std::vector<int>(n, 0));
132     std::vector<std::vector<double>>> wM(n, std::vector<double>(n, 0.0));
133
134     if (useMST) {
135         for (auto& e : mstEdges) {
136             adjM[e.u][e.v] = adjM[e.v][e.u] = 1;
137             wM[e.u][e.v] = wM[e.v][e.u] = e.weight;
138         }
139     } else {
140         for (int i = 0; i < n; i++)
141             for (int j = i + 1; j < n; j++) {
142                 bool edge = g.hasEdge(i, j) || g.hasEdge(j, i);
143                 if (!edge) continue;
144                 adjM[i][j] = adjM[j][i] = 1;
145                 double w = W[i][j] < SHIMBELL_INF / 2 ? W[i][j]
146                     : (W[j][i] < SHIMBELL_INF / 2 ? W[j][i] : 0.0);
147                 wM[i][j] = wM[j][i] = w;
148             }
149     }
150
151     // Find maximum matching
152     std::vector<int> match;
153     int ms = maxMatching(adjM, n, match);
154
155     // Build edge cover: start with matching edges
156     std::vector<bool> covered(n, false);
157     std::vector<MSTEdge> cover;
158
159     for (int u = 0; u < n; u++) {
160         if (match[u] != -1 && u < match[u]) {
161             cover.push_back({ u, match[u], wM[u][match[u]] });
162             covered[u] = covered[match[u]] = true;
163         }
164     }
165
166     // For each unmatched vertex, add any incident edge
167     for (int u = 0; u < n; u++) {
168         if (covered[u]) continue;
169         bool found = false;
170         for (int v = 0; v < n && !found; v++) {
171             if (adjM[u][v]) {
172                 cover.push_back({ u, v, wM[u][v] });
173                 covered[u] = true;
174                 found = true;
175             }
176         }
177         if (!found)

```

```

178         std::cout << " Warning: vertex " << u + 1
179             << " is isolated - cannot be covered.\n";
180     }
181
182     return { cover, ms };
183 }
184
185 void printEdgeCoverResult(const EdgeCoverResult& res) {
186     const std::string sep(60, '-');
187     std::cout << "\n " << sep << "\n";
188     std::cout << " MINIMUM EDGE COVER\n";
189     std::cout << " " << sep << "\n";
190     std::cout << "\n Maximum matching |M| = " << res.matchingSize << "\n";
191     std::cout << " Min edge cover |C| = " << res.cover.size()
192         << " (= n - |M| when no isolated vertices)\n";
193     std::cout << "\n Cover edges:\n";
194     std::cout << " Arc weight\n";
195     std::cout << " " << std::string(22, '-') << "\n";
196     for (auto& e : res.cover)
197         std::cout << " (" << e.u + 1 << " -- " << e.v + 1
198             << ") " << e.weight << "\n";
199     std::cout << "\n " << sep << "\n";
200 }

```

Листинг 31. Файл edge_cover.cpp

```

1 #pragma once
2 #include "boruvka.h"
3 #include <vector>
4
5 struct PruferResult {
6     std::vector<int> code; // Prüfer sequence (1-based labels, length n-2)
7     std::vector<double> weights; // weights of n-1 edges in removal order
8 };
9
10 // Encode MST (n vertices, n-1 edges) as Prüfer code with weights.
11 // Algorithm: repeatedly remove smallest leaf, record its neighbor and edge weight.
12 PruferResult pruferEncode(const std::vector<MSTEdge>& mstEdges, int n);
13
14 // Decode Prüfer code + weights back to a set of edges.
15 // Returns n-1 edges with assigned weights.
16 std::vector<MSTEdge> pruferDecode(const std::vector<int>& code,
17     const std::vector<double>& weights,
18     int n);
19
20 void printPruferResult(const PruferResult& res, int n);
21 void printDecodedTree(const std::vector<MSTEdge>& edges);

```

Листинг 32. Файл prufer.h

```

1 #include "prufer.h"
2 #include <iostream>
3 #include <iomanip>
4 #include <vector>
5 #include <set>
6 #include <map>
7 #include <algorithm>
8
9 PruferResult pruferEncode(const std::vector<MSTEdge>& mstEdges, int n) {
10     // Build adjacency for the tree: adj[u] = set of (neighbor, weight)
11     std::vector<std::map<int, double>> adj(n);

```

```

12     for (auto& e : mstEdges) {
13         adj[e.u][e.v] = e.weight;
14         adj[e.v][e.u] = e.weight;
15     }
16
17     // Degree array
18     std::vector<int> deg(n, 0);
19     for (int i = 0; i < n; i++) deg[i] = (int)adj[i].size();
20
21     // Min-heap of leaves (vertices with degree 1)
22     std::set<int> leaves;
23     for (int i = 0; i < n; i++)
24         if (deg[i] == 1) leaves.insert(i);
25
26     std::vector<int> code;
27     std::vector<double> weights;
28
29     // n-2 iterations produce the Prüfer code
30     int remaining = n;
31     while (remaining > 2) {
32         int leaf = *leaves.begin();
33         leaves.erase(leaves.begin());
34
35         // The single neighbor of leaf
36         auto it = adj[leaf].begin();
37         int neighbor = it->first;
38         double w = it->second;
39
40         code.push_back(neighbor + 1); // store 1-based
41         weights.push_back(w);
42
43         std::cout << "    Remove leaf " << leaf + 1
44                 << "    neighbor=" << neighbor + 1
45                 << "    weight=" << w << "\n";
46
47         // Remove leaf from tree
48         adj[leaf].clear();
49         adj[neighbor].erase(leaf);
50         deg[leaf] = 0;
51         deg[neighbor]--;
52         if (deg[neighbor] == 1) leaves.insert(neighbor);
53
54         remaining--;
55     }
56
57     // The last edge (between the two remaining vertices)
58     std::vector<int> last;
59     for (int i = 0; i < n; i++)
60         if (!adj[i].empty()) last.push_back(i);
61
62     if (last.size() == 2) {
63         double w = adj[last[0]][last[1]];
64         weights.push_back(w);
65         std::cout << "    Last edge: (" << last[0] + 1 << ", "
66                 << last[1] + 1 << ")    weight=" << w << "\n";
67     }
68
69     return { code, weights };
70 }
71

```



```

72 std::vector<MSTEdge> pruferDecode(const std::vector<int>& code,
73                                   const std::vector<double>& weights,
74                                   int n) {
75     // Degree[v] = 1 + count(v in code), then degree for vertices not in code = 1
76     std::vector<int> deg(n, 1);
77     for (int x : code) deg[x - 1]++; // code is 1-based
78
79     // Min-set of leaves (degree 1)
80     std::set<int> leaves;
81     for (int i = 0; i < n; i++)
82         if (deg[i] == 1) leaves.insert(i);
83
84     std::vector<MSTEdge> edges;
85     int wi = 0;
86
87     for (int i = 0; i < (int)code.size(); i++) {
88         int leaf = *leaves.begin();
89         leaves.erase(leaves.begin());
90         int parent = code[i] - 1; // convert to 0-based
91         double w = weights[wi++];
92
93         edges.push_back({ leaf, parent, w });
94         std::cout << "    Connect " << leaf + 1 << " -- " << parent + 1
95                     << "    weight=" << w << "\n";
96
97         deg[leaf]--;
98         deg[parent]--;
99         if (deg[parent] == 1) leaves.insert(parent);
100     }
101
102     // Connect the two remaining vertices with the last stored weight
103     std::vector<int> rem;
104     for (int i = 0; i < n; i++)
105         if (deg[i] == 1) rem.push_back(i);
106
107     if (rem.size() == 2) {
108         double w = weights[wi];
109         edges.push_back({ rem[0], rem[1], w });
110         std::cout << "    Connect " << rem[0] + 1 << " -- " << rem[1] + 1
111                     << "    weight=" << w << " (final edge)\n";
112     }
113
114     return edges;
115 }
116
117 void printPruferResult(const PruferResult& res, int n) {
118     const std::string sep(60, '-');
119     std::cout << "\n " << sep << "\n";
120     std::cout << " PRUFER CODE (n=" << n << ", code length=" << res.code.size() << ")\n";
121     std::cout << " " << sep << "\n";
122     // std::cout << "\n Prufer sequence: [ ";
123     // for (int i = 0; i < (int)res.code.size(); i++) {
124     //     if (i) std::cout << ", ";
125     //     std::cout << res.code[i];
126     // }
127     // std::cout << " ]\n";
128     std::cout << "\n Prufer sequence: [ ";
129     if (res.code.empty())
130     {
131         std::cout << "0";

```

```

132     } else
133     {
134         for (int i = 0; i < (int)res.code.size(); i++ )
135         {
136             if (i) std::cout << ", ";
137             std::cout << res.code[i];
138         }
139     }
140     std::cout << " ]\n";
141     std::cout << "\n Edge weights (in removal order, length " << res.weights.size() << "):\n [ ";
142     for (int i = 0; i < (int)res.weights.size(); i++) {
143         if (i) std::cout << ", ";
144         std::cout << res.weights[i];
145     }
146     std::cout << " ]\n";
147     std::cout << " (last weight is for the final edge not in Prufer sequence)\n";
148     std::cout << "\n " << sep << "\n";
149 }
150
151 void printDecodedTree(const std::vector<MSTEdge>& edges) {
152     const std::string sep(60, '-');
153     std::cout << "\n " << sep << "\n";
154     std::cout << " DECODED TREE EDGES\n";
155     std::cout << " " << sep << "\n";
156     double total = 0.0;
157     for (auto& e : edges) {
158         std::cout << " (" << e.u + 1 << ", " << e.v + 1
159             << ") weight=" << e.weight << "\n";
160         total += e.weight;
161     }
162     std::cout << "\n Total weight = " << total << "\n";
163     std::cout << "\n " << sep << "\n";
164 }

```

Листинг 33. Файл prufer.cpp

Приложение Е. Исходный код лабораторной работы №5

```
1 #pragma once
2 #include "graph.h"
3 #include <string>
4 #include <utility>
5 #include <vector>
6
7 struct EulerianResult {
8     bool originalConnected;
9     bool originalEulerian;
10    bool modificationPossible;
11    std::vector<int> originalDegrees;
12    std::vector<int> originalOddVertices;
13    std::vector<int> modifiedDegrees;
14    std::vector<std::pair<int, int>> addedEdges;
15    std::vector<std::pair<int, int>> removedEdges;
16    std::vector<std::string> log;
17    std::vector<int> eulerCycle;
18 };
19
20 // Checks the undirected graph, modifies only edges of a local simple graph
21 // when needed, and constructs an Euler cycle by Hierholzer's algorithm.
22 // The input Graph is not mutated.
23 EulerianResult buildEulerianCycle(const Graph& g);
24
25 void printEulerianResult(const EulerianResult& res);
```

Листинг 34. Файл eulerian.h

```
1 #include "eulerian.h"
2 #include <algorithm>
3 #include <iomanip>
4 #include <iostream>
5 #include <queue>
6 #include <sstream>
7
8 static inline int D(int v) { return v + 1; }
9
10 static int degreeOf(const std::vector<std::vector<int>>& adj, int v) {
11     int deg = 0;
12     for (int u = 0; u < (int)adj.size(); u++)
13         deg += adj[v][u];
14     return deg;
15 }
16
17 static std::vector<int> degreesOf(const std::vector<std::vector<int>>& adj) {
18     std::vector<int> deg(adj.size(), 0);
19     for (int v = 0; v < (int)adj.size(); v++)
20         deg[v] = degreeOf(adj, v);
21     return deg;
22 }
23
24 static std::vector<int> oddVertices(const std::vector<int>& deg) {
25     std::vector<int> odd;
26     for (int i = 0; i < (int)deg.size(); i++)
27         if (deg[i] % 2 != 0)
28             odd.push_back(i);
```

```

29     return odd;
30 }
31
32 static bool hasAnyEdge(const std::vector<std::vector<int>>& adj) {
33     for (int i = 0; i < (int)adj.size(); i++)
34         for (int j = i + 1; j < (int)adj.size(); j++)
35             if (adj[i][j] > 0)
36                 return true;
37     return false;
38 }
39
40 static std::vector<int> bfsReachable(const std::vector<std::vector<int>>& adj,
41                                     int start) {
42     const int n = (int)adj.size();
43     std::vector<int> visited(n, 0);
44     std::queue<int> q;
45     visited[start] = 1;
46     q.push(start);
47
48     while (!q.empty()) {
49         int v = q.front();
50         q.pop();
51         for (int u = 0; u < n; u++) {
52             if (adj[v][u] > 0 && !visited[u]) {
53                 visited[u] = 1;
54                 q.push(u);
55             }
56         }
57     }
58
59     return visited;
60 }
61
62 static std::vector<std::vector<int>> connectedComponents(
63     const std::vector<std::vector<int>>& adj) {
64     const int n = (int)adj.size();
65     std::vector<int> used(n, 0);
66     std::vector<std::vector<int>> comps;
67
68     for (int start = 0; start < n; start++) {
69         if (used[start]) continue;
70
71         std::vector<int> comp;
72         std::queue<int> q;
73         q.push(start);
74         used[start] = 1;
75
76         while (!q.empty()) {
77             int v = q.front();
78             q.pop();
79             comp.push_back(v);
80             for (int u = 0; u < n; u++) {
81                 if (adj[v][u] > 0 && !used[u]) {
82                     used[u] = 1;
83                     q.push(u);
84                 }
85             }
86         }
87         comps.push_back(comp);
88     }

```

```

89
90     return comps;
91 }
92
93 static bool isConnected(const std::vector<std::vector<int>>& adj) {
94     const int n = (int)adj.size();
95     if (n <= 1) return true;
96     auto seen = bfsReachable(adj, 0);
97     return std::all_of(seen.begin(), seen.end(), [](int x) { return x != 0; });
98 }
99
100 static void addSimpleEdge(std::vector<std::vector<int>>& adj, int u, int v) {
101     adj[u][v] = 1;
102     adj[v][u] = 1;
103 }
104
105 static void removeSimpleEdge(std::vector<std::vector<int>>& adj, int u, int v) {
106     adj[u][v] = 0;
107     adj[v][u] = 0;
108 }
109
110 static bool canRemoveWithoutDisconnecting(
111     const std::vector<std::vector<int>>& adj, int u, int v) {
112     if (adj[u][v] == 0) return false;
113     auto test = adj;
114     removeSimpleEdge(test, u, v);
115     return isConnected(test);
116 }
117
118 static bool tryEdgeSwapForOddPair(
119     std::vector<std::vector<int>>& adj,
120     int u,
121     int v,
122     EulerianResult& res) {
123     const int n = (int)adj.size();
124
125     for (int x = 0; x < n; x++) {
126         if (x == u || x == v) continue;
127
128         if (adj[u][x] > 0 && adj[v][x] == 0) {
129             addSimpleEdge(adj, v, x);
130             if (canRemoveWithoutDisconnecting(adj, u, x)) {
131                 removeSimpleEdge(adj, u, x);
132                 res.addedEdges.push_back({v, x});
133                 res.removedEdges.push_back({u, x});
134
135                 std::ostringstream add;
136                 add << "Added missing edge (" << D(v) << " -- " << D(x)
137                     << ").";
138                 res.log.push_back(add.str());
139
140                 std::ostringstream rem;
141                 rem << "Removed edge (" << D(u) << " -- " << D(x)
142                     << "); graph remains connected.";
143                 res.log.push_back(rem.str());
144                 return true;
145             }
146             removeSimpleEdge(adj, v, x);
147         }
148     }

```

```

149     if (adj[v][x] > 0 && adj[u][x] == 0) {
150         addSimpleEdge(adj, u, x);
151         if (canRemoveWithoutDisconnecting(adj, v, x)) {
152             removeSimpleEdge(adj, v, x);
153             res.addedEdges.push_back({u, x});
154             res.removedEdges.push_back({v, x});
155
156             std::ostringstream add;
157             add << "Added missing edge (" << D(u) << " -- " << D(x)
158                 << ").";
159             res.log.push_back(add.str());
160
161             std::ostringstream rem;
162             rem << "Removed edge (" << D(v) << " -- " << D(x)
163                 << "); graph remains connected.";
164             res.log.push_back(rem.str());
165             return true;
166         }
167         removeSimpleEdge(adj, u, x);
168     }
169 }
170
171 return false;
172 }
173
174 static std::vector<int> hierholzer(std::vector<std::vector<int>> adj) {
175     const int n = (int)adj.size();
176     int start = 0;
177     for (int i = 0; i < n; i++) {
178         if (degreeOf(adj, i) > 0) {
179             start = i;
180             break;
181         }
182     }
183
184     std::vector<int> stack;
185     std::vector<int> cycle;
186     stack.push_back(start);
187
188     while (!stack.empty()) {
189         int v = stack.back();
190         int next = -1;
191         for (int u = 0; u < n; u++) {
192             if (adj[v][u] > 0) {
193                 next = u;
194                 break;
195             }
196         }
197
198         if (next == -1) {
199             cycle.push_back(v);
200             stack.pop_back();
201         } else {
202             adj[v][next] = 0;
203             adj[next][v] = 0;
204             stack.push_back(next);
205         }
206     }
207
208     std::reverse(cycle.begin(), cycle.end());

```

```

209     return cycle;
210 }
211
212 EulerianResult buildEulerianCycle(const Graph& g) {
213     const int n = g.n;
214     std::vector<std::vector<int>> adj(n, std::vector<int>(n, 0));
215
216     for (int i = 0; i < n; i++)
217         for (int j = i + 1; j < n; j++)
218             if (g.hasEdge(i, j) || g.hasEdge(j, i))
219                 addSimpleEdge(adj, i, j);
220
221     EulerianResult res;
222     res.originalConnected = isConnected(adj);
223     res.originalDegrees = degreesOf(adj);
224     res.originalOddVertices = oddVertices(res.originalDegrees);
225     res.originalEulerian = res.originalConnected &&
226                           res.originalOddVertices.empty() &&
227                           hasAnyEdge(adj);
228     res.modificationPossible = true;
229
230     if (!hasAnyEdge(adj)) {
231         res.log.push_back("Graph has no edges; Euler cycle is not constructed.");
232         res.modificationPossible = false;
233         res.modifiedDegrees = res.originalDegrees;
234         return res;
235     }
236
237     if (!res.originalConnected) {
238         auto comps = connectedComponents(adj);
239         std::ostringstream os;
240         os << "Graph is disconnected: " << comps.size()
241            << " connected components found.";
242         res.log.push_back(os.str());
243
244         for (int i = 0; i + 1 < (int)comps.size(); i++) {
245             int u = comps[i][0];
246             int v = comps[i + 1][0];
247             addSimpleEdge(adj, u, v);
248             res.addedEdges.push_back({u, v});
249             std::ostringstream add;
250             add << "Added edge (" << D(u) << " -- " << D(v)
251                << ") to connect components.";
252             res.log.push_back(add.str());
253         }
254     }
255
256     auto currentDegrees = degreesOf(adj);
257     auto odd = oddVertices(currentDegrees);
258     if (odd.empty()) {
259         res.log.push_back("All vertex degrees are even; no parity modification needed.");
260     } else {
261         const int maxSteps = n * n + 1;
262         for (int step = 0; step < maxSteps; step++) {
263             currentDegrees = degreesOf(adj);
264             odd = oddVertices(currentDegrees);
265             if (odd.empty()) break;
266
267             std::ostringstream os;
268             os << "Odd vertices:";

```

```

269     for (int v : odd) os << " " << D(v);
270     res.log.push_back(os.str());
271
272     if (n < 3) {
273         res.log.push_back("Cannot modify this graph under restrictions: "
274             "need at least 3 vertices to avoid duplicate edges "
275             "and keep the graph connected.");
276         res.modificationPossible = false;
277         break;
278     }
279
280     bool changed = false;
281
282     for (int i = 0; i < (int)odd.size() && !changed; i++) {
283         for (int j = i + 1; j < (int)odd.size() && !changed; j++) {
284             int u = odd[i];
285             int v = odd[j];
286             if (adj[u][v] == 0) {
287                 addSimpleEdge(adj, u, v);
288                 res.addedEdges.push_back({u, v});
289                 std::ostringstream add;
290                 add << "Added missing edge (" << D(u) << " -- "
291                     << D(v) << ").";
292                 res.log.push_back(add.str());
293                 changed = true;
294             }
295         }
296     }
297
298     for (int i = 0; i < (int)odd.size() && !changed; i++) {
299         for (int j = i + 1; j < (int)odd.size() && !changed; j++) {
300             int u = odd[i];
301             int v = odd[j];
302             if (adj[u][v] > 0 && canRemoveWithoutDisconnecting(adj, u, v)) {
303                 removeSimpleEdge(adj, u, v);
304                 res.removedEdges.push_back({u, v});
305                 std::ostringstream rem;
306                 rem << "Removed edge (" << D(u) << " -- "
307                     << D(v) << "); graph remains connected.";
308                 res.log.push_back(rem.str());
309                 changed = true;
310             }
311         }
312     }
313
314     for (int i = 0; i < (int)odd.size() && !changed; i++) {
315         for (int j = i + 1; j < (int)odd.size() && !changed; j++) {
316             int u = odd[i];
317             int v = odd[j];
318             if (adj[u][v] > 0)
319                 changed = tryEdgeSwapForOddPair(adj, u, v, res);
320         }
321     }
322
323     if (!changed) {
324         res.log.push_back("Cannot modify this graph under restrictions: "
325             "no legal add/remove operation was found.");
326         res.modificationPossible = false;
327         break;
328     }

```



```

329     }
330
331     currentDegrees = degreesOf(adj);
332     odd = oddVertices(currentDegrees);
333     if (!odd.empty() && res.modificationPossible) {
334         res.log.push_back("Cannot modify this graph under restrictions: "
335             "the add/remove process did not reach all even degrees.");
336         res.modificationPossible = false;
337     }
338 }
339
340 res.modifiedDegrees = degreesOf(adj);
341 if (res.modificationPossible && isConnected(adj) &&
342     oddVertices(res.modifiedDegrees).empty()) {
343     res.eulerCycle = hierholzer(adj);
344 } else {
345     res.eulerCycle.clear();
346 }
347 return res;
348 }
349
350 void printEulerianResult(const EulerianResult& res) {
351     const std::string sep(60, '-');
352     const int n = (int)res.originalDegrees.size();
353
354     std::cout << "\n " << sep << "\n";
355     std::cout << "  LAB 5: EULERIAN GRAPH CHECK AND EULER CYCLE\n";
356     std::cout << " " << sep << "\n\n";
357
358     std::cout << "  Connected: " << (res.originalConnected ? "YES" : "NO") << "\n";
359     std::cout << "  Original graph is Eulerian: "
360         << (res.originalEulerian ? "YES" : "NO") << "\n\n";
361
362     std::cout << "  Original degrees:\n";
363     std::cout << "    Vertex | Degree | Parity\n";
364     std::cout << "    " << std::string(26, '-') << "\n";
365     for (int i = 0; i < n; i++) {
366         std::cout << "    " << std::setw(3) << D(i)
367             << "    | " << std::setw(3) << res.originalDegrees[i]
368             << "    | " << (res.originalDegrees[i] % 2 ? "odd" : "even")
369             << "\n";
370     }
371
372     std::cout << "\n  Modification log:\n";
373     for (const auto& line : res.log)
374         std::cout << "    - " << line << "\n";
375
376     if (!res.modificationPossible) {
377         std::cout << "\n " << sep << "\n";
378         return;
379     }
380
381     std::cout << "\n  Modified degrees:\n";
382     for (int i = 0; i < (int)res.modifiedDegrees.size(); i++) {
383         std::cout << "    deg(" << D(i) << ") = " << res.modifiedDegrees[i]
384             << "    " << (res.modifiedDegrees[i] % 2 ? "odd" : "even")
385             << "\n";
386     }
387
388     std::cout << "\n  Euler cycle:\n  ";

```

```

389     if (res.eulerCycle.empty()) {
390         std::cout << "Not constructed.";
391     } else {
392         for (int i = 0; i < (int)res.eulerCycle.size(); i++) {
393             if (i) std::cout << " -> ";
394             std::cout << D(res.eulerCycle[i]);
395         }
396     }
397     std::cout << "\n\n" << sep << "\n";
398 }

```

Листинг 35. Файл eulerian.cpp

```

1  #pragma once
2  #include "boruvka.h"
3  #include "graph.h"
4  #include <vector>
5
6  struct CutsetEdge {
7      int u, v;
8  };
9
10 struct FundamentalCutset {
11     int index;
12     MSTEdge treeEdge;
13     std::vector<CutsetEdge> edges;
14 };
15
16 std::vector<FundamentalCutset> buildFundamentalCutsets(
17     const Graph& g,
18     const std::vector<MSTEdge>& mstEdges);
19
20 void printFundamentalCutsets(const std::vector<FundamentalCutset>& cutsets);
21
22 void printCutsetSymmetricDifference(
23     const std::vector<FundamentalCutset>& cutsets,
24     const std::vector<int>& selectedIndices);

```

Листинг 36. Файл fundamental_cutsets.h

```

1  #include "fundamental_cutsets.h"
2  #include <algorithm>
3  #include <iomanip>
4  #include <iostream>
5  #include <queue>
6  #include <set>
7
8  static inline int D(int v) { return v + 1; }
9
10 static std::vector<CutsetEdge> graphEdges(const Graph& g) {
11     std::vector<CutsetEdge> edges;
12     for (int i = 0; i < g.n; i++)
13         for (int j = i + 1; j < g.n; j++)
14             if (g.hasEdge(i, j) || g.hasEdge(j, i))
15                 edges.push_back({i, j});
16     return edges;
17 }
18
19 static std::vector<int> treeComponentAfterRemoving(
20     int n,
21     const std::vector<MSTEdge>& tree,

```

```

22     int removedIndex,
23     int start) {
24     std::vector<std::vector<int>>> adj(n);
25     for (int i = 0; i < (int)tree.size(); i++) {
26         if (i == removedIndex) continue;
27         int u = tree[i].u;
28         int v = tree[i].v;
29         adj[u].push_back(v);
30         adj[v].push_back(u);
31     }
32
33     std::vector<int> inComponent(n, 0);
34     std::queue<int> q;
35     q.push(start);
36     inComponent[start] = 1;
37
38     while (!q.empty()) {
39         int v = q.front();
40         q.pop();
41         for (int u : adj[v]) {
42             if (!inComponent[u]) {
43                 inComponent[u] = 1;
44                 q.push(u);
45             }
46         }
47     }
48
49     return inComponent;
50 }
51
52 std::vector<FundamentalCutset> buildFundamentalCutsets(
53     const Graph& g,
54     const std::vector<MSTEdge>& mstEdges) {
55     std::vector<FundamentalCutset> result;
56     auto edges = graphEdges(g);
57
58     for (int i = 0; i < (int)mstEdges.size(); i++) {
59         const auto& treeEdge = mstEdges[i];
60         auto left = treeComponentAfterRemoving(g.n, mstEdges, i, treeEdge.u);
61
62         FundamentalCutset cut;
63         cut.index = i + 1;
64         cut.treeEdge = treeEdge;
65
66         for (auto e : edges) {
67             if (left[e.u] != left[e.v])
68                 cut.edges.push_back(e);
69         }
70
71         std::sort(cut.edges.begin(), cut.edges.end(),
72             [](const CutsetEdge& a, const CutsetEdge& b) {
73                 if (a.u != b.u) return a.u < b.u;
74                 return a.v < b.v;
75             });
76
77         result.push_back(cut);
78     }
79
80     return result;
81 }

```

```

82
83 void printFundamentalCutsets(const std::vector<FundamentalCutset>& cutsets) {
84     const std::string sep(60, '-');
85     std::cout << "\n " << sep << "\n";
86     std::cout << "    LAB 5: FUNDAMENTAL CUTSET SYSTEM (EVEN VARIANT)\n";
87     std::cout << " " << sep << "\n\n";
88
89     if (cutsets.empty()) {
90         std::cout << "    No fundamental cutsets exist.\n";
91         std::cout << "    Reason: a fundamental cutset is built for each MST edge,\n";
92         std::cout << "    and this graph has no MST edges.\n";
93         std::cout << "\n " << sep << "\n";
94         return;
95     }
96
97     for (const auto& cut : cutsets) {
98         std::cout << "    Cutset " << cut.index
99             << "    for tree edge (" << D(cut.treeEdge.u)
100             << " -- " << D(cut.treeEdge.v) << "):\n";
101         std::cout << "        { ";
102         for (int i = 0; i < (int)cut.edges.size(); i++) {
103             if (i) std::cout << ", ";
104             std::cout << "(" << D(cut.edges[i].u)
105                 << "--" << D(cut.edges[i].v) << " ";
106         }
107         std::cout << " }\n";
108     }
109
110     std::cout << "\n " << sep << "\n";
111 }
112
113 void printCutsetSymmetricDifference(
114     const std::vector<FundamentalCutset>& cutsets,
115     const std::vector<int>& selectedIndices) {
116     std::set<std::pair<int, int>> result;
117
118     for (int idx : selectedIndices) {
119         if (idx < 1 || idx > (int)cutsets.size()) continue;
120         for (auto e : cutsets[idx - 1].edges) {
121             auto key = std::make_pair(e.u, e.v);
122             auto it = result.find(key);
123             if (it == result.end())
124                 result.insert(key);
125             else
126                 result.erase(it);
127         }
128     }
129
130     std::cout << "\n Symmetric difference of selected cutsets";
131     if (!selectedIndices.empty()) {
132         std::cout << " (";
133         for (int i = 0; i < (int)selectedIndices.size(); i++) {
134             if (i) std::cout << ", ";
135             std::cout << selectedIndices[i];
136         }
137         std::cout << ")";
138     }
139     std::cout << ":\n";
140
141     std::cout << "    { ";

```

```
142     int printed = 0;
143     for (auto [u, v] : result) {
144         if (printed++) std::cout << ", ";
145         std::cout << "(" << D(u) << "--" << D(v) << ")";
146     }
147     std::cout << " }\n";
148 }
```

Листинг 37. Файл fundamental_cutsets.cpp